



**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**

**Khoa Công nghệ thông tin 1**

# **LẬP TRÌNH WEB: PHP và MySQL**

**TS. Nguyễn Quang Hưng**



## Mục tiêu của học phần

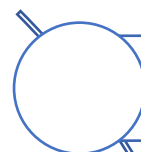
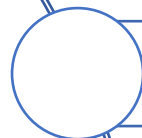
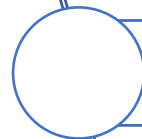
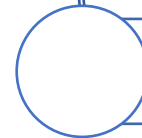
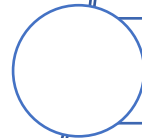
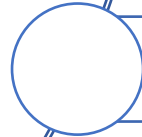
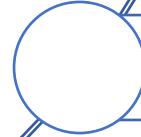
---

- Trình bày tổng quan về Web Server và cách thức hoạt động của nó.
- Cung cấp các kiến thức cơ bản về ngôn ngữ PHP và CSDL MySQL như: cách cài đặt, câu lệnh, cú pháp, các hàm thông dụng trong việc lập trình Web động
- Vận dụng các kiến thức được học vào việc thiết kế và xây dựng các ứng dụng Web hoạt động trên mô hình Client/Server



# Tài liệu tham khảo

- ***Giáo trình nhập môn PHP & MySQL xây dựng ứng dụng Web***, Nguyễn Thiên Bằng – Nhà Xuất bản Lao động xã hội
- ***Sử dụng PHP & MySQL Thiết kế Web động***, Nguyễn Trường Sinh – Nhà xuất bản thống kê
- ***Xây dựng ứng dụng Web bằng PHP & MySQL***, Phạm Hữu Khang
- ***Beginning PHP and MySQL***, W. Jason Gilmore – Third edition
- ***Advanced PHP for Web Professionals***, Christopher Cosentino – Prentice Hall PTR (2002)

-  **Chương 1: Giới thiệu HTML form và mô hình client/ server và web server**
-  **Chương 2: Tổng quan về ngôn ngữ PHP**
-  **Chương 3: Mảng và chuỗi ký tự**
-  **Chương 4: Hàm**
-  **Chương 5: Session và cookie**
-  **Chương 6: Tổng quan về MySQL**
-  **Chương 7: Kết nối CSDL MySQL trong PHP**



HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

# Chương 1: Giới thiệu HTML form và web server



# Chương 1: giới thiệu html form và web server

---

**1.1. Form và các thuộc tính**

**1.2. Các kiểu phần tử trên Form**

**1.3. Mô hình Client/ Server và Web Server**



## 1.1. Form và các thuộc tính

---

- Form là nơi để người dùng nhập thông tin
- Mỗi Form được bao bởi cặp thẻ `<form>` `</form>`
- Các thuộc tính của form
  - Action = “URL”: chỉ đến script xử lý form
  - Method = “GET” hoặc “POST”: mặc định là GET
  - Name: thuộc tính tên
  - Enctype = “Mime\_type”: loại dữ liệu sẽ gửi đi



## 1.2. Các kiểu phần tử trên Form

---

- **Phần tử Input**
- **Phần tử Select**
- **Phần tử Textarea**
- **Phần tử Button**



## Phần tử Input

---

- Hầu hết công việc trên Forms thường được hoàn tất với các phần tử nhập (Input).
- Một thẻ Input và thuộc tính của nó sẽ quyết định loại phần tử Form nào được hiển thị trên trình duyệt
- Cú pháp:

**<Input type=“chọn loại phần tử” các thuộc tính cần thiết: name, value, ...>**

**Ví dụ:** Xét đoạn Script sau và xem kết quả hiển thị trên trình duyệt

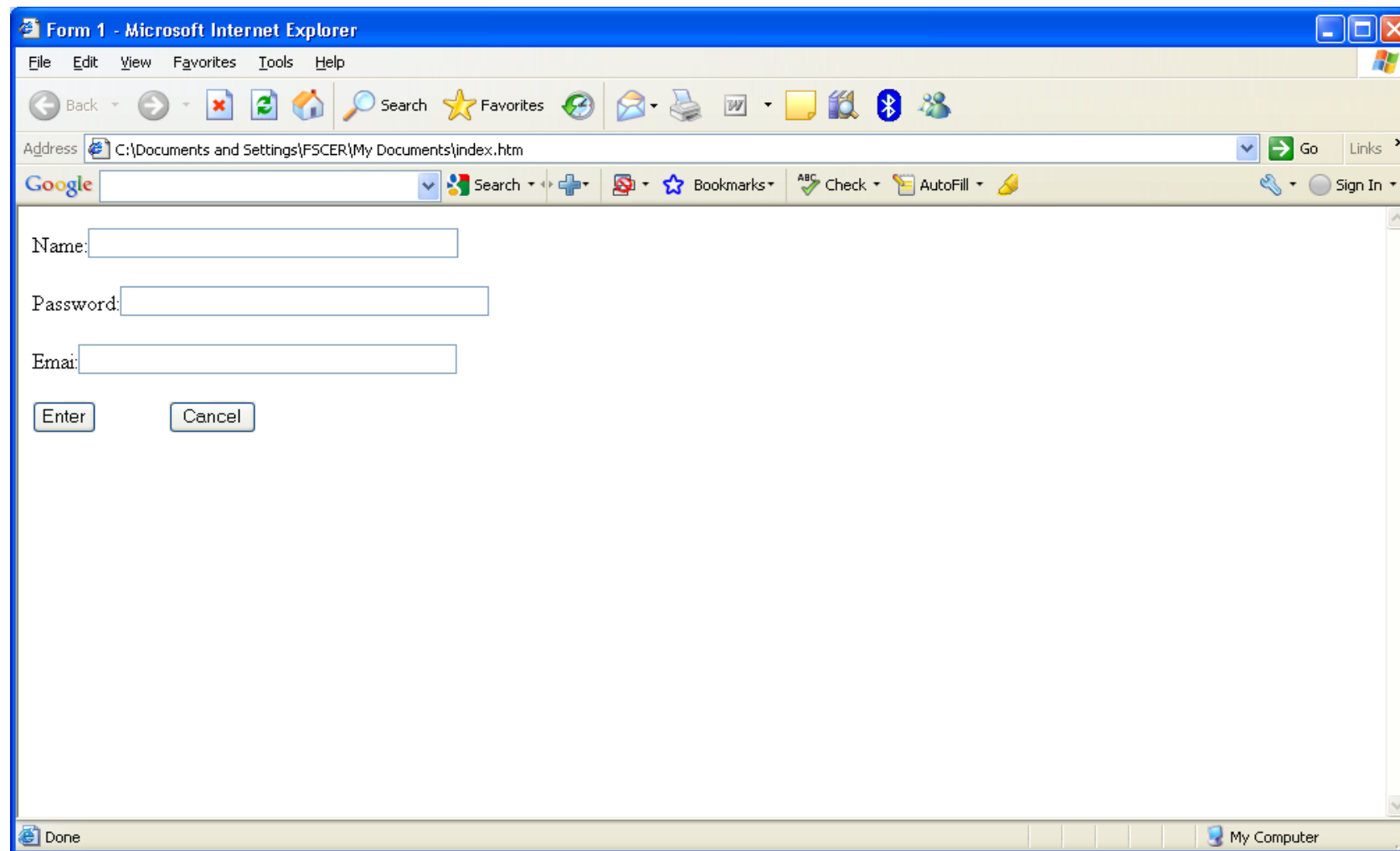


## Phần tử input

[illegible]

# Phần tử input

## Hiển thị trên Browser





## Phần tử input

---

- Một số loại (type) phần tử input:
  - Text
  - Password
  - Hidden
  - Submit
  - Reset
  - Checkbox
  - Radio
  - File
  - Image

## Phần tử Select

---

- Phần tử Select dùng để tạo một danh sách liệt kê.
- Được đặt trong cặp thẻ `<Select>` `</Select>` với một thuộc tính tên
- Trong phần tử này, các thẻ `<Option>` sẽ chỉ ra các giá trị chọn lựa. Mỗi giá trị có một thuộc tính giá trị
- Ta có thể chọn nhiều hơn một giá trị bằng cách thêm vào thẻ `<Select>` thuộc tính “multiple”
- Cú pháp:  
`<select` các thuộc tính: size, name, multiple, ...  
`<option value=số thứ tự hiển thị>` giá trị `</option>`  
`</select>`

## Phần tử select

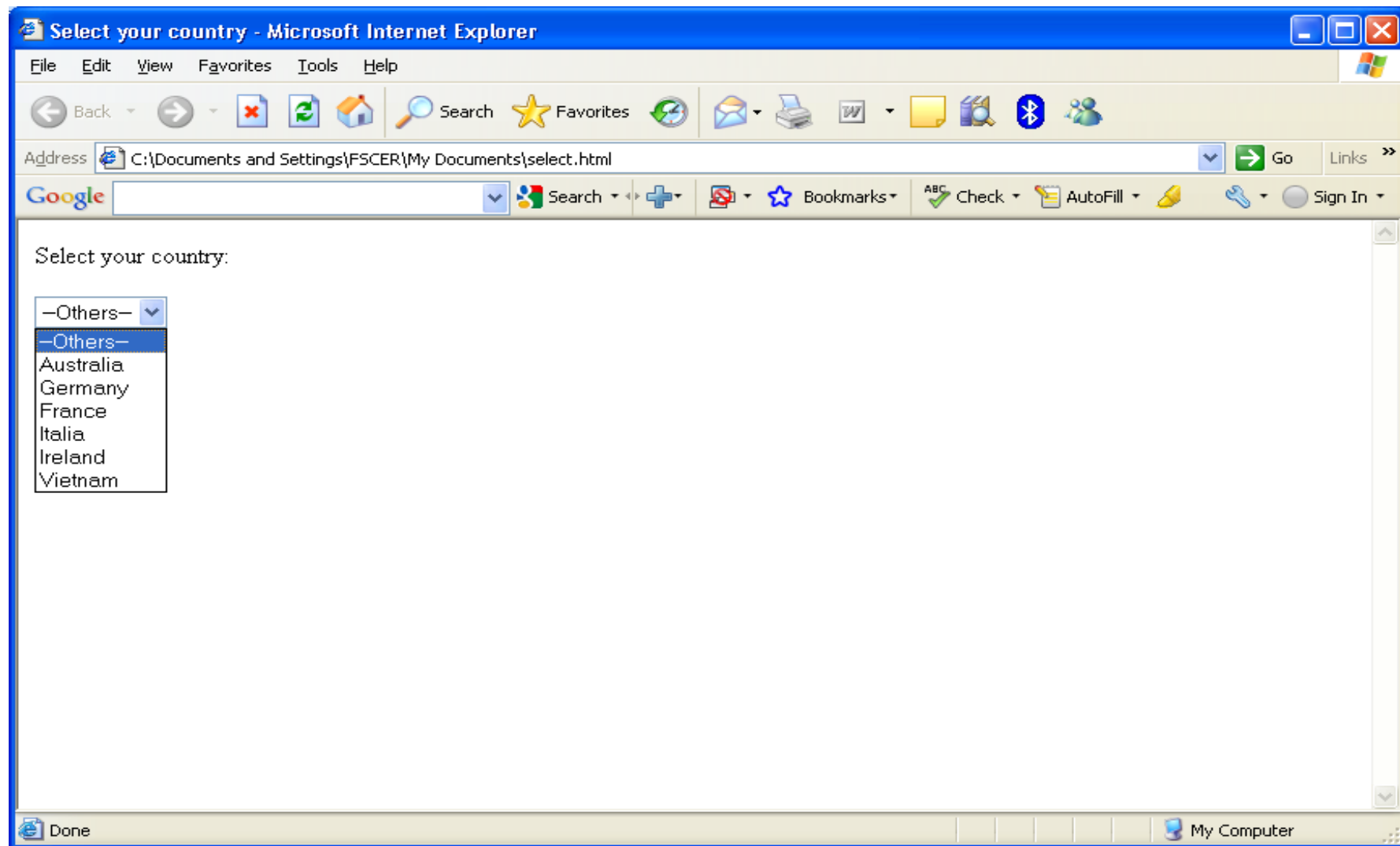
---

**Ví dụ:** Xét đoạn Script và xem kết quả trên trình duyệt

```
<form method="POST" name="Form 1"
      action="">
<p>Select your country: </p>
  <select name="S1">
    <option value=0> --Others--</option>
    <option value=1> Australia</option>
    <option value=2> Germany</option>
    <option value=3> France</option>
    <option value=4> Italia</option>
    <option value=5> Ireland</option>
    <option value=6> Vietnam</option>
  </select>
</form>
```

# Phần tử select

## Hiển thị trên Browser





## Phần tử Textarea

---

- Được sử dụng khi cần nhập vào một đoạn văn bản gồm nhiều dòng (khối văn bản)
- Được đặt trong cặp thẻ `<textarea>` `</textarea>`
- Các thuộc tính `cols` và `rows` dùng để thiết lập số cột và số dòng dùng để hiển thị đoạn văn bản
- Cú pháp:  
`<textarea name, cols, rows>` Đoạn văn bản cần nhập `</textarea>`



## Phần tử textarea

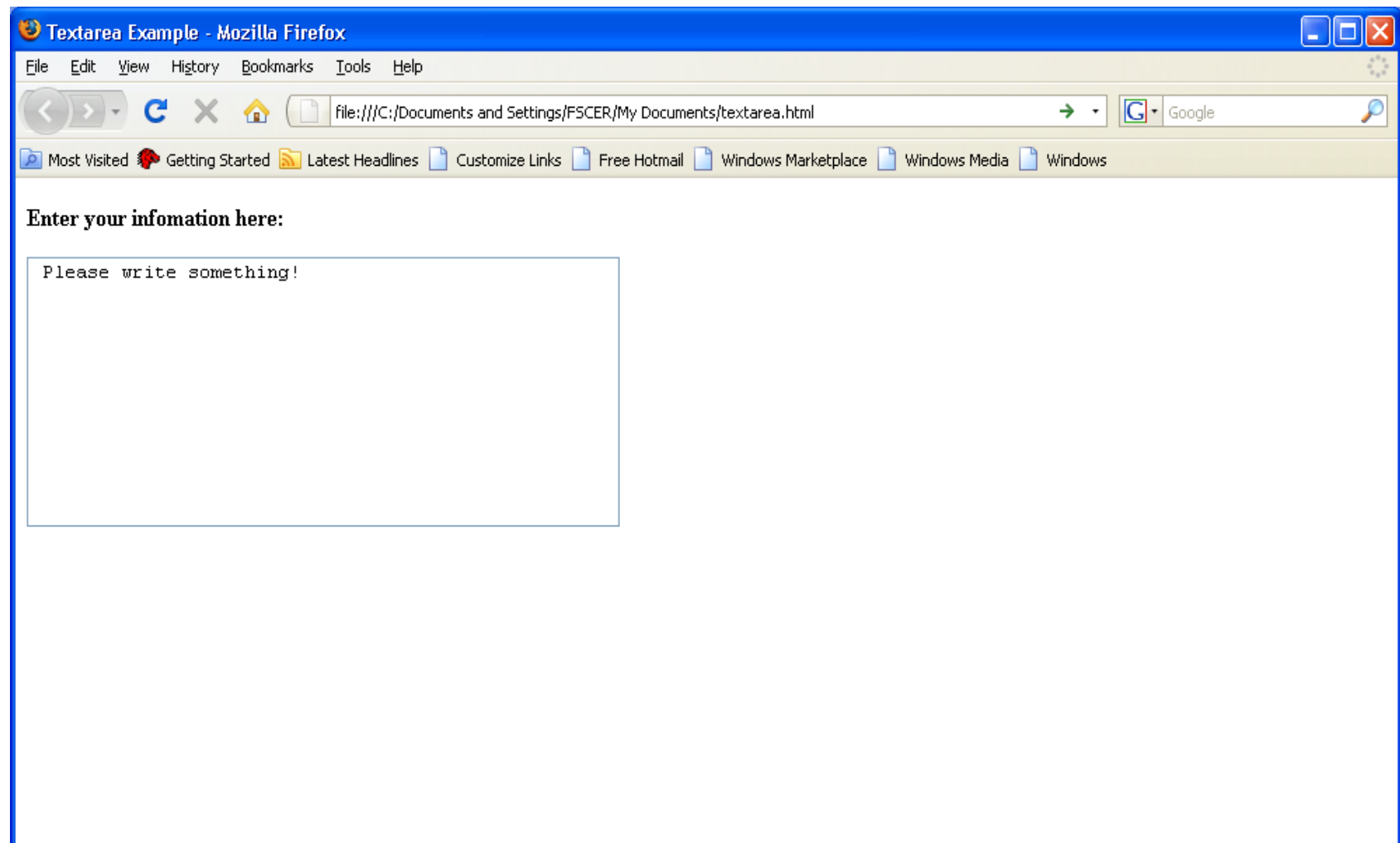
---

**Ví dụ:** Xét đoạn Script sau:

```
<html>
<head> <title>Textarea Example</title>
</head>
<body>
  <form action="" name="F1" >
    <p><b>Enter your infomation here:
      </b></p>
    <textarea rows=10 cols=50> Please
      write something!
    </textarea>
  </form>
</body>
</html>
```

# Phần tử select

## Hiển thị trên Browser



## Phần tử Button

---

- Button là một phiên bản của nút Submit, nó cho phép cả văn bản và hình ảnh được đặt trên cùng một nút
- Được đặt trong cặp thẻ <button> </button>

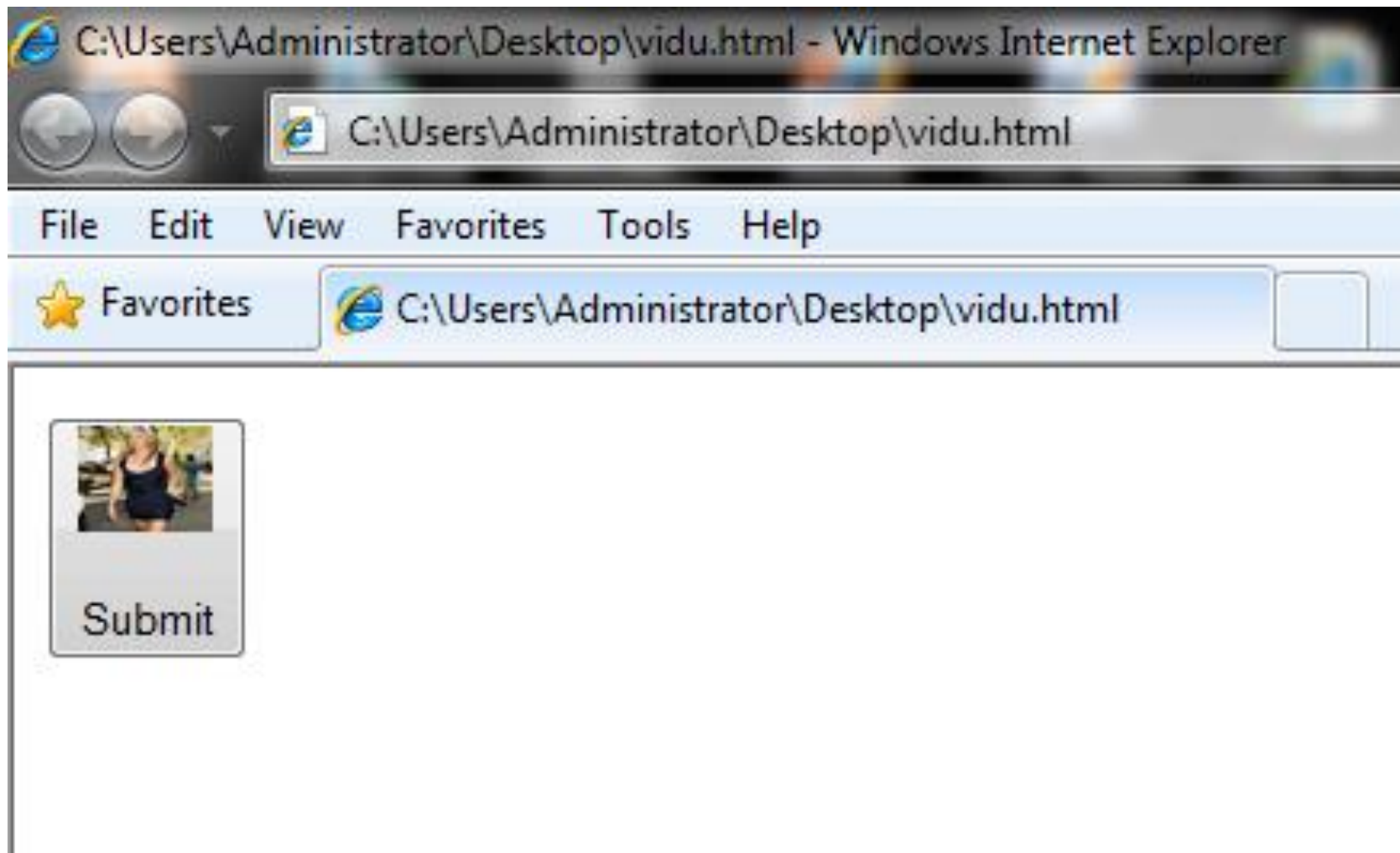
### Ví dụ:

```
<form method=POST name="form 1" action="">
<button>
  <img border = "0" src = "nhan.JPG" width =
    "42" height = "33">
  <br>Submit
</button>
</form>
```

## Phần tử button

---

Hiển thị trên Browser





## 1.2. Các kiểu phần tử trên form

---

Lưu ý:

- Để nhóm các phần tử trên form lại ta có thể dùng cặp thẻ `<fieldset>` `</fieldset>`
- Để tạo tiêu đề cho nhóm các phần tử ta dùng cặp thẻ `<legend>` `</legend>`

Bài tập: Tìm hiểu một số Website trên mạng sau đó sử dụng các kiểu phần tử trên form trong HTML để thiết kế một WebPage.



## 1.2. Các kiểu phần tử trên Form

### Đăng ký thông tin

Họ

Tên

Mật khẩu

Email

Ngày sinh

Giới Tính : ☐ Nam ☐ Nữ ☐ Khác

Thành phố

Sở thích ☐ Đọc sách ☐ Nghe nhạc ☐ Xem phim ☐ Bóng đá ☐ Bóng chuyền ☐ cầu lông

Mô tả bản thân:

Đăng ký

Làm lại

Click to turn off Vietnamese mode



### 1.3. Mô hình client/ server và webserver

---

- ❑ Mô hình Client/ Server là môi trường, kiến trúc căn bản nhất để trang Dynamic Web hoạt động được
- ❑ Clients có thể xem như là các máy do người dùng sử dụng để truy cập vào Website
- ❑ Server là nơi dùng để lưu trữ các Website và Database. Xử lý việc cùng một lúc nhiều Clients truy cập vào cùng một Website



### 1.3. Mô hình client/ server và webserver

---

- ❑ Hầu hết các ứng dụng Web đều hoạt động tập trung trên Server
- ❑ Một Cơ sở dữ liệu (CSDL) trên Server sẽ lưu trữ tất cả những thông tin đáp ứng yêu cầu cho công việc của ứng dụng Web
- ❑ Webserver là một ứng dụng dùng để đảm trách việc giao tiếp với các trình duyệt. Nhận các yêu cầu từ phía Clients, xử lý và trả lời các yêu cầu đó
- ❑ Có nhiều loại Webserver khác nhau nhưng thông dụng nhất là: Apache và IIS (Internet Information Server)





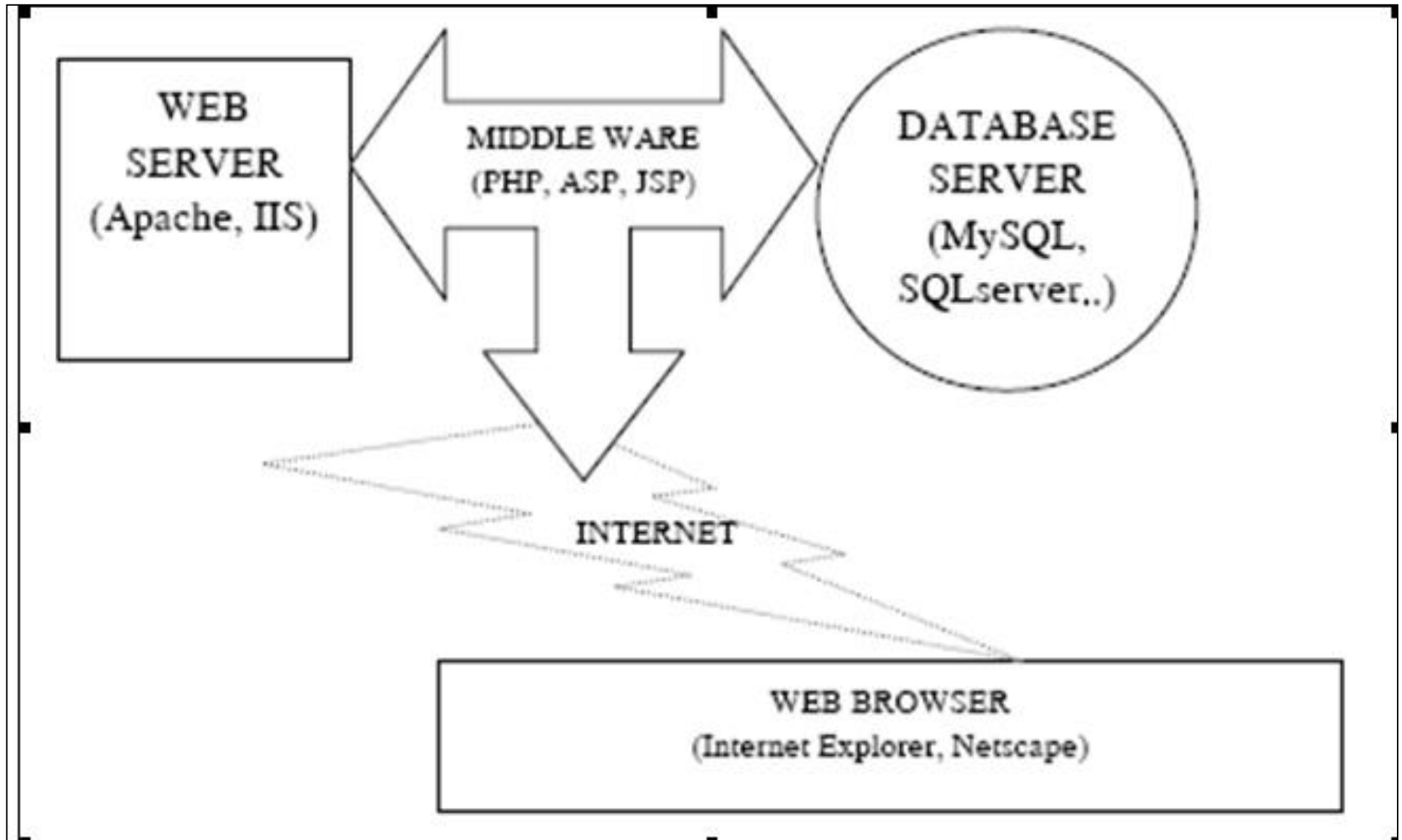
### 1.3. Mô hình client/ server và webserver

---

- ❑ Để tạo sự liên kết giữa Webserver và CSDL lưu trữ trên Server cần phải có một ngôn ngữ lập trình. Chẳng hạn như: asp, PHP, jsp, ...thuộc lớp ngôn ngữ lập trình MiddleWare
- ❑ Ta có mô hình hoạt động như sau:



## 1.3. Mô hình client/ server và webserver





HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

## Chương 2: Tổng quan về ngôn ngữ PHP



## Chương 2: Tổng quan về ngôn ngữ PHP

---

**2.1. GIỚI THIỆU VỀ PHP**

**2.2. CÚ PHÁP**

**2.3. CÁC KIỂU DỮ LIỆU**

**2.4. BIẾN VÀ HẰNG**

**2.5. PHÉP GÁN VÀ CÁC PHÉP TOÁN**

**2.6. TRUY CẬP ĐẾN FORM**

**2.7. CÁC CẤU TRÚC ĐIỀU KHIỂN**



## 2.1. Giới thiệu về PHP

---

- **Php là gì?**
- **Đặc điểm của file PHP**
- **Lịch sử phát triển**
- **Download, cài đặt và cấu hình ứng dụng PHP**
- **Quá trình thông dịch trang PHP**

## Php là gì?

---

- ❑ **PHP** được viết tắt của chữ **PHP: Hypertext Preprocessor**
- ❑ Là ngôn ngữ kịch bản trình chủ (Server Script) chạy trên phía máy chủ (Server side) giống như các Server script khác: asp, jsp, cold fusion, ...
- ❑ Là kịch bản cho phép chúng ta xây dựng ứng dụng web trên mạng Internet hay Intranet tương tác với mọi cơ sở dữ liệu như: Informix, MySQL, PostgreSQL, Oracle, Sybase, SQL Server,...
- ❑ Là phần mềm mở, dùng cho mục đích tổng quát. Thích hợp với Web và có thể dễ dàng nhúng vào trang HTML



## Đặc điểm của file PHP

---

- ☐ Các file PHP trả về kết quả cho trình duyệt là một trang thuần HTML
- ☐ Các file PHP có thể chứa văn bản (Text), các thẻ HTML (HTML tags) và các đoạn mã kịch bản (Script)
- ☐ Các file PHP có phần mở rộng là: .PHP, .PHP3, .Phpml
- ☐ Lưu ý rằng, từ phiên bản 4.0 trở về sau mới hỗ trợ session

## Lịch sử phát triển

---

- ❑ Năm 1995, phiên bản đầu tiên ra đời có tên là PHP/FI được viết bởi nhà phát triển phần mềm Rasmus Lerdorf.
- ❑ PHP/FI, viết tắt từ " Hypertext Preprocessor /Forms Interpreter", bao gồm một số các chức năng cơ bản của PHP ngày nay.
- ❑ Năm 1997, phiên bản PHP/FI 2.0 ra đời nhưng chỉ được công bố dưới dạng các bản beta. Đến tháng 11 năm 1997 mới chính thức được công bố
- ❑ Năm 1998, phiên bản PHP 3.0 được chính thức công bố



## Lịch sử phát triển

---

- ❑ Andi Gutmans và Zeev Suraski tiếp tục hoàn tất phần lõi nhằm cải tiến PHP 3.0.
- ❑ Tháng 05/2000, phiên bản PHP 4.0 với hàng loạt các tính năng mới bổ sung, đã chính thức được công bố
- ❑ 29/06/2003, phiên bản PHP 5 Beta 1 đã chính thức được công bố
- ❑ Tháng 10/2003, phiên bản Beta 2 ra mắt với sự xuất hiện của hai tính năng rất được chờ đợi: Iterators, Reflection nhưng namespace một tính năng gây tranh cãi khác đã bị loại khỏi mã nguồn

## Lịch sử phát triển

---

- ❑ Ngày 21/12/2003: phiên bản PHP 5 Beta 3
- ❑ Ngày 13/07/2004, phiên bản PHP 5 bản chính thức đã ra mắt sau một chuỗi khá dài các bản kiểm tra thử bao gồm Beta 4, RC 1, RC2, RC3
- ❑ Ngày 14/07/2005, phiên bản PHP 5.1 Beta 3 được PHP Team công bố đánh dấu sự chín muồi mới của PHP với sự có mặt của PDO
- ❑ Phiên bản được dùng phổ biến nhất: PHP 5.6
- ❑ Hiện nay, phiên bản tiếp theo của PHP đang được phát triển, PHP 7, có thể được download tại địa chỉ <http://snaps.PHP.net>



## Download, cài đặt và cấu hình ứng dụng PHP

---

- ☐ Nếu máy chủ chưa được hỗ trợ PHP thì cần phải cài đặt nó.
- ☐ Download miễn phí tại:  
<http://www.PHP.net/downloads.PHP>
- ☐ Để truy cập được vào Web server có hỗ trợ PHP, cần:
  - ☐ Cài đặt Apache hoặc IIS trên máy chủ, cài PHP, MySQL
  - ☐ Hoặc thuê một Web hosting có hỗ trợ PHP và MySQL
- ☐ Có thể sử dụng một số phần mềm tích hợp sẵn Apache, PHP, MySQL. Chẳng hạn, như XAMPP download tại: [www.apachefriends.org](http://www.apachefriends.org)



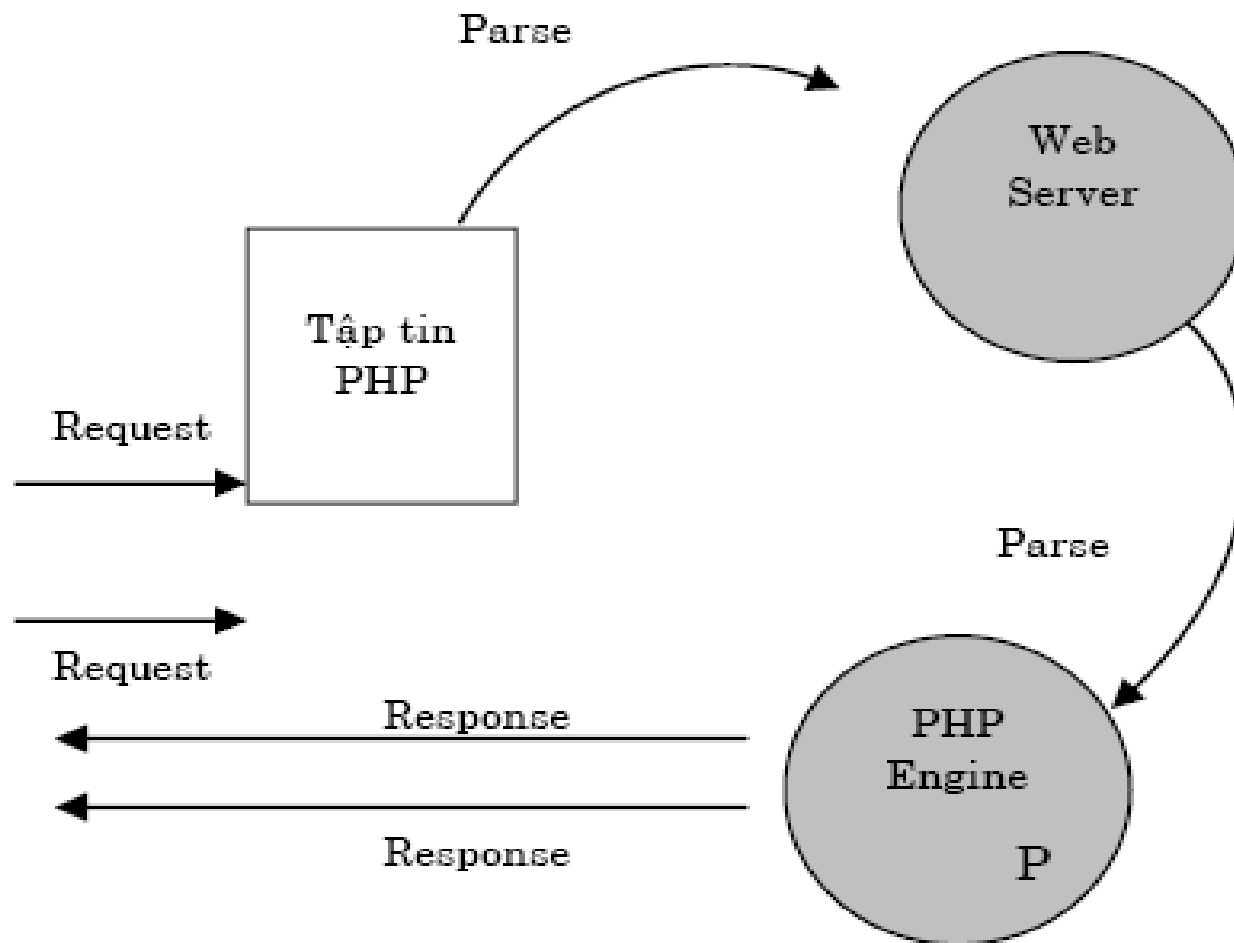
## Quá trình thông dịch trang PHP

---

- ❑ PHP là kịch bản trình chủ được chạy trên nền PHP Engine, cùng với ứng dụng Web Server để quản lý chúng.
- ❑ Khi trang PHP được gọi, Web Server triệu gọi PHP Engine để thông dịch, dịch trang PHP và trả về kết quả cho người sử dụng là một trang thuần HTML
- ❑ Ta có mô hình như sau:



## Quá trình thông dịch trang PHP



Hình 1-9: Quá trình thông dịch trang PHP

## 2.2. Cú pháp

---

- ❑ Ta có thể nhúng các lệnh của PHP vào trang HTML
- ❑ Đoạn mã PHP luôn được bắt đầu và kết thúc bởi cặp thẻ theo cú pháp:  
**<?PHP**  
**các lệnh của PHP;**  
**?>**
- ❑ Đoạn mã PHP có thể đặt bất kỳ đâu trong tài liệu
- ❑ Thông thường một trang PHP bao gồm các thẻ HTML như một trang HTML nhưng có thêm các đoạn mã PHP

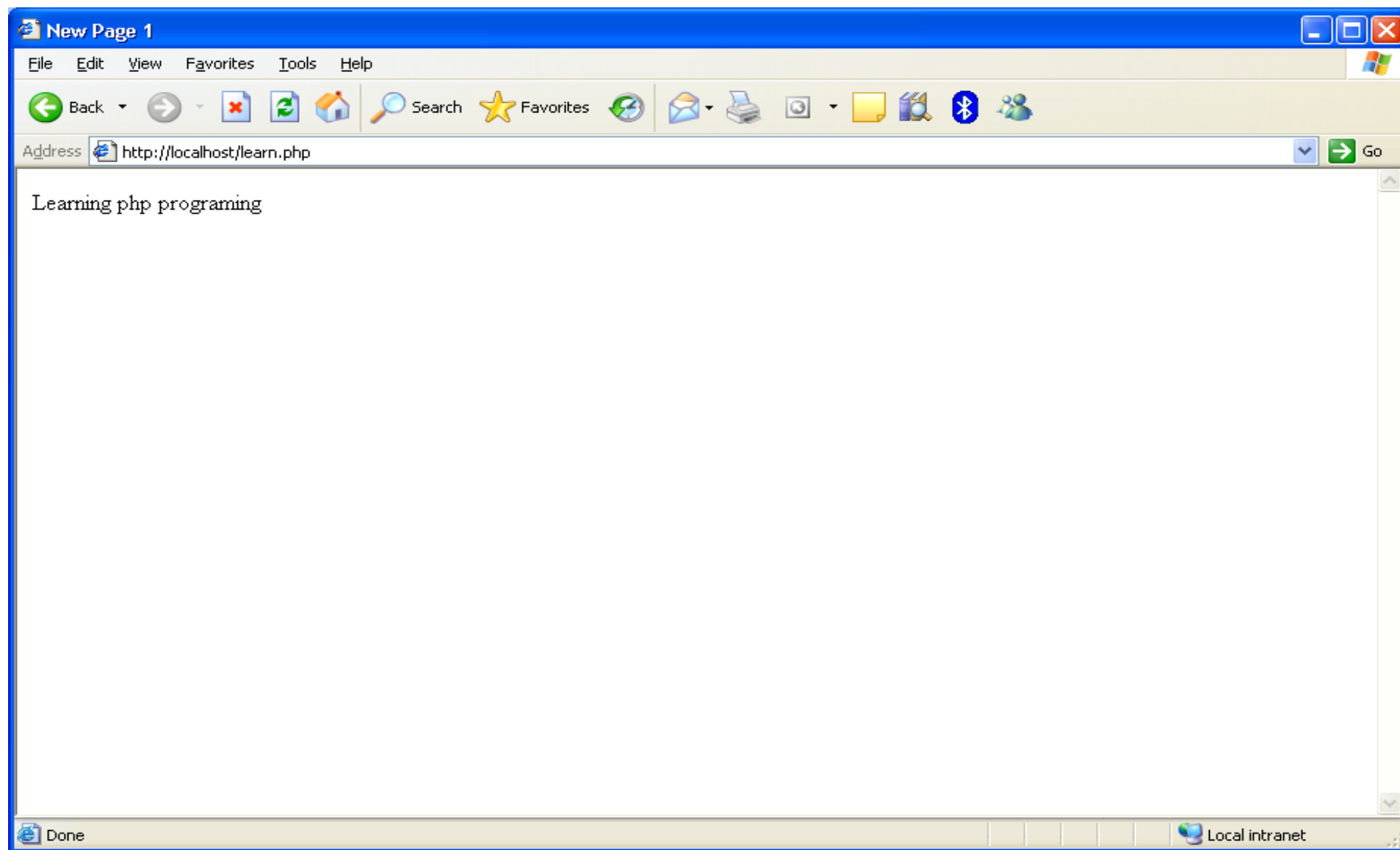
## 2.2. Cú pháp

---

**Ví dụ:** Ta có đoạn mã PHP hiển thị câu “Learning PHP programing” lên trình duyệt như sau:

```
<html>
<head>    </head>
<body>
    <?PHP
        echo "Learning PHP programing";
    ?>
</body>
</html>
```

## 2.2. Cú pháp







## 2.2. Cú pháp

---

- ❑ Mỗi câu lệnh trong PHP được kết thúc bằng dấu (;). Dấu này là một toán tử dùng để phân biệt các cấu trúc với nhau
- ❑ Có hai câu lệnh cơ bản dùng để hiển thị các câu text ra browser là: *echo* và *print*



## 2.2. Cú pháp

---

### Lưu ý:

- ☐ Các file PHP phải có phần mở rộng là .PHP. Nếu phần mở rộng là .html thì đoạn mã PHP sẽ không được thực thi
- ☐ Có thể viết các câu chú thích cho đoạn mã PHP. Có hai cách viết là: */\* chú thích \*/* hoặc *// chú thích*
- ☐ Đoạn mã PHP cũng có thể đặt trong cặp thẻ:  
**<? Đoạn mã PHP ?>**

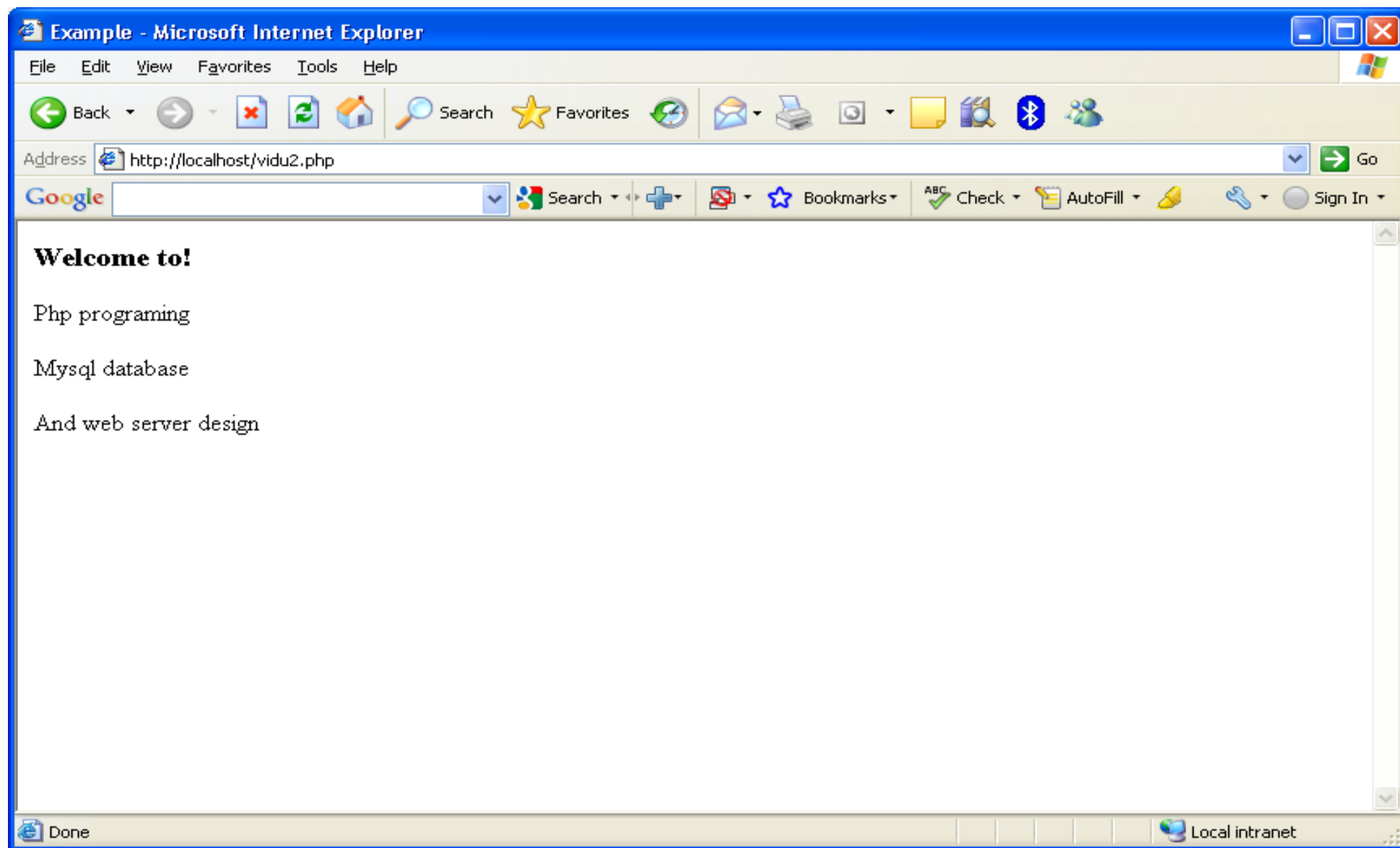
## 2.2. Cú pháp

---

**Ví dụ:** Ta có trang vidu2.php như sau:

```
<html>
  <head>
    <title>Example</title>
  </head>
  <body>
    <h3> Welcome to!</h3>
    <?PHP echo "<p>Php programing</p>";?>
    <?PHP print "<p>Mysql database</p>";
    ?>
    <p>And web server design</p>
  </body>
</html>
```

## 2.2. Cú pháp





## 2.3. Các kiểu dữ liệu

---

- PHP hỗ trợ 5 kiểu dữ liệu như sau:
  - **Integer**: sử dụng cho giá trị có kiểu dữ liệu là số nguyên
  - **Double**: sử dụng cho giá trị có kiểu dữ liệu là số thực
  - **String**: sử dụng cho các giá trị có kiểu dữ liệu là chuỗi và ký tự
  - **Array**: sử dụng cho các giá trị có kiểu dữ liệu là mảng
  - **Object**: sử dụng cho các giá trị có kiểu dữ liệu là đối tượng của lớp



## 2.4. Biến và hằng trong PHP

---

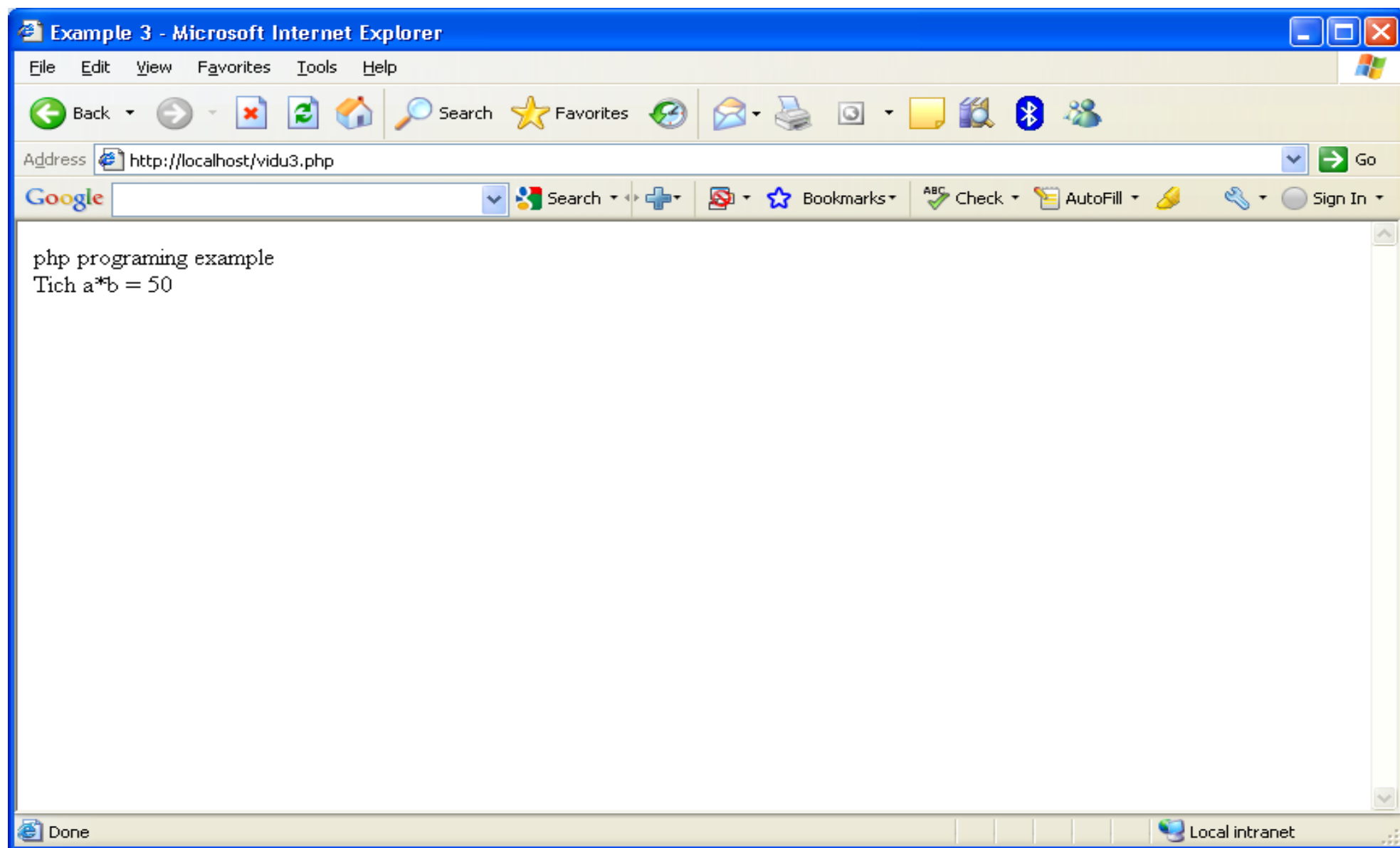
- ☐ **Biến**
- ☐ **Hằng**

- Biến dùng để lưu giá trị như: chuỗi, số, ký tự, mảng,...
- Tất cả các biến trong PHP đều bắt đầu bằng ký hiệu \$
- Biến được khai báo tự động khi sử dụng (gán giá trị) cho nó theo cú pháp: **\$Tên\_biến = Giá\_trị;**
- PHP là ngôn ngữ không định kiểu, nghĩa là không cần khai báo kiểu cho biến. PHP sẽ chuyển kiểu của biến một cách tự động tùy thuộc vào giá trị của nó
- Lưu ý: Quy tắc đặt tên biến trong PHP giống như trong C, C++, ...

**Ví dụ:** Ta có trang vidu3.PHP như sau:

```
<html>
  <head>
    <title>Example 3</title>
  </head>
  <body>
    <?PHP $a = "PHP programing example";
    echo "$a <br>";
    $a = 5;  $b = 10;  $c = $a*$b;
    echo "Tich a*b = $c";
    ?>
  </body>
</html>
```





- Phạm vi của biến:
  - Nếu biến được khai báo trong Script thì có phạm vi trong toàn Script
  - Nếu biến được khai báo trong một hàm nào đó thì chỉ có tác dụng trong hàm đó
- Kiểm tra/ loại bỏ biến:
  - Sử dụng hàm **isset(Tên\_biến)** để kiểm tra biến đó có tồn tại hay không? Kết quả trả về kiểu boolean
  - Sử dụng hàm **unset(Tên\_biến)** để loại bỏ biến đang tồn tại ra khỏi trạng thái thực thi
  - Sử dụng hàm **empty(Tên\_biến)** để kiểm tra biến tồn tại và không rỗng

**Ví dụ:** Xét đoạn mã sau đây:

```
<?PHP
    $a = 10;
    echo  empty($a) . "</br>";
    echo  isset($a) . "</br>";
    unset($a);
    echo  isset($a) . "</br>";
?>
```

- Ngoài ra, để kiểm tra kiểu dữ liệu của biến ta có thể sử dụng các hàm sau:
  - `is_array()`
  - `is_double()`
  - `is_float()`
  - `is_long()`
  - `is_int()`
  - `is_string()`
  - `is_object()`

- Hằng trong PHP được khai báo giống như các ngôn ngữ C, C++
- Tên hằng thường được viết bằng chữ hoa
- Cú pháp khai báo hằng như sau:

**define (“Tên\_hằng”, giá\_trị);**

hoặc **define (Tên\_hằng, giá trị);**

**Ví dụ:** Ta có khai báo hằng MAX = 100 như sau:

**define (“MAX”, 100);**

**define (MAX, 100);**



## 2.5 Các phép toán

---

- Phép gán
- Các phép toán số học
- Các phép toán quan hệ
- Các phép toán logic
- Phép toán trên chuỗi
- Các phép toán tự tăng giảm
- Biểu thức điều kiện

## Phép gán

---

- Phép gán là phép toán cơ bản của mọi ngôn ngữ lập trình.
- Phép gán đơn: cú pháp: \$Tên\_biến = Giá\_trị;
- Phép gán mở rộng: \$Tên\_biến **pt**= Giá\_trị;
- Trong đó: **pt** có thể là: +, -, \*, /, %



## Các phép toán số học

---

- Phép toán số học một ngôi: - (đảo dấu)
- Phép toán số học hai ngôi:

| Toán tử | Tên         | Ví dụ        |
|---------|-------------|--------------|
| +       | Cộng        | $\$a + \$b$  |
| -       | Trừ         | $\$a - \$b$  |
| *       | Nhân        | $\$a * \$b$  |
| /       | Chia nguyên | $\$a / \$b$  |
| %       | Chia lấy dư | $\$a \% \$b$ |





## Các phép toán quan hệ

- Các phép toán quan hệ trả về kết quả là true hoặc false

| Toán tử | Tên                       | Ví dụ                   |
|---------|---------------------------|-------------------------|
| <       | Bé thua                   | $3 < 5$                 |
| <=      | Bé thua hoặc bằng         | $A \leq b$              |
| >       | Lớn hơn                   | $\$a > \$b$             |
| >=      | Lớn hơn hoặc bằng         | $\$a \geq \$b$          |
| ==      | Bằng                      | $123 = \text{"123"}$    |
| ===     | Bằng và cùng kiểu dữ liệu | $123 = \text{"123"}$    |
| !=      | Khác                      | $123 \neq \text{"123"}$ |
| !==     | Khác kiểu dữ liệu         | $123 \neq \text{"123"}$ |
| <>      | Khác                      | $123 <> \text{"123"}$   |



## Các phép toán Logic

---

- Giống như các phép toán quan hệ các phép toán Logic trả về kết quả là True hoặc False

| Toán tử | Tên            | Ví dụ      |
|---------|----------------|------------|
| &&      | And (và)       | \$a && \$b |
|         | Or (hoặc)      | \$a or \$b |
| !       | Not (phủ định) | !\$b       |



## Các phép toán tự tăng giảm

---

- Để tăng (hoặc giảm) giá trị của một biến lên (xuống) một đơn vị có thể sử dụng phép toán tự tăng ++ và tự giảm --
- Có hai cách viết phép toán tự tăng giảm:
  - ++&Tên\_biến (hoặc --\$Tên\_biến)
  - &Tên\_biến++ (hoặc \$Tên\_biến--)
- Lưu ý: cần phân biệt hai cách viết trên.

## Phép toán về chuỗi

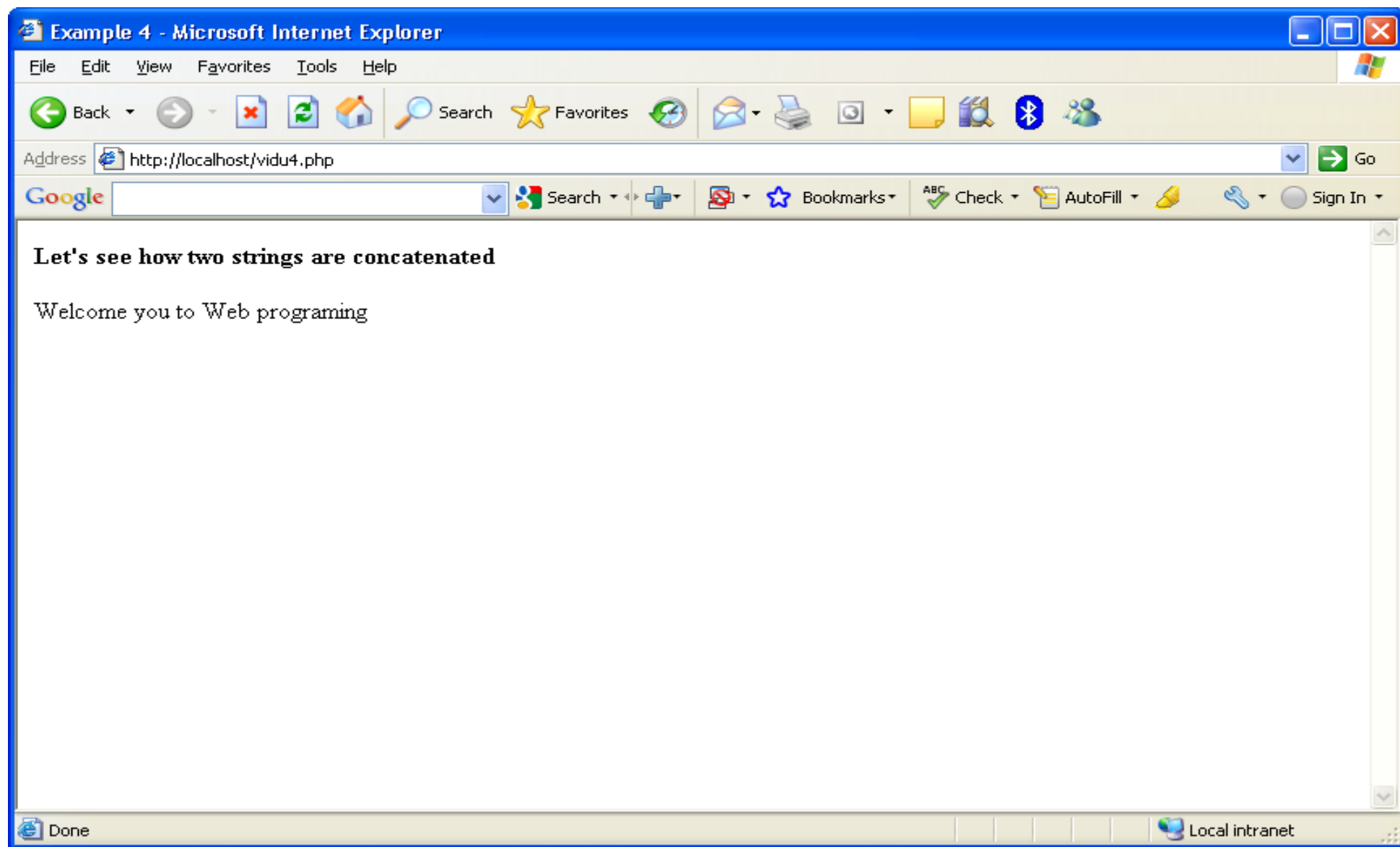
---

- **Phép cộng chuỗi:** Để cộng (ghép) hai chuỗi lại với nhau ta sử dụng dấu chấm (.)

**Ví dụ:** xét đoạn mã sau:

```
<html>
    <head>
        <title>Example 4</title>
    </head>
    <h4> Let's see how two strings are
    concatenated </h4>
    <?PHP $st1 = "Welcome you to "; $st2 = "Web
    programing";
        echo $st1.$st2; ?>
    </body>
</html>
```

# Phép toán về chuỗi





## Biểu thức điều kiện

---

- Cú pháp:

**Biến = Giá trị 1 > Giá trị 2 ? Giá trị 1 : Giá trị 2;**

Ví dụ:

```
<?PHP
```

```
$a = 'a'; $b = 'b';
```

```
echo $a > $b ? $a : $b;
```

```
?>
```



## 2.6. Truy cập đến Form

---

- Form cùng với các thuộc tính của nó là nơi để người sử dụng nhập dữ liệu vào
- Để làm việc với các dữ liệu đó đòi hỏi phải kết nối đến form
- Khi làm việc với Form thì các phần tử Form trên trang HTML sẽ tự động trở thành biến trong đoạn mã PHP
- Để lấy giá trị từ các phần tử Form ta sử dụng các hàm `$_GET` hoặc `$_POST`

## Hàm \$\_GET

---

- Là hàm xây dựng sẵn dùng để lấy các giá trị từ Form có sử dụng Method = GET
- Thông tin khi truyền đi với phương thức GET sẽ được hiển thị trên Browser's address bar
- Mọi người có thể nhìn thấy thông tin và số ký tự tối đa là 2000
- Cú pháp lấy giá trị từ các phần tử Form

**`$_GET["Tên phần tử Form"]`**



**Ví dụ:** Ta có trang login.html như sau:

```
<h2> Login user: </h2>
<form name="f1" method = "GET" action =
    "display.php">
    <p>User name: <input type="text"
        name="username" size="35" maxlength="30"
        value=""></p>
    <p>Password: <input type="password"
        name="password" size="35" value=""></p>
    <p><input type="submit" name="submit"
        value="Ok" style="width: 50; height: 25">
        <input type="reset" name="reset"
        value="Cancel" style="width:50;
        height:25">
    </p>
</form>
```

## Hàm \$\_GET

---

Trang display.php như sau:

```
<html>
  <head> </head>
  <body>
    <h2> Data on form will be display on
        browser through PHP</h2>
    <?PHP
      echo "User name:
        " . $_GET["username"] . "<br>";
      echo "Password: " . $_GET["password"] ;
    ?>
  </body>
</html>
```

## Hàm \$\_GET

Trang login - Windows Internet Explorer

   http://localhost/login.html

File Edit View Favorites Tools Help

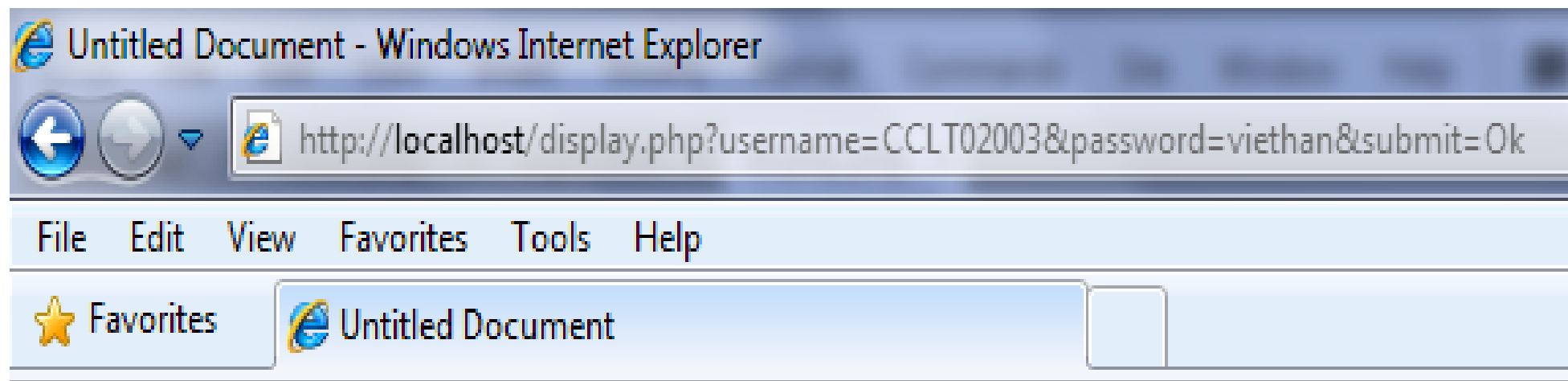
★ Favorites  Trang login

**Login user:**

User name:

Password:

## Hàm \$\_GET



**Data on form will be display on browser through php**

User name: CCLT02003

Password: viethan

## Hàm \$\_POST

---

- Là hàm xây dựng sẵn dùng để lấy các giá trị từ form có sử dụng Method = POST
- Thông tin khi truyền đi với phương thức POST sẽ không được hiển thị trên Browser's address bar
- Không thể nhìn thấy các thông tin (biến và giá trị) đang truyền.
- Cú pháp lấy giá trị từ các phần tử Form:

**`$_POST["Tên phần tử form"]`**

**Ví dụ:** Thiết kế Form và trang PHP để giải quyết bài toán tìm nghiệm của phương trình bậc nhất, bậc hai



## 2.7 Các cấu trúc điều khiển

---

- Cấu trúc rẽ nhánh
- Cấu trúc lặp

## Cấu trúc rẽ nhánh

---

- **Cấu trúc if:**

Cú pháp: *if (điều kiện) câu lệnh PHP;*

**Ví dụ:**

```
<?PHP
```

```
    $a = 7; $b = 3;
```

```
    if ($a>$b) echo "Gia trị lớn nhất  
    là: ".$a;
```

```
?>
```



## Cấu trúc rẽ nhánh

---

### Cấu trúc if ... else:

Cú pháp: *if (điều kiện) công việc 1;*  
*else công việc 2;*

**Ví dụ:** <?PHP

```
$a = 7; $b = 3;  
if ($a>$b)  
    echo "Giá trị lớn nhất là:  
". $a;  
else  
    echo "Giá trị lớn nhất là:  
". $b; ?>
```





## Cấu trúc rẽ nhánh

---

Lưu ý:

- *Ta có thể sử dụng cấu trúc if lồng nhau khi có nhiều hơn 2 sự lựa chọn*
- *Nếu cần thực thi nhiều câu lệnh thì cần đặt nó trong cặp dấu ngoặc móc { }*

- **Cấu trúc switch:** sử dụng khi có nhiều sự lựa chọn

Cú pháp:

```
switch (n)  
{  
  case label 1: code to be executed if n=label 1;  
  break;  
  case label 2: code to be executed if n=label 2;  
  break;  
  ...  
  case label n: code to be executed if n=label n;  
  break;  
  default: code to be executed if n is different from label 1 to label n;  
}
```

## Cấu trúc rẽ nhánh

---

Ví dụ:

```
<?PHP
$kt=insert;
switch ($kt)
{
    case "edit":echo"sửa dữ liệu <br/>"; break;
    case "insert":echo"chèn dữ liệu<br/>";
    break;
    case "delete":echo"xoa dữ liệu<br/>";
    break;
    case "save":echo"lưu dữ liệu<br/>"; break;
}
?>
```

## Cấu trúc rẽ nhánh

---

Ví dụ:

```
<?PHP
```

```
$a=4;
```

```
switch ($a)
```

```
{
```

```
    case $a%2==0: echo $a." là số  
    chẵn"; break;
```

```
    case $a%2!=0: echo $a." là số lẻ";  
    break;
```

```
}
```

```
?>
```

- **Cấu trúc While:**

Cú pháp:     while (*condition*)  
                  {  
                      *code to be executed;*  
                  }

- **Cấu trúc do ... while:**

Cú pháp:   do  
                  {  
                      *code to be executed;*  
                  }  
          while (*condition*);



## Cấu trúc rẽ nhánh

---

**Ví dụ:**

```
<?PHP
```

```
$a=0;
```

```
while ($a<10)
```

```
{
```

```
    do {echo $a; $a++;}
```

```
    while ($a<5) ;
```

```
    echo "<br/>"; $a++;
```

```
}
```

```
?>
```

- **Cấu trúc for:**

Cú pháp:     for (*init; condition; increment*)  
                  {  
                      *code to be executed;*  
                  }

- **Cấu trúc foreach:** sử dụng khi lặp trên mảng

Cú pháp:     foreach (*\$array as \$value*)  
                  {  
                      *code to be executed;*  
                  }

## Cấu trúc rẽ nhánh

---

Ví dụ:

```
<?PHP
$tpho1 = array("HoChiMinh", "HaNoi",
               "HaiPhong", "DaNang");
$tpho2 = array("HCM" => "HoChiMinh", "HN" =>
               "HaNoi", "HP" => "HaiPhong", "DN" =>
               "DaNang");
foreach($tpho1 as $tp)
{
    echo $tp."<br/>";
}
foreach($tpho2 as $chiso=>$giatri)
echo $chiso.": ".$giatri."<br/>";
?>
```





## Chương 3: Mạng và chuỗi ký tự



## Chương 3: Mảng và chuỗi ký tự

---

**3.1 GIỚI THIỆU VỀ MẢNG**

**3.2 MẢNG MỘT CHIỀU**

**3.3 MẢNG HAI CHIỀU**

**3.4 CÁC HÀM XỬ LÝ TRÊN MẢNG**

**3.5 CHUỖI KÝ TỰ**

**3.6 CÁC HÀM XỬ LÝ TRÊN CHUỖI**



## 3.1. Giới thiệu về mạng

---

- **Khái niệm mạng**
- **Phân loại mạng**



## Phân loại mảng

---

- Căn cứ vào chỉ số mảng:
  - Mảng có chỉ số kiểu number
  - Mảng có chỉ số kiểu associative

**Ví dụ:** Ta có các mảng

```
$tpho = array("HoChiMinh", "HaNoi", "HaiPhong", "DaNang");  
$tpho = array("HCM" => "HoChiMinh", "HN" => "HaNoi",  
             "HP" => "HaiPhong", "DN" => "DaNang");
```

- Căn cứ vào số chiều của mảng:
  - Mảng một chiều
  - Mảng hai chiều (nhiều chiều)

**Ví dụ:** Ta có các mảng

```
$lop= array(array("LT01A", 34), array("LT01B", 35));
```



## 3.2 Mạng một chiều

---

- **Khai báo mạng**
- **Làm việc với phần tử mạng**
- **Sắp xếp mạng**



## Khai báo mảng

---

Cú pháp:

- Khai báo mảng có chỉ số kiểu number

`$biến_mảng = array(danh sách các giá trị cần gán);`

Hoặc: `$biến_mảng[0]=giá trị; // $biến_mảng[]=giá trị;`

`$biến_mảng[1]=giá trị; // $biến_mảng[]=giá trị`

....

**Ví dụ:** Ta có các khai báo

`$color = array("gray", "green", "red", "blue", "yellow");`

`$color[0]="gray"; $color[1]="green"; ...`

Hoặc `$color[]="gray"; $color[]="green"; ...`

◆ Khai báo mảng có chỉ số kiểu **associative**

```
$biến_mảng=array(“chỉ số 1”=>“giá trị 1”, “chỉ số 2” => “giá trị 2”, ...);
```

Hoặc: \$biến\_mảng[“chỉ số 1”]=giá trị 1;

```
$biến_mảng[“chỉ số 2”]=giá trị 2;
```

....

**Ví dụ:** Ta có các khai báo

```
$tpho = array(“HCM” => “HoChiMinh”, “HN” => “HaNoi”,  
“HP” => “HaiPhong”, “DN” => “DaNang”);
```

```
$tpho[“HCM”]=“HoChiMinh”; $tpho[“HN”]=“HaNoi”; ...
```

## Khai báo mảng

---

- ◆ Khi các giá trị gán cho mảng là các giá trị nguyên hoặc ký tự có tính chất sắp xếp (tăng hoặc giảm) dần. Ta sử dụng hàm range() để khai báo mảng

Cú pháp:

```
$biến_mảng=range(giá trị đầu, giá trị cuối);
```

**Ví dụ:** Ta có mảng:

```
$number = range(0, 20);
```

```
$charac = range("A", "Z");
```





## Làm việc với mảng

---

- ◆ Có thể thao tác với giá trị của các phần tử mảng thông qua tên biến\_mảng kèm với chỉ số của nó

Cú pháp: `$biến_mảng[“chỉ số”]`

**Ví dụ:** Ta có đoạn code sau:

```
<?PHP
```

```
$color=array(“gray”, “green”, “red”, “blue”, “yellow”);
```

```
$tpho=array(“HCM”=>“Ho Chi Minh”, “HN”=>“Ha  
Noi”, “HP”=>“Hai Phong”, “DN”=>“Da Nang”);
```

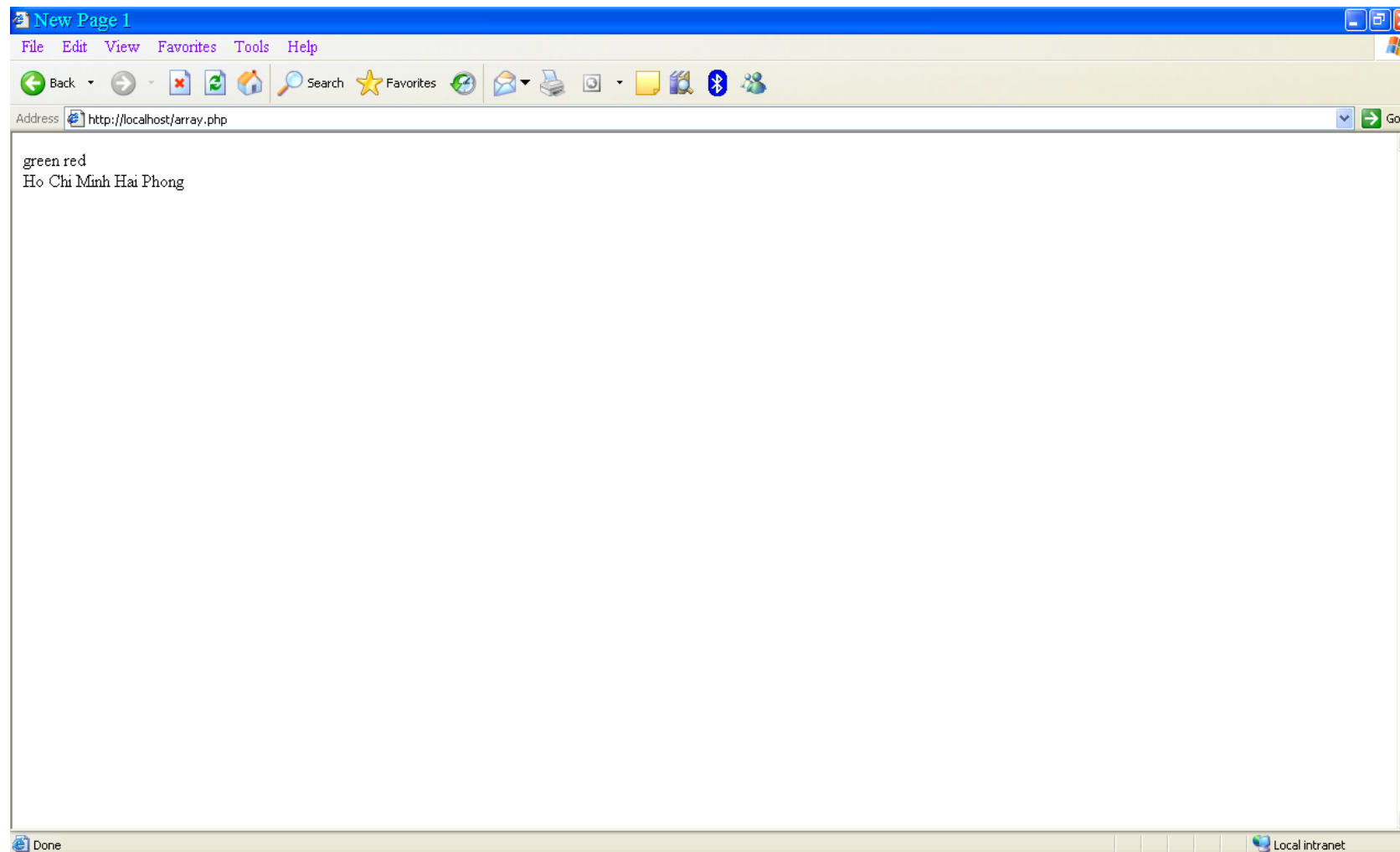
```
echo $color[1]. “ ”. $color[2]. “<br>”;
```

```
Echo $tpho[“HCM”]. “ ”. $tpho[“HP”];
```

```
?>
```

# Làm việc với mảng

---





## Làm việc với mảng

---

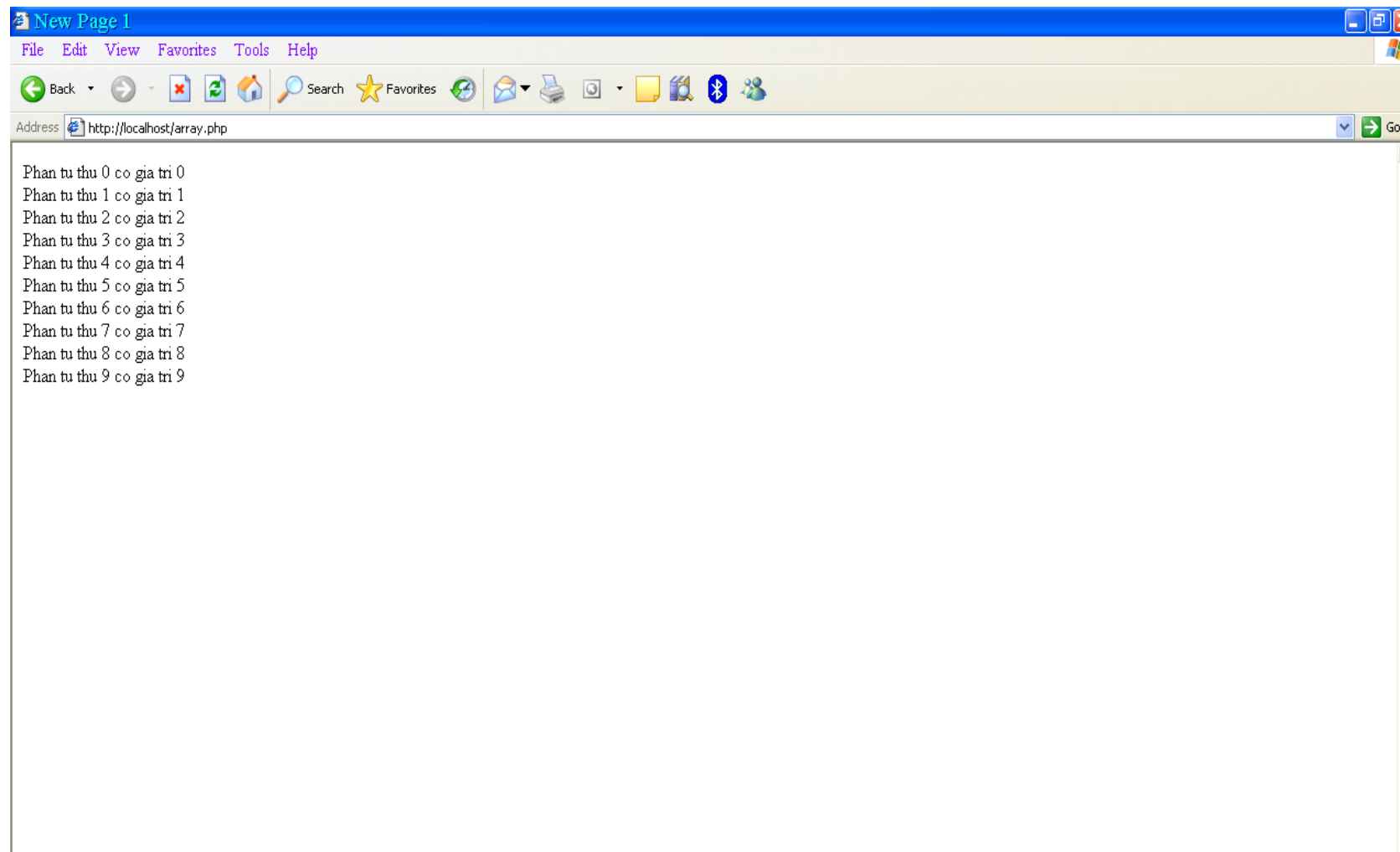
◆ Sử dụng vòng lặp để duyệt các phần tử mảng

**Ví dụ:** Xét đoạn code sau

```
<?PHP $n=10; $mang = array($n);  
    for ($i= 0; $i<$n; $i++)  
        $mang[$i] = $i;  
    for ($i= 0; $i<$n; $i++)  
        echo "Phan tu thu ".$i." co gia tri  
        ".$mang[$i]."<br>"  
?>
```

# Làm việc với mảng

---





## Làm việc với mảng

---

- ◆ Sử dụng hàm `count()` để trả về số phần tử của mảng

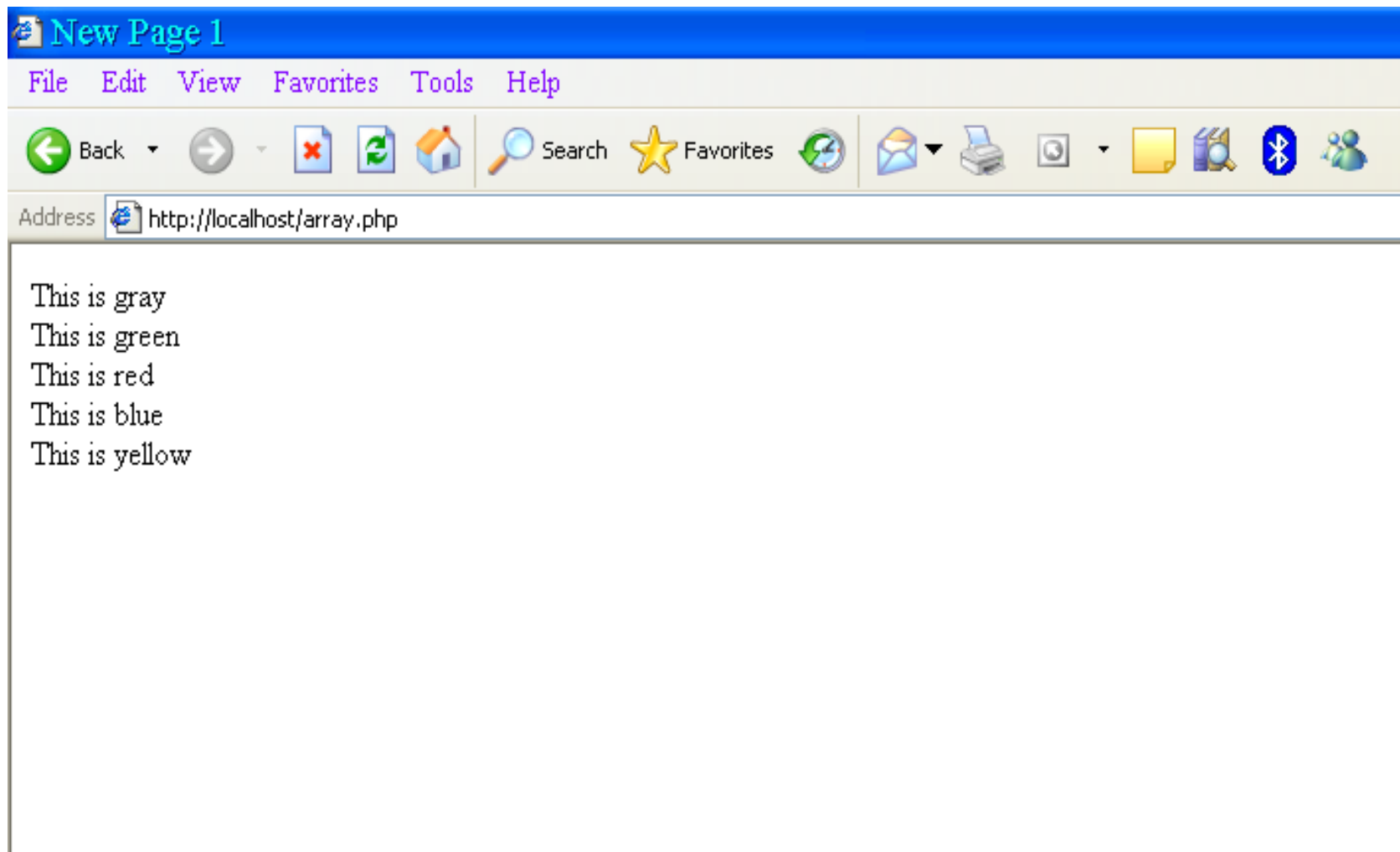
Cú pháp: **`count($biến_mảng)`**, **`sizeof($biến_mảng)`**;

**Ví dụ:** Xét đoạn code sau

```
<?PHP $color=array("gray", "green", "red", "blue",  
    "yellow");  
for ($i= 0; $i<count($color); $i++)  
    echo "This is ".$st[$i]."<br>"  
?>
```



## Làm việc với mảng





## Làm việc với mảng

---

- ◆ Sử dụng hàm **list()** để hiển thị cả chỉ số mảng và giá trị các phần tử mảng

Cú pháp: `list($key, $value)=each($biến_mảng);`

Trong đó:

- Hàm `each()` sẽ lấy cặp chỉ số (khóa) và giá trị của phần tử mảng
- Hàm `list` lấy các giá trị này gán cho `$key` và `$value`. Quá trình này tiếp tục cho đến khi mảng được duyệt hết
- Nếu muốn duyệt lại mảng cần thiết lập lại con trỏ mảng bằng hàm `reset($biến_mảng)`



## Làm việc với mảng

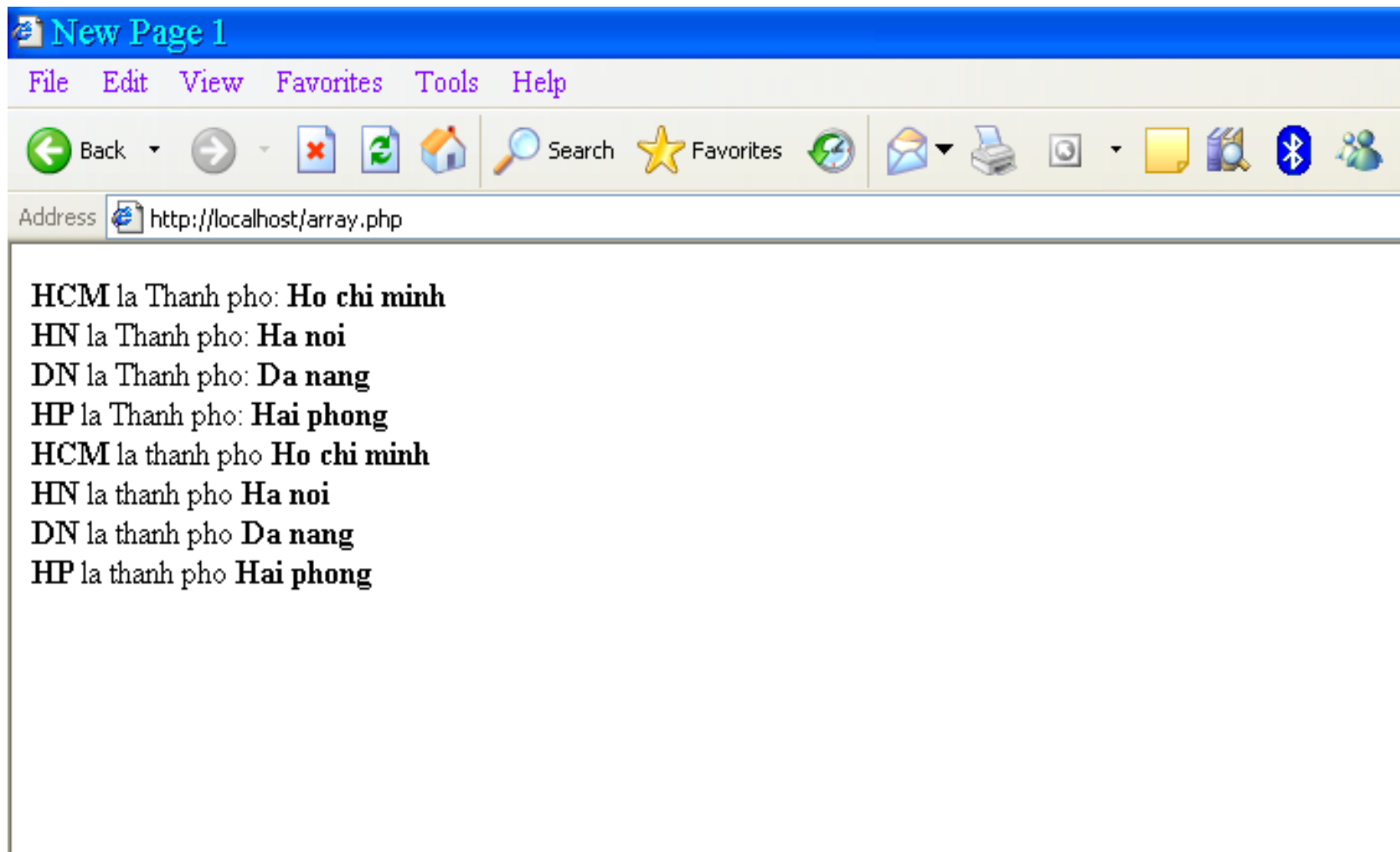
**Ví dụ:** Xét đoạn code sau

```
<?PHP    $tpho=array("HCM"=>"Ho Chi Minh",
                "HN"=>"Ha Noi",    "HP"=>"Hai Phong",
                "DN"=>"Da Nang");
    while(list($key,$value)=each($tpho))
        echo "<b>$key</b> la Thanh pho:
    <b>$value</b><br>";
    reset($tpho);
    while($row=each($tpho)
        echo "<b>". $row["$key"]. "</b> la thanh
    pho <b>". $row["$value"]. "</b><br>";
?>
```





## Làm việc với mảng



## Sắp xếp mảng

---

◆ Đối với mảng một chiều, sau khi khai báo và gán giá trị cho các phần tử mảng, để sắp xếp mảng ta có một số hàm sau:

- ◆ Hàm sort()
- ◆ Hàm rsort()
- ◆ Hàm asort()
- ◆ Hàm arsort()
- ◆ Hàm ksort()
- ◆ Hàm krsort()

## Hàm sort()

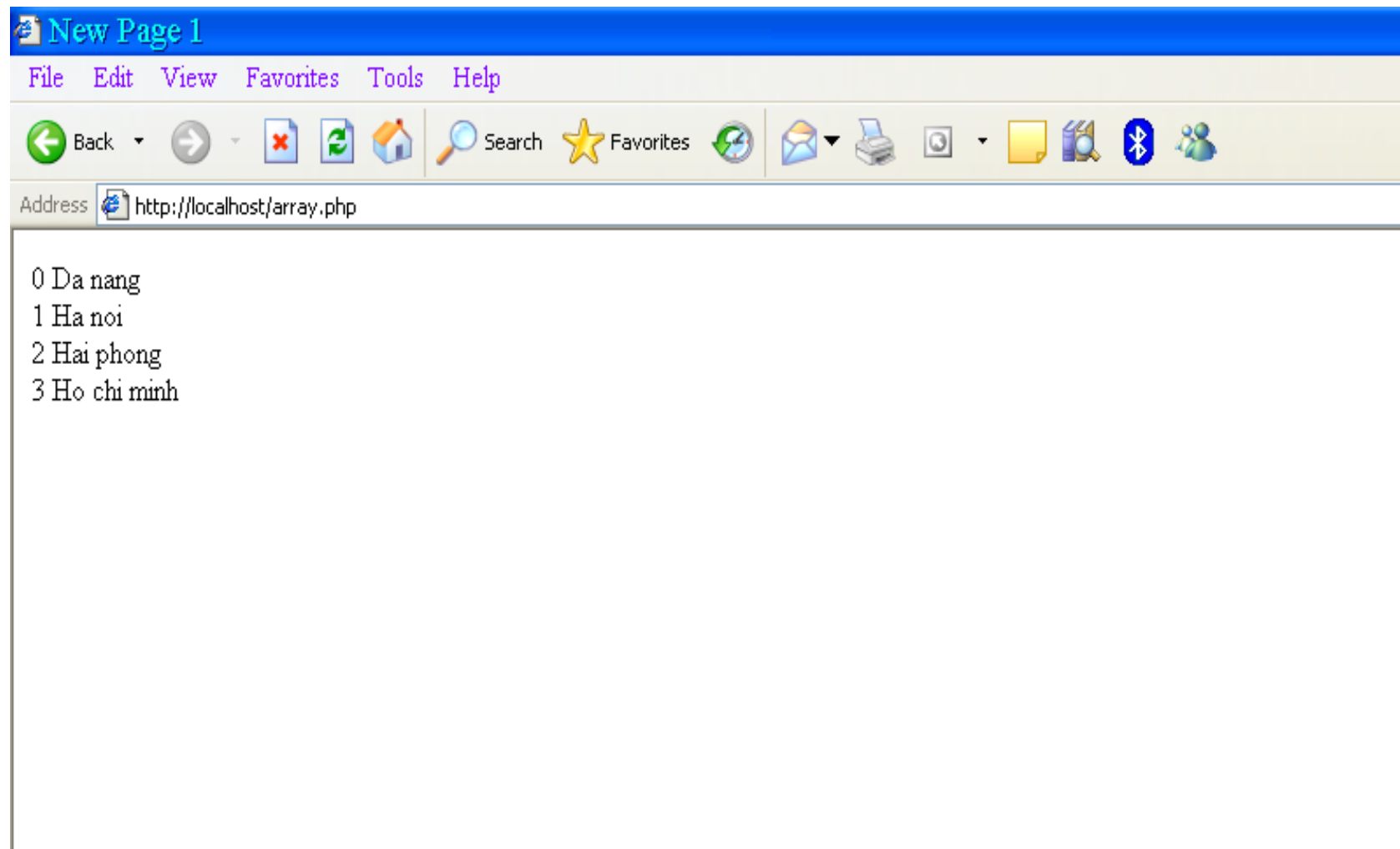
- ◆ Sắp xếp mảng theo chiều tăng dần của giá trị các phần tử mảng nhưng chỉ số tương ứng bị thay đổi

Cú pháp: `sort($biến_mảng)`

**Ví dụ:** sử dụng hàm `sort()`

```
<?PHP $tp=array("HCM"=>"Ho chi minh", "HN"=>"Ha noi",  
                "DN"=>"Da nang", "HP"=>"Hai phong");  
sort($tp);  
while ($row=each($tp))  
{  
    echo $row["key"]."\t";  
    echo $row["value"]."<br>";  
}
```

# Hàm sort()



## Hàm rsort()

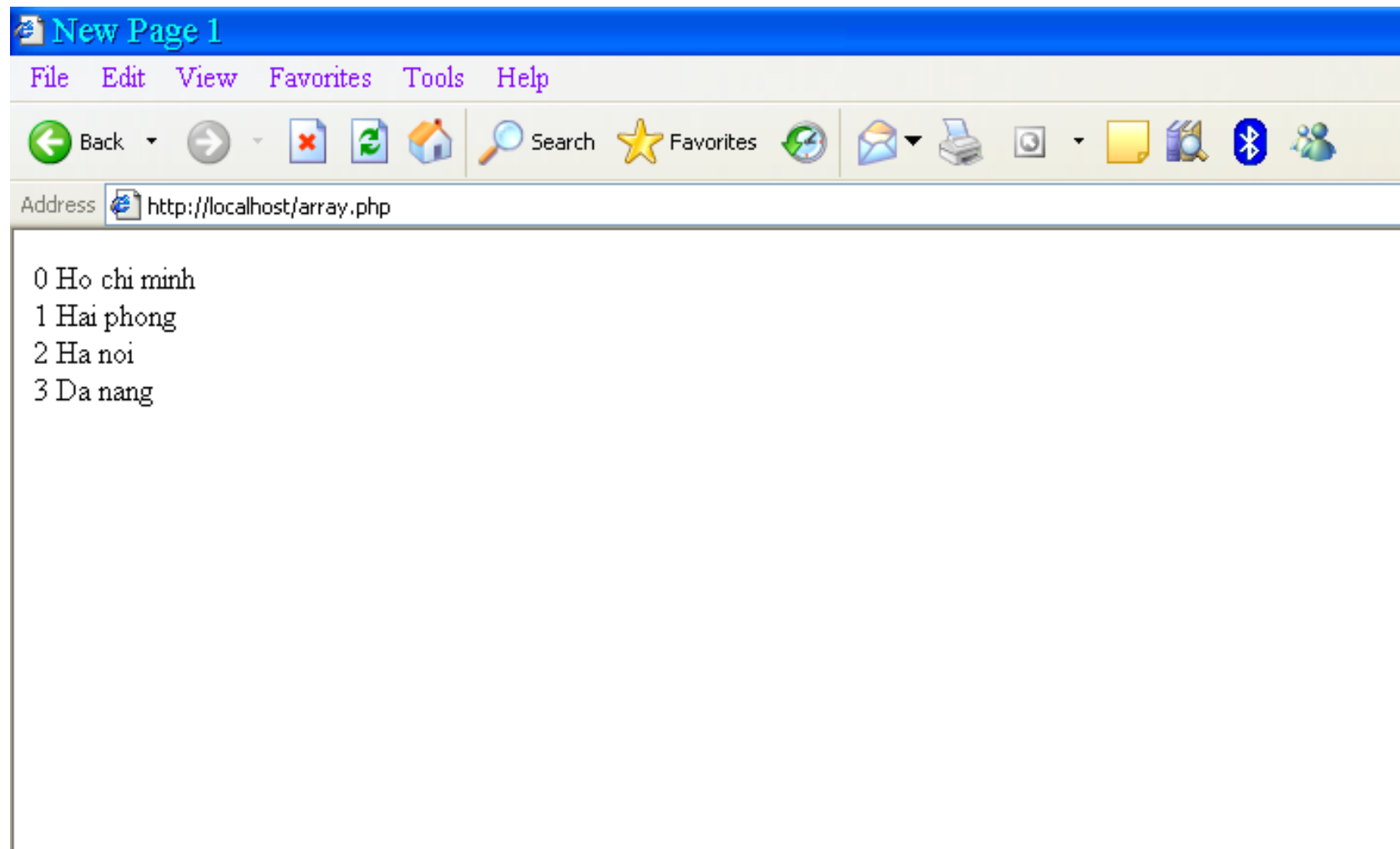
- ◆ Sắp xếp mảng theo chiều giảm dần của giá trị các phần tử mảng nhưng chỉ số tương ứng bị thay đổi

Cú pháp: `rsort($biến_mảng)`

**Ví dụ:** sử dụng hàm rsort

```
<?PHP $tp=array("HCM"=>"Ho chi minh", "HN"=>"Ha noi",  
                "DN"=>"Da nang", "HP"=>"Hai phong");  
rsort($tp);  
while ($row=each($tp))  
{  
    echo $row["key"]."\t";  
    echo $row["value"]."<br>";  
}  
?>
```

# Hàm rsort()





## Hàm asort()

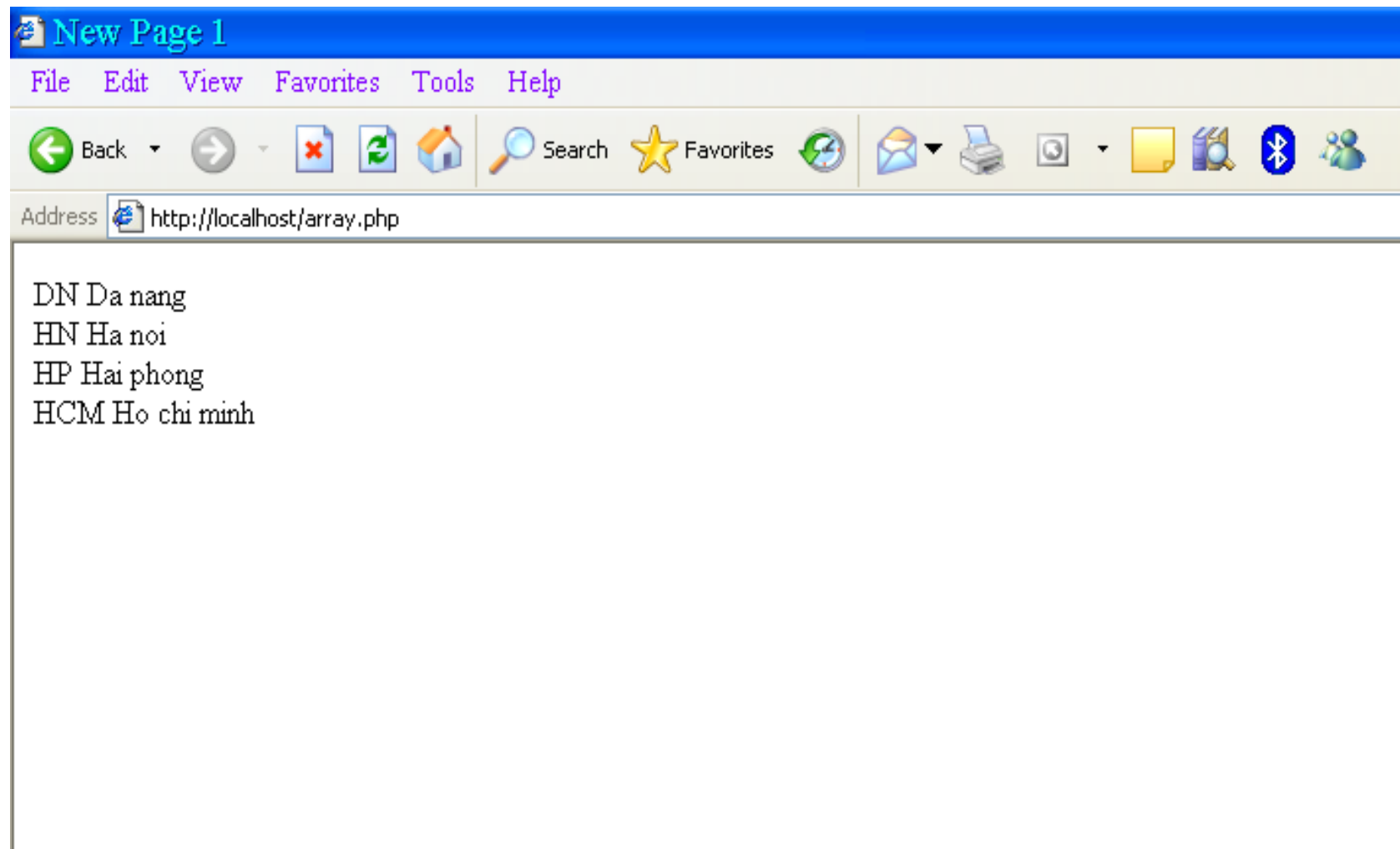
- ◆ Sắp xếp mảng theo chiều tăng dần của giá trị các phần tử mảng nhưng chỉ số vẫn giữ nguyên

Cú pháp: `asort($biến_mảng)`

**Ví dụ:** sử dụng hàm asort

```
<?PHP $tp=array("HCM"=>"Ho chi minh", "HN"=>"Ha noi",  
                "DN"=>"Da nang", "HP"=>"Hai phong");  
asort($tp);  
while ($row=each($tp))  
{  
    echo $row["key"]."\t";  
    echo $row["value"]."<br>";  
}
```

# Hàm asort()





## Hàm arsort()

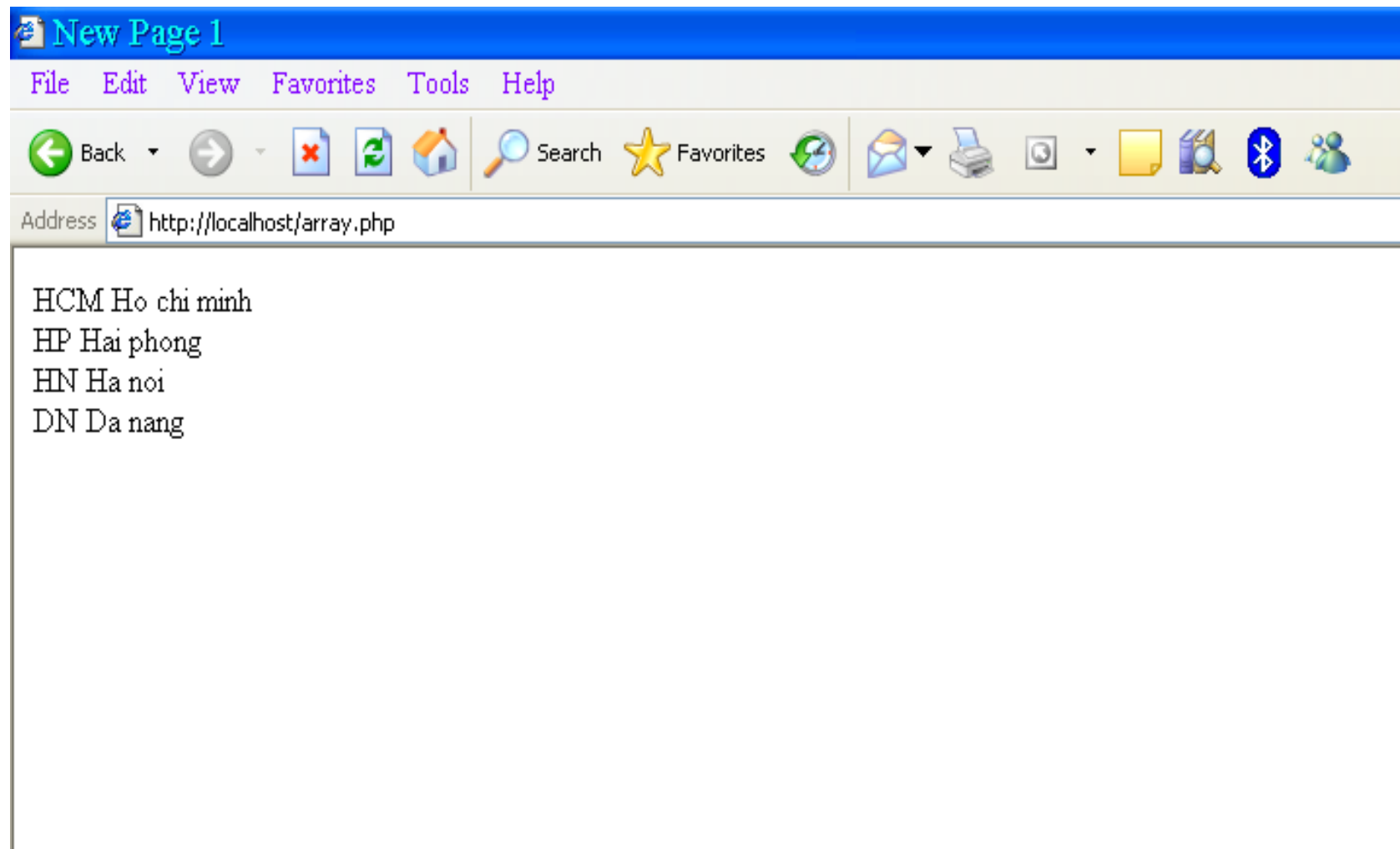
- ◆ Sắp xếp mảng theo chiều giảm dần của giá trị các phần tử mảng nhưng chỉ số vẫn giữ nguyên

Cú pháp: `arsort($biến_mảng)`

**Ví dụ:** sử dụng hàm arsort

```
<?PHP $tp=array("HCM"=>"Ho chi minh", "HN"=>"Ha noi",  
                "DN"=>"Da nang", "HP"=>"Hai phong");  
arsort($tp);  
while ($row=each($tp))  
{  
    echo $row["key"]."\t";  
    echo $row["value"]."<br>";  
}  
?>
```

# Hàm arsort()



## Hàm ksort()

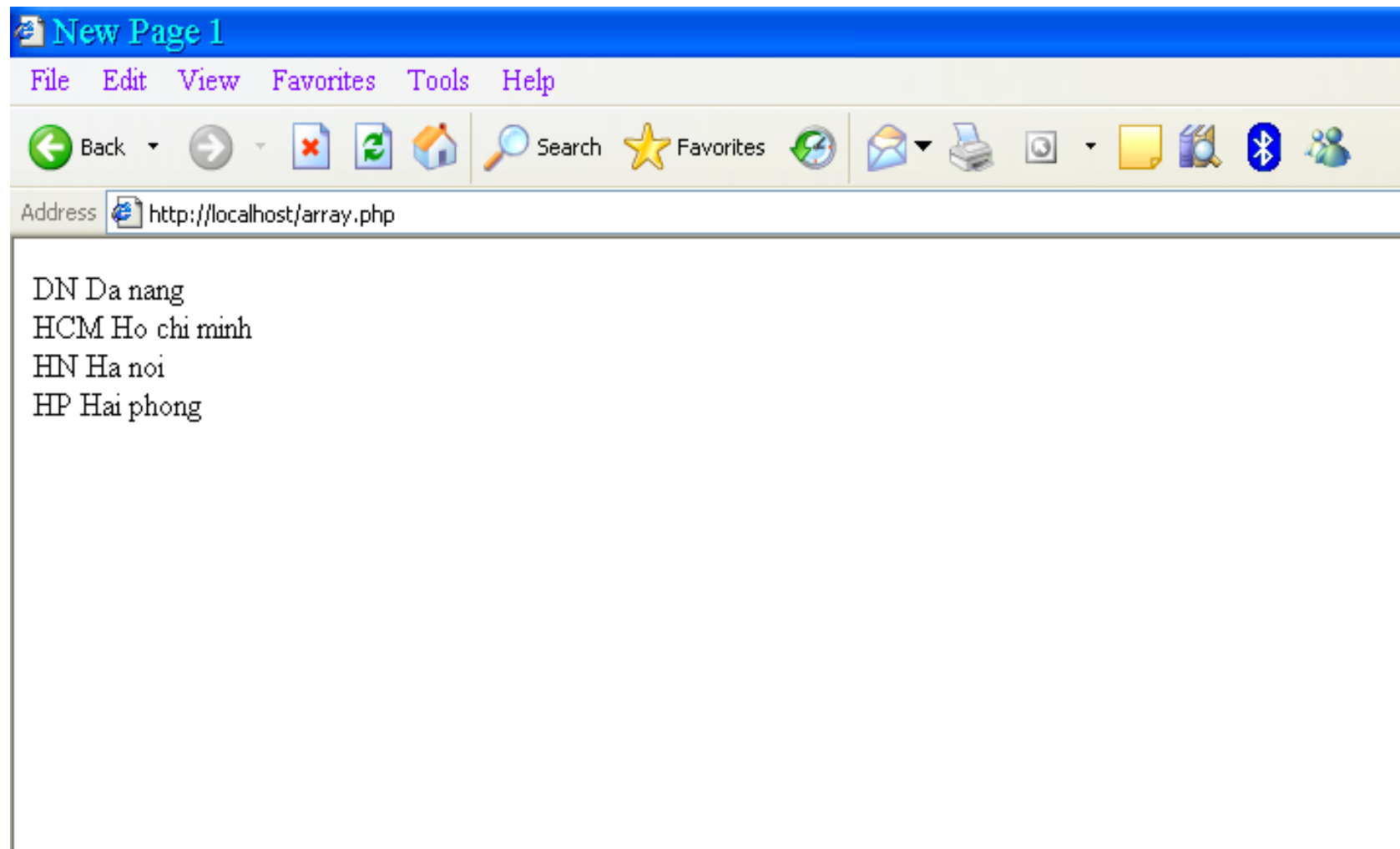
---

◆ Sắp xếp mảng theo chiều tăng dần của chỉ số mảng  
Cú pháp: ksort(\$biến\_mảng)

**Ví dụ:** sử dụng hàm ksort

```
<?PHP $tp=array("HCM"=>"Ho chi minh", "HN"=>"Ha noi",  
                "DN"=>"Da nang", "HP"=>"Hai phong");  
ksort($tp);  
while ($row=each($tp))  
{  
    echo $row["key"]."\t";  
    echo $row["value"]."<br>";  
}  
?>
```

# Hàm ksort()



## Hàm krsort()

---

- ◆ Sắp xếp mảng theo chiều giảm dần của chỉ số mảng

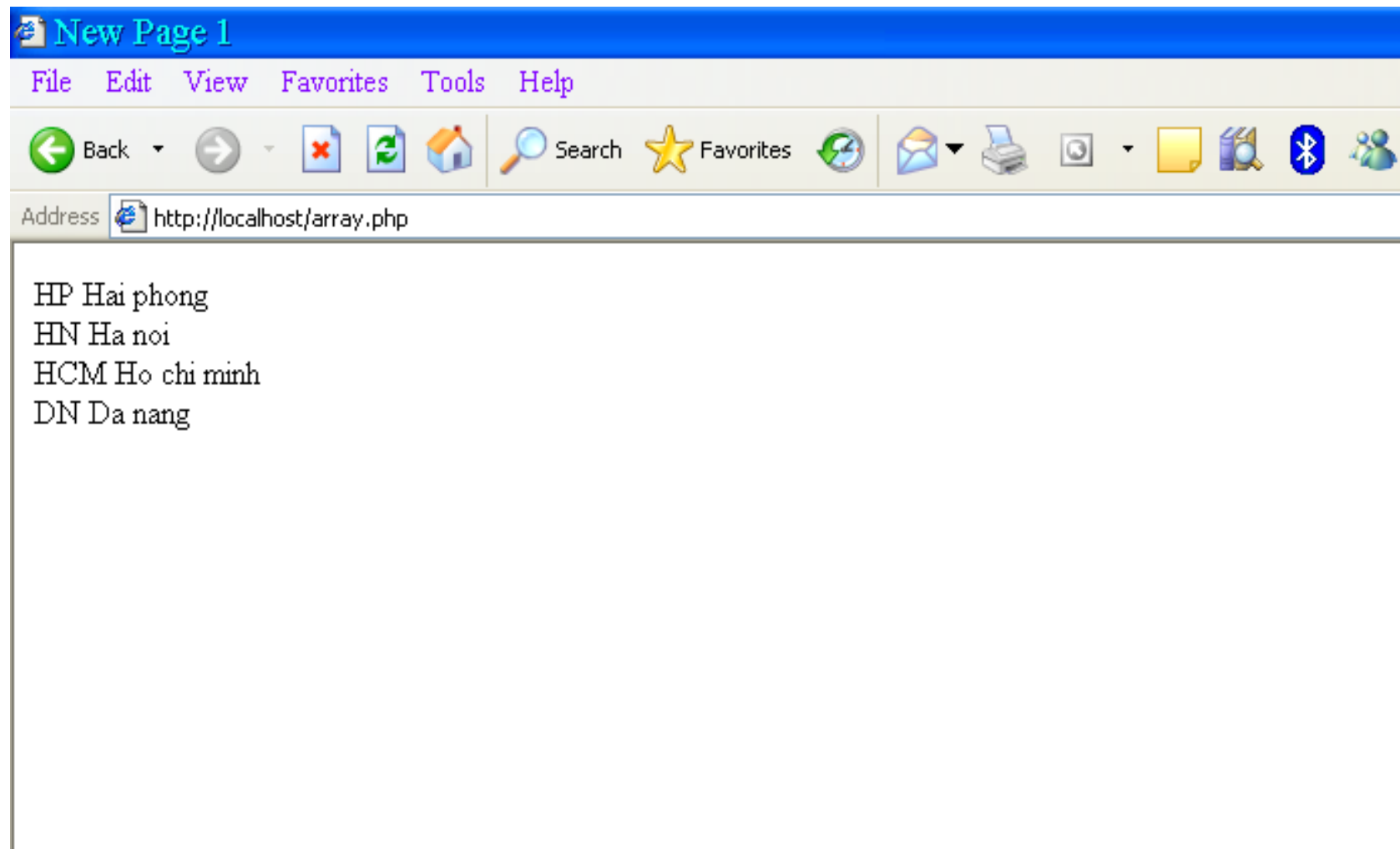
Cú pháp: krsort(\$biến\_mảng)

**Ví dụ:** sử dụng hàm krsort

```
<?PHP $tp=array("HCM"=>"Ho chi minh", "HN"=>"Ha noi",  
                "DN"=>"Da nang", "HP"=>"Hai phong");  
krsort($tp);  
while ($row=each($tp))  
{  
    echo $row["key"]."\t";  
    echo $row["value"]."<br>";  
}  
?>
```

## Hàm krsort()

---





## 3.3 mảng hai chiều

---



**Khai báo mảng**



**Làm việc với phần tử mảng**



**Sắp xếp mảng**

Cú pháp:

◆ Khai báo mảng có chỉ số kiểu number  
\$biến\_mảng=array(array(các giá trị mảng 1),  
array(các giá trị mảng 2), ....);  
Hoặc:\$biến\_mảng[0][0]=giá trị;  
\$biến\_mảng[0][1]=giá trị; ....

**Ví dụ:** Ta có các khai báo

```
$Lop=array(array("LT01A", "LT01B", "MM01",  
"DL01"), array("LT02A", "LT02B", "MM02A",  
"MM02B", "HT02"));  
$Lop[0][0]="LT01A"; $Lop[0][1]="LT01B"; ...
```





## Khai báo mảng

Cú pháp:



Khai báo mảng có chỉ số kiểu associative

```
$biến_mảng=array(“chỉ số h1”=>array(“chỉ số c1”=>giá trị, “chỉ số c2”=>giá trị 2, ...), “chỉ số h2”=>array(“chỉ số c1”=>giá trị, “chỉ số c2”=>giá trị, ...), ...);
```

Hoặc: `$biến_mảng[“chỉ số h1”][“chỉ số c1”]=giá trị 1;`

`$biến_mảng[“chỉ số h1”][“chỉ số c2”]=giá trị 2;`

....

**Ví dụ:** Ta có các khai báo

```
$tpho=array(“HCM”=>array(“Ten”=>“HoChiMinh”, “dan so”=>10000000),  
            “HN”=>array(“Ten”=>“HaNoi”, “dan so”=>6000000),  
            “HP”=>array(“Ten”=>“Hai Phong”), “DN”=>array(“Ten”=>“Da  
            Nang”));
```

```
$tpho[“HCM”][“Ten”]=“HoChiMinh”; $tpho[“HN”][“dan so”]=6000000;
```

...



## Làm việc với mảng

- ◆ Có thể thao tác với các phần tử mảng hai chiều thông qua tên biến\_mảng và hai chỉ số hàng, cột

Cú pháp: `$biến_mảng[“chỉ số hàng”][“chỉ số cột”];`

**Ví dụ:** Ta có đoạn code sau:

```
<?PHP
```

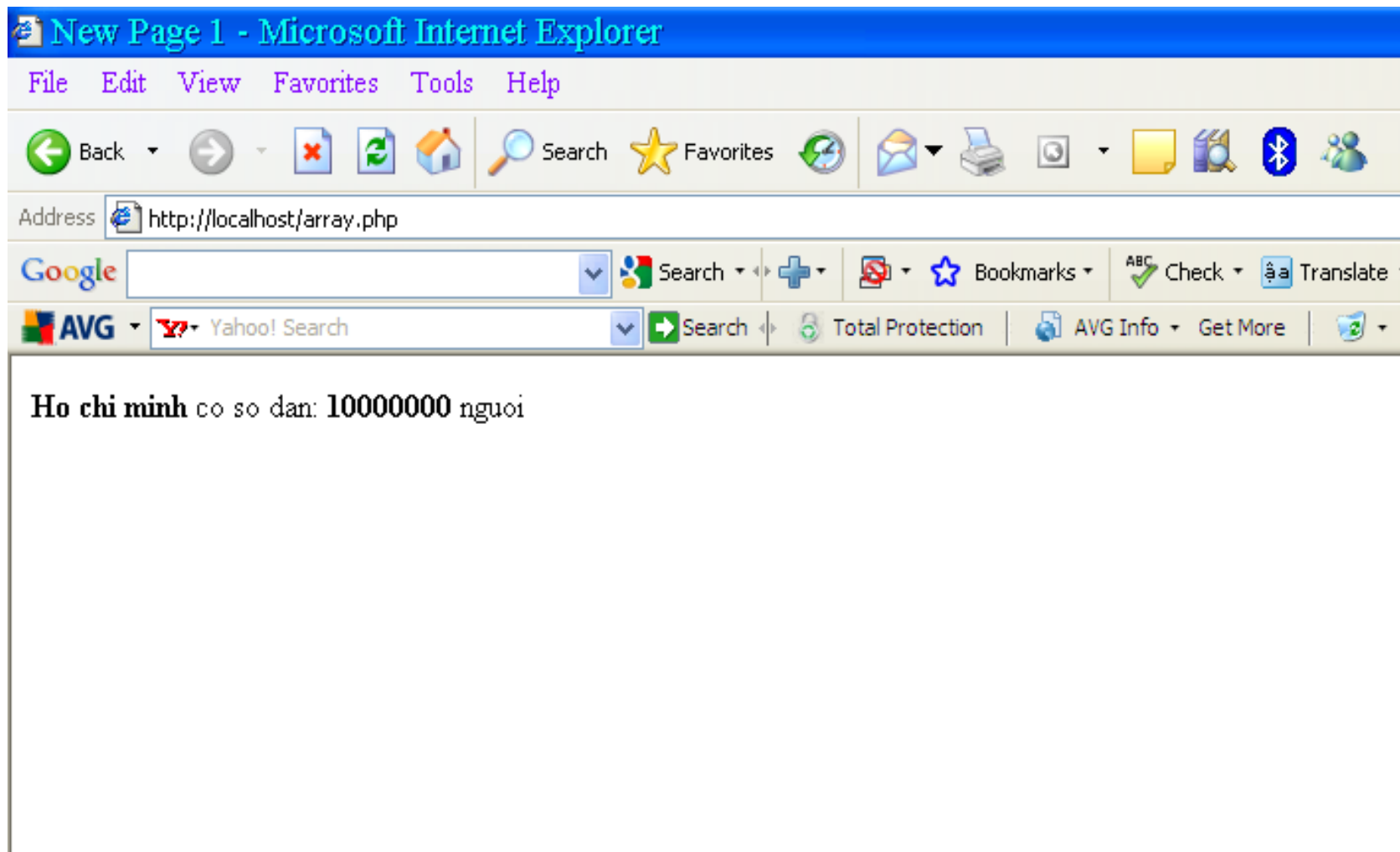
```
$tpho=array("HCM"=>array("Ten"=>"Ho chi minh", "dan  
so"=>10000000), "HN"=>array("Ten"=>"Ha noi","dan  
so"=>6000000), "DN"=>array("Ten"=>"Da nang"),  
"HP"=>array("Ten"=>"Hai phong"));
```

```
echo "<b>".$tpho["HCM"]["Ten"]."</b>co so dan:  
<b>".$tpho["HCM"]["dan so"]."</b>ngươi <br>";
```

```
?>
```



## Làm việc với mảng





## Làm việc với mảng

---

- ◆ Sử dụng hàm list() để duyệt tất cả các phần tử của mảng

**Ví dụ:** sử dụng list() cho ví dụ trên

```
<?PHP
```

```
while(list($hang,$tpho1)=each($tpho))
```

```
{    echo "<b> $hang: </b>";
```

```
    while (list($cot, $value)=each($tpho1)
```

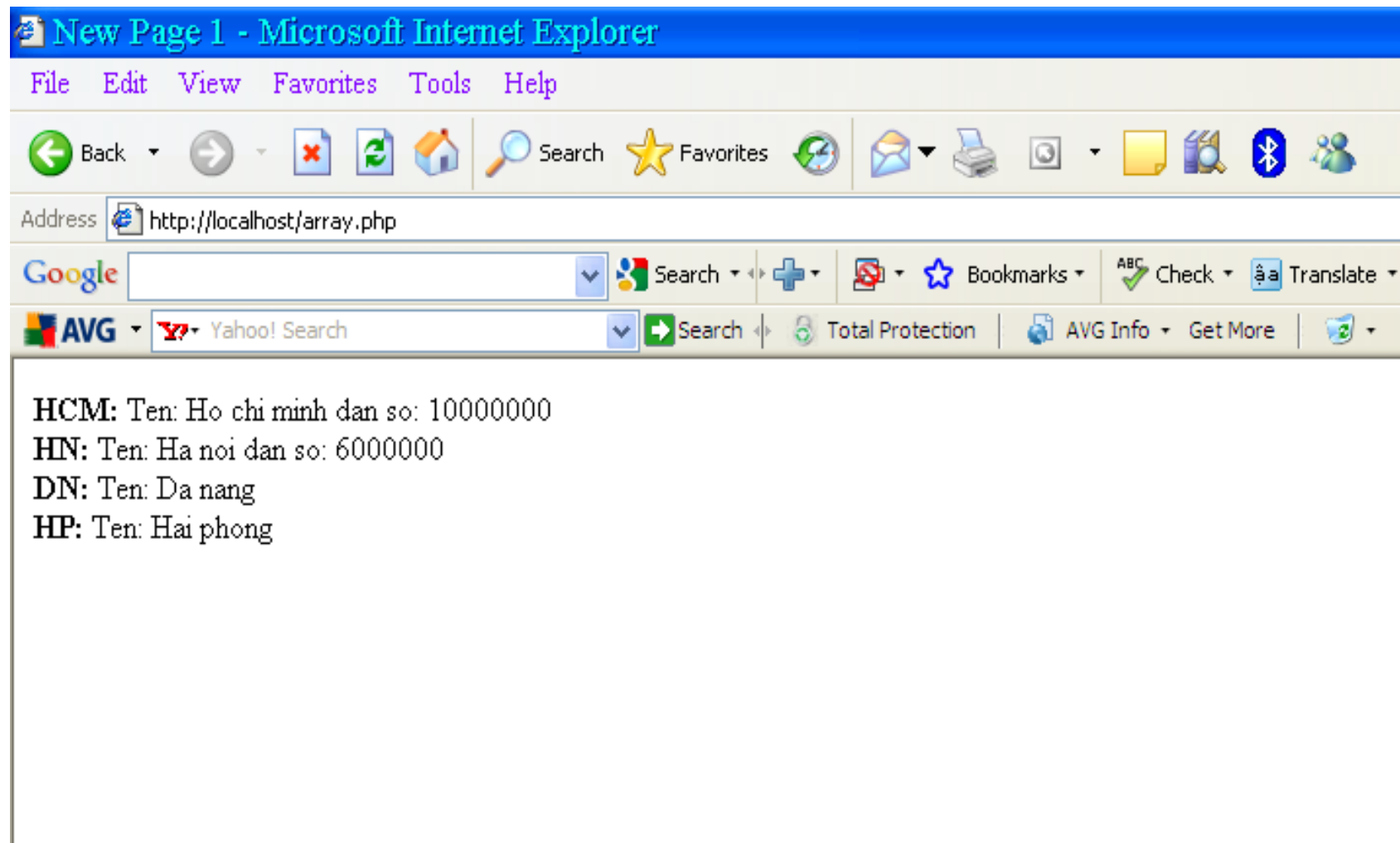
```
        echo "$cot: $value ";
```

```
    echo "<br>"
```

```
}
```

```
?>
```

## Làm việc với mảng



## Sắp xếp mảng

---

❖ Để sắp xếp mảng hai chiều, có thể sử dụng các hàm như:

❖ Hàm `usort()`

❖ Hàm `uasort()`

❖ Hàm `uksort()`

❖ Tuy nhiên, để kết hợp với hàm trên ta cần khai báo hàm so sánh phần tử như sau:

```
function my_sort($a, $b)
{if ($a == $b) return 0;
  return ($a > $b)? -1: 1;
}
```



## Hàm usort()

---

- ◆ Sắp xếp mảng theo thứ tự tăng dần của một cột nào đó nhưng chỉ số của các phần tử mảng thay đổi

Cú pháp: `usort($biến_mảng, my_sort)`



## Hàm usort()

---

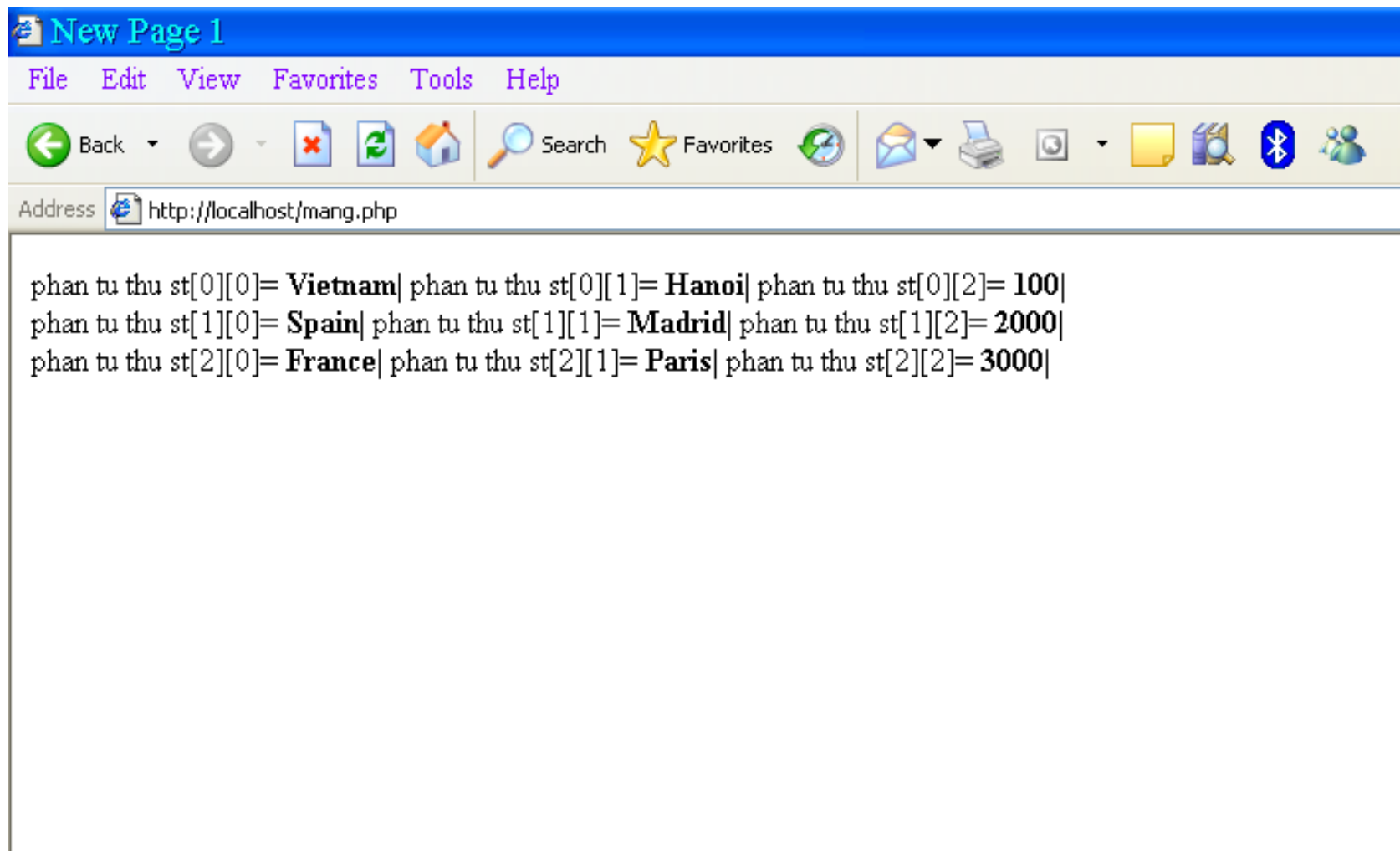
**Ví dụ:** xét đoạn code sau

```
<?PHP function SX($a, $b)
{ if ($a[1]==$b[1]) return 0;
  else if ($a[1]>$b[1]) return 1;
  else return -1;}

$st=array(array("Vietnam", "Hanoi", 100),array("France",
"Paris", 3000), array("Spain", "Madrid", 2000));
usort($st, SX);
for($i=0; $i<3; $i++)
{ for($j=0; $j<3; $j++)
  echo "phan tu thu st[".$i.""][$j.] = <b>".$st[$i][$j]."</b>|
";
  echo "<br>"; }      ?>
```



## Hàm usort()





## Hàm uasort()

---

- ◆ Sắp xếp mảng theo thứ tự tăng dần của một cột nào đó nhưng chỉ số của phần tử mảng không thay đổi theo

Cú pháp: `uasort($biến_mảng, my_sort)`

## Hàm uksort()

---

- ◆ Sắp xếp các phần tử mảng theo chỉ số của chúng. Sử dụng đối với các mảng có chỉ số kiểu associative

Cú pháp: `uksort($biến_mảng, my_sort)`

- Để sắp xếp mảng theo một cột nào đó ta có thể thay đổi nội dung của hàm `my_sort()`



### 3.4 các hàm xử lý trên mảng

- ◆ Hàm kiểm tra sự tồn tại của mảng
- ◆ Hàm di chuyển trên các phần tử
- ◆ Hàm trả về kích thước mảng
- ◆ Hàm tìm kiếm trên mảng
- ◆ Hàm thêm hoặc xóa phần tử mảng

## Hàm kiểm tra sự tồn tại của mảng

---

❖ Dùng hàm `is_array()` để kiểm tra một biến mảng nào đó có tồn tại hay không.

Cú pháp: **`is_array($biến_mảng);`**

Ví dụ:

```
$states = array("Florida");  
$state = "Ohio";  
printf("\$states is an array: %s <br />", (is_array($states) ? "TRUE" : "FALSE"));  
printf("\$state is an array: %s <br />", (is_array($state) ? "TRUE" : "FALSE"));
```

Executing this example produces the following:

---

```
$states is an array: TRUE  
$state is an array: FALSE
```

---



## Hàm di chuyển trên các phần tử

---

### **Các hàm thường dùng để di chuyển con trỏ trên các phần tử mảng**

- ◆ **Hàm current()**
- ◆ **Hàm next()**
- ◆ **Hàm each()**
- ◆ **Hàm prev()**
- ◆ **Hàm reset()**
- ◆ **Hàm end()**

## Hàm current()

---

Tác dụng: trả về giá trị của phần tử hiện tại trong mảng.

Cú pháp: **current(\$biển\_mảng)**

**Ví dụ:** xét câu lệnh sau:

```
<?PHP
$people = array("Peter", "Joe",
"Glenn", "Cleveland");
echo current($people) . "<br />";
?>
```

## Hàm next()

---

Tác dụng: trả về giá trị của phần tử kế tiếp; trả về false nếu hết mảng hoặc phần tử rỗng

Cú pháp: **next(\$biến\_mảng)**

**Ví dụ:** xét câu lệnh sau:

```
<?PHP
$people = array("Peter", "Joe",
"Glenn", "Cleveland");
echo current($people) . "<br />";
echo next($people);
?>
```



## Hàm each()

---

Tác dụng: trả về chỉ số và giá trị của phần tử hiện tại và chuyển con trỏ sang phần tử tiếp theo

Cú pháp: **each(\$biển\_mảng)**

**Ví dụ:** xét câu lệnh sau:

```
<?PHP

$people = array("Peter", "Joe",
"Glenn", "Cleveland");
while (list($key, $val) =
each($people))
    echo "$key => $val" ;

?>
```

## Hàm prev()

---

Tác dụng: trả về giá trị của phần tử ngay trước phần tử hiện tại; trả về false nếu đầu mảng hoặc phần tử rỗng

Cú pháp: **prev(\$biến\_mảng)**

**Ví dụ:** xét câu lệnh sau:

```
<?PHP
$people = array("Peter", "Joe",
"Glenn", "Cleveland");
echo current($people) . "<br />";
echo next($people) . "<br>" ;
echo prev($people) ;?>
```

## Hàm reset()

---

Tác dụng: chuyển con trỏ mảng về đầu mảng và trả về giá trị của phần tử đầu tiên của mảng

Cú pháp: **reset(\$biến\_mảng)**

**Ví dụ:** xét câu lệnh sau:

```
<?PHP
$people = array("Peter", "Joe",
"Glenn", "Cleveland");
echo current($people) . "<br />";
echo next($people) . "<br>" ; echo
reset($people);
?>
```

## Hàm end()

---

Tác dụng: chuyển con trỏ mảng về cuối mảng và trả về giá trị của phần tử cuối cùng của mảng

Cú pháp: **end(\$biến\_mảng)**

**Ví dụ:** xét câu lệnh sau:

```
<?PHP
$people = array("Peter", "Joe",
"Glenn", "Cleveland");
echo current($people) . "<br />";
echo next($people) . "<br>" ; echo
end($people) ;
?>
```

## Hàm trả về kích thước mảng

---

- ❖ Có thể sử dụng hàm `count()` hoặc hàm `sizeof()` để đếm tổng số phần tử có trong mảng

Cú pháp: **`count($biến_mảng)`** hoặc  
**`sizeof($biến_mảng)`**

```
<?PHP
```

```
$people = array("Peter", "Joe",  
"Glenn", "Cleveland");  
$result1 = count($people);  
$result2 = sizeof($people);  
echo $result1; echo $result2;
```

```
?>
```



## Hàm tìm kiếm trên mảng

---

- ◆ Hàm `in_array()`
- ◆ Hàm `array_key_exists()`
- ◆ Hàm `array_search()`

## Hàm in\_array()

---

Tác dụng: Tìm xem một giá trị nào đó có trong mảng hay không? Nếu có trả về true, ngược lại trả về false

Cú pháp: **in\_array**(“giá trị cần tìm”, \$biến\_mảng, **type**)

Trong đó: type là tham số nhận 2 giá trị true hoặc false. Nếu thiết lập là true thì tìm kiếm có phân biệt hoa, thường; chuỗi, số. Và mặc định là false

## Hàm in\_array()

---

**Ví dụ:** xét đoạn mã:

```
<?PHP
$people = array("Peter", "Joe",
"Glenn", "Cleveland");
if (in_array("Glenn", $people))
    echo "Match found";
else
    echo "Match not found";
?>
```





## Hàm array\_key\_exists()

---

Tác dụng: Kiểm tra xem một khóa nào đó có trong mảng hay không? Trả về true nếu tìm thấy, ngược lại trả về false

Cú pháp: array\_key\_exists("giá trị khóa", \$biến\_mảng)

**Ví dụ:** Xét đoạn mã sau:

```
<?PHP
    $a=array("a"=>"Dog","b"=>"Cat");
    if (array_key_exists("a",$a))
        echo "Key exists!";
    else
        echo "Key does not exist!";

?>
```

## Hàm array\_search()

---

Tác dụng: Kiểm tra xem một giá trị nào đó có trong mảng hay không? Trả về khóa tương ứng nếu tìm thấy

Cú pháp: array\_search(“giá trị cần tìm”, \$biến\_mảng, type)

**Ví dụ:** xét đoạn mã sau:

```
<?PHP
    $a=array("a"=>"Dog", "b"=>"Cat", "c"=>
    "Horse");
    echo array_search("Dog", $a);

    $b=array("a"=>"5", "b"=>5, "c"=>"15");
    echo array_search(5, $b, true);

?>
```



## Hàm thêm hoặc xóa phần tử mảng

---

- ◆ Hàm `array_unshift()`
- ◆ Hàm `array_push()`
- ◆ Hàm `array_shift()`
- ◆ Hàm `array_pop()`

## Hàm array\_unshift()

---

Tác dụng: thêm các phần tử vào đầu mảng. Giá trị trả về của hàm là số phần tử của mảng sau khi thêm.

Cú pháp: **array\_unshift(\$biến\_mảng, giá trị 1, ...)**

**Ví dụ:** xét đoạn mã sau:

```
<?PHP
```

```
    $a=array("a"=>"Cat", "b"=>"Dog");  
    echo array_unshift($a, "Horse");  
    echo $a;
```

```
?>
```

## Hàm array\_push()

---

Tác dụng: cũng giống như hàm array\_unshift() nhưng lại thêm vào cuối mảng

Cú pháp: **array\_push(\$biến\_mảng, giá trị 1, giá trị 2, ...)**

**Ví dụ:** xét đoạn mã sau:

```
<?PHP
    $a=array("a"=>"Cat", "b"=>"Dog");
    echo array_push($a, "Horse");
    echo $a;

?>
```

## Hàm array\_shift()

---

Tác dụng: loại bỏ phần tử đầu tiên của mảng. Kết quả trả về của hàm là giá trị phần tử vừa bị loại bỏ.

Cú pháp: **array\_shift(\$biến\_mảng)**

**Ví dụ:** xét đoạn mã sau:

```
<?PHP
    $a=array("a"=>"Dog","b"=>"Cat","c"
=>"Horse");
    echo array_shift($a);
    echo $a;

?>
```

## Hàm array\_pop()

---

Tác dụng: cũng giống như array\_shift nhưng loại bỏ phần tử cuối cùng

Cú pháp: **array\_pop(\$biến\_mảng)**

**Ví dụ:** xét đoạn mã sau:

```
<?PHP
    $a=array("a"=>"Dog", "b"=>"Cat", "c"=>"Horse");
    echo array_pop($a);
    echo $a;

?>
```

## 3.5 chuỗi ký tự (string)

---

- ◆ **Khái niệm chuỗi ký tự**
- ◆ **Khai báo chuỗi**
- ◆ **Làm việc với chuỗi**





## Khái niệm chuỗi ký tự (string)

---

- ❖ String trong PHP là một chuỗi các ký tự 1 byte. PHP không hỗ trợ Unicode, để làm việc với Unicode bạn phải sử dụng UTF8 với các hàm `utf8_encode()` – `utf8_decode()`
- ❖ Chuỗi ký tự của PHP hỗ trợ chiều dài rất lớn

## Khai báo chuỗi

---

- ◆ Trong PHP, chuỗi ký tự được khai báo theo 3 cách
  - ◆ Dấu nháy đơn (single quote)
  - ◆ Dấu nháy kép (double quote)
  - ◆ Heredoc

## ❖ Kiểu dấu nháy đơn

```
<?PHP
```

```
echo 'this is a simple string';
```

```
echo 'You can also have embedded newlines in  
strings this way as it is  
okay to do';
```

```
echo 'Arnold once said: "I\'ll be back"';
```

```
echo 'You deleted C:\\*. *?';
```

```
echo 'You deleted C:/*. *?';
```

```
echo 'This will not expand: \n a newline';
```

```
echo 'Variables do not $expand $either';
```

```
?>
```

## Khai báo chuỗi

---

- ◆ Kiểu dấu nháy kép: kiểu này giống với kiểu dấu nháy đơn nhưng có nhiều hỗ trợ cho các ký tự đặc biệt hơn
- ◆ Khi thực thi, PHP sẽ tìm và thay thế những ký tự đặc biệt được escape (như `\n`, `\t...`) cùng với các biến (nếu có) trong chuỗi
- ◆ Khi sử dụng dấu nháy đơn, giá trị của biến trong chuỗi, cùng với các ký tự đặc biệt cần escape sẽ không được in ra.



## Khai báo chuỗi

---

### Ví dụ:

```
<?PHP
    $a = 1;
    echo 'Biến \$a có giá trị là $a';
    echo "Biến \$a có giá trị là $a";
?>
```



## ❖ Kiểu Heredoc

```
<?PHP
$str = <<<EOA
Example of string <br>
spanning multiple lines<br>
using heredoc syntax.<br>
EOA;echo($str);
$name = "quang";
$d = date("d/m/y");
$str = <<<EOQ
This is a lecture of $name.<br>
Ngày $d<br>
EOQ;echo($str);
?>
```



## Làm việc với chuỗi

---

### ❖ Truyền biến kiểu chuỗi

```
<?PHP
```

```
$beer = 'Heineken';
```

```
echo "$beer's taste is great";
```

```
echo "He drank some $beers";
```

```
echo "He drank some ${beer}s";
```

```
echo "He drank some {$beer}s";
```

```
?>
```



## Làm việc với chuỗi

---

### ❖ Truy cập đến từng ký tự của chuỗi

```
<?PHP
    $str = "This is a test";
    echo $str[0];
    echo $str[1];
    echo $str[strlen($str)-1];
    echo "<br>$str";
    $str{strlen($str)-1} = "s";
    echo "<br>$str";
?>
```





## Làm việc với chuỗi

---

### ◆ Các toán tử trên string: toán tử cộng chuỗi

```
<?PHP
$txt1="Hello World!";
$txt2="What a nice day!";
echo $txt1 . " " . $txt2;
?>
```



## 3.6 các hàm xử lý trên chuỗi

---

- ◆ **Hàm định dạng chuỗi**
- ◆ **Hàm kết hợp hay tách chuỗi**
- ◆ **Hàm so sánh chuỗi**
- ◆ **Hàm tìm kiếm và thay thế chuỗi**

## Hàm định dạng chuỗi

---

- ◆ *strtoupper(biến\_chuỗi)*: Chuyển tất cả các ký tự trong chuỗi thành chữ hoa
- ◆ *strtolower(biến\_chuỗi)*: Chuyển tất cả các ký tự trong chuỗi thành chữ thường
- ◆ *ucfirst(biến\_chuỗi)*: Chuyển ký tự đầu tiên trong chuỗi thành chữ hoa
- ◆ *ucwords(biến\_chuỗi)*: Chuyển các ký tự đầu của mỗi từ trong chuỗi thành chữ hoa
- ◆ Hàm cắt ký tự trắng bên trái: *ltrim(biến\_chuỗi)*, *rtrim(biến\_chuỗi)* và *trim(biến\_chuỗi)*, *chop(biến\_chuỗi)*



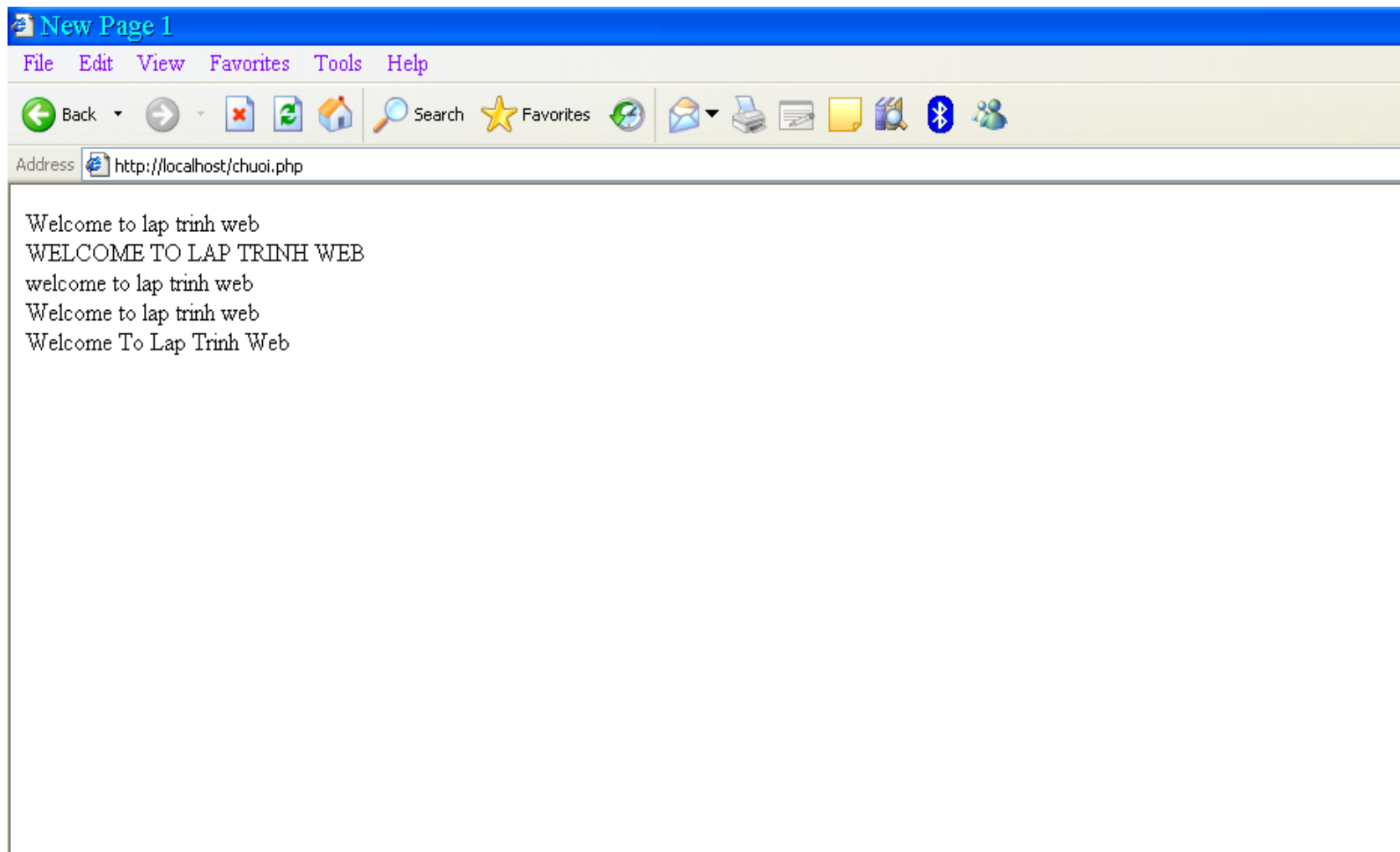
## Hàm định dạng chuỗi

---

```
<?PHP
    $str=" Welcome to lap trinh web
    ";
    echo $str; echo "<br>";
    echo strtoupper($str); echo
    "<br>";
    echo strtolower($str); echo
    "<br>";
    echo ucfirst($str); echo "<br>";
    echo ucwords($str); echo "<br>";
?>
```



# Hàm định dạng chuỗi



## Hàm tách hay kết hợp chuỗi

---

- ❖ *strtok(biến\_chuỗi, str)*: Tách biến chuỗi thành các chuỗi con, sử dụng chuỗi str làm chuỗi xác định cách tách
- ❖ *explode(str, biến\_chuỗi)*: Tách chuỗi thành mảng các chuỗi con, sử dụng str làm chuỗi xác định cách tách
- ❖ *substr(biến\_chuỗi, k [, n])*: Trả về chuỗi con từ biến chuỗi, tại vị trí k, lấy n ký tự.
- ❖ *implode(str, \$mang)*: Kết hợp các phần tử của mảng thành chuỗi, lấy str làm chuỗi liên kết



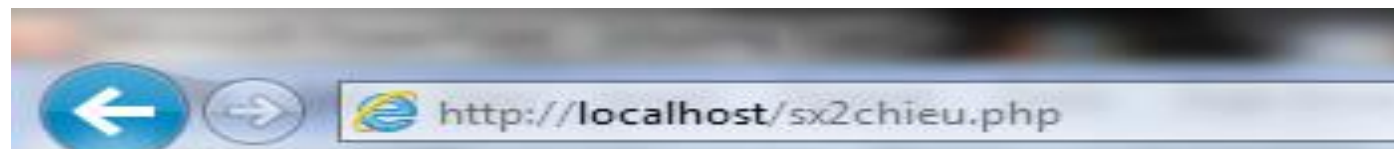
## Hàm tách hay kết hợp chuỗi

```
9 </pnp
10 $string = "Xin chao ban da den voi viethanit.edu.vn";
11 $str = $string;
12 $tok = strtok($string, " ");
13 while ($tok)
14     {
15         echo "Word= $tok<br />";
16         $tok = strtok(" ");
17     }
18 echo substr($str,24,7)."<br>";
19 $a=array();
20 $a=explode(" ", $str);
21 while($i=each($a))
22     echo $i["value"]."<br>";
23 ?>
```



## Hàm tách hay kết hợp chuỗi

---



```
Word= Xin  
Word= chao  
Word= ban  
Word= da  
Word= den  
Word= voi  
Word= viethanit.edu.vn  
viethan  
Xin  
chao  
ban  
da  
den  
voi  
viethanit.edu.vn
```





## Hàm so sánh chuỗi

---

- ❖ **(int) strcmp (str1, str2):** 0 nếu  $str1 == str2$ , n nếu  $str1 > str2$ , -n nếu  $str1 < str2$  (phân biệt chữ hoa, chữ thường, n là số ngẫu nhiên)
- ❖ **(int) strcasecmp (str1, str2):** 0 nếu  $str1 == str2$ , n nếu  $str1 > str2$ , -n nếu  $str1 < str2$  (không phân biệt chữ hoa, chữ thường, n là một giá trị ngẫu nhiên)
- ❖ **(int) strnatcmp (str1, str2):** 0 nếu  $str1 == str2$ , n nếu  $str1 > str2$ , -n nếu  $str1 < str2$  (phân biệt chữ hoa, chữ thường, n là số ngẫu nhiên)



## Hàm tách hay kết hợp chuỗi

---

```
9  <?php
10 $str1 = "Viethan";
11 $str2 = "VietHan";
12 echo (int) strcmp($str1, $str2);
13 echo (int) strcasecmp($str1, $str2);
14 echo (int) strnatcmp($str1, $str2);
15 ?>
```



## Hàm tìm kiếm và thay thế chuỗi

---

- ❖ ***strpos(biến\_chuỗi, str)***: trả về vị trí chuỗi con str xuất hiện đầu tiên trong biến\_chuỗi, nếu không tìm thấy trả về rỗng.
- ❖ ***strstr(str1, biến\_chuỗi)***: trả về str1 nếu tìm thấy str1 trong biến\_chuỗi. Ngược lại, trả về false.
- ❖ ***str\_replace(str1, str2, biến\_chuỗi)***: thay chuỗi con str1 bằng str2 trong biến\_chuỗi.
- ❖ ***substr\_replace(biến\_chuỗi, str, n)***: thay phần chuỗi tính từ vị trí n trong biến\_chuỗi bằng chuỗi str.



## Hàm tìm kiếm và thay thế chuỗi

---

```
8  <!--
9  <?php
10
11  $str="Do you have any jobs, please let me know you jobs as soon as possible.";
12  echo $str."<br>";
13  echo str_replace("job","BOOK",$str)."<br>";
14  echo substr_replace($str,"PHP PROGRAMING.",41)."<br>";
15
16
17  ?>
```

1. Chạy các ví dụ trong slides để xem kết quả hiện thị lên trình duyệt.
2. Viết chương trình nhập vào một chuỗi ký tự bất kỳ.  
Sau đó:
  - ◆ Đếm số từ có trong chuỗi (các từ cách nhau bởi các khoảng trắng)
  - ◆ Đếm số các ký tự xuất hiện trong chuỗi (trong trường hợp có phân biệt chữ hoa chữ thường và không phân biệt chữ hoa chữ thường)
3. Tách chuỗi nhập vào thành các phần:
  - ◆ Chuỗi địa chỉ email thành username và domain
  - ◆ Chuỗi số ngày tháng thành ngày, tháng, năm.
  - ◆ Chuỗi họ tên đầy đủ thành họ và tên



# Chương 4: Hàm



## Chương 4: hàm

---

**4.1 GIỚI THIỆU VỀ HÀM**

**4.2 CÁC HÀM XÂY DỰNG SẴN**

**4.3 HÀM TỰ TẠO**

**4.4 SỬ DỤNG LẠI MÃ PHP**



## 4.1. Giới thiệu về hàm

---



**Khái niệm hàm**



**Phân loại hàm**





## Khái niệm hàm

---

- ❖ Hàm dùng để thực hiện một công việc nào đó. Chẳng hạn như: lấy ngày tháng của hệ thống, kết nối tới CSDL, thông báo lỗi, ...
- ❖ Tất cả các hàm trong PHP đều có dạng cơ bản sau:  
**Tên\_hàm(/Đối số/)**

Trong đó:

- ☐ Đối số: Tùy thuộc vào các hàm khác nhau mà có thể có một hoặc nhiều đối số hoặc không có đối số.
- ☐ Cần nắm được kết quả trả về của hàm
- ☐ Hàm chỉ được thực thi khi có lời gọi hàm
- ☐ Có thể gọi hàm ở bất kỳ đâu trong trang



## Phân loại hàm

---

- ◆ Hàm trong PHP được phân làm hai loại
  - ◆ Các hàm xây dựng sẵn: Là các hàm dùng để thực hiện các chức năng cơ bản của PHP
  - ◆ Các hàm do người dùng tự định nghĩa: được xây dựng trong quá trình viết code



## 4.2 các hàm xây dựng sẵn

---

- ◆ **Các hàm ngày giờ**
- ◆ **Các hàm thao tác với MySQL**
- ◆ **Các hàm thao tác với file**
- ◆ **Các hàm lọc filter**
- ◆ **Các hàm thông báo lỗi**
- ◆ **Các hàm xử lý ngoại lệ**



## Các hàm ngày giờ

---

- ◆ Dùng để lấy hoặc thiết lập ngày tháng trên Server
- ◆ Một số hàm ngày giờ cơ bản:
  - ◆ Hàm date()
  - ◆ Hàm time()
  - ◆ Hàm getdate()
  - ◆ Hàm checkdate()
  - ◆ Hàm mktime()

## Hàm date()

---

- ◆ Trả về ngày, giờ dựa trên ngày giờ của Server theo định dạng chỉ ra.

Cú pháp: **date(format, [timestamp])**

Trong đó:

**Format:** Là ký tự định dạng đầu ra của ngày, giờ

**timestamp:** Tùy chọn

## Hàm date()

---

- ◆ Một số ký tự định dạng:
  - ◆ **a**: am hoặc pm
  - ◆ **A**: AM hoặc PM
  - ◆ **B**: mẫu giờ Internet
  - ◆ **d**: ngày của tháng (01 ... 31)
  - ◆ **D**: ngày của tuần (Mon ... Sun)
  - ◆ **F**: Tháng có tên đầy đủ (January ... December)
  - ◆ **g**: giờ (1 ... 12)
  - ◆ **G**: giờ (0 ... 23)
  - ◆ **h**: giờ (01 ... 12)
  - ◆ **H**: giờ (00 ... 23)
  - ◆ **i**: phút (00 ... 59)

## Hàm date()

---

- ◆ **j**: ngày của tháng (1 ... 31)
- ◆ **l**: ngày của tuần (Monday ... Sunday)
- ◆ **L**: kiểm tra năm nhuận (1: nếu năm nhuận, 0: ngược lại)
- ◆ **m**: tháng (01 ... 12)
- ◆ **M**: tháng (Jan ... Dec)
- ◆ **n**: tháng (1 ... 12)
- ◆ **s**: giây (00 ... 59)
- ◆ **S**: hậu tố thứ tự của tiếng anh (st, nd, rd, th)
- ◆ **t**: số ngày của tháng (28 ... 30)
- ◆ **T**: thiết lập timezone của máy
- ◆ **w**: ngày của tuần dạng số (0: Sunday, ..., 6: Saturday)
- ◆ **Y**: năm 4 số (2009)
- ◆ **y**: năm 2 số (09)
- ◆ **z**: ngày của năm (0 ... 365)



## Hàm date()

---

### Ví dụ 1:

```
<body>
<h1> VI DU VE HAM DATE() </h1>
<?PHP
echo(date("l") . "<br />");
echo(date("l dS \of F Y h:i:s A") . "<br />");
echo("Oct 3,1975 was on a ".date("l",
    mktime(0,0,0,10,3,1975))."<br />");
?>
</body>
```





## Hàm date()

---

### Ví dụ 2:

```
<?PHP
```

```
$nextWeek = time() + (7 * 24 * 60 * 60);  
echo 'Now: ' . date('Y-m-d') . "\n";  
echo 'Next Week: ' . date('Y-m-d', $nextWeek) . "\n";  
// or using strtotime()  
echo 'Next Week: ' . date('Y-m-d', strtotime('+1 week')) . "\n";  
?> </body>
```

## Hàm time()

---

- ◆ Trả về số giây đã qua tính từ 00:00:00 GMT ngày 1/1/1970

Cú pháp: **time()**

Ví dụ:

```
<body>
<h1> VI DU VE HAM TIME() </h1>
<?PHP
    $t=time();
    echo($t . "<br />");
    echo(date("D F d Y",$t));
?>
</body>
```

## Hàm getdate()

---

- ◆ Trả về một mảng gồm các phần tử như sau:
  - [seconds] – Giây
  - [minutes] - phút
  - [hours] - giờ
  - [mday] – ngày của tháng
  - [wday] – ngày của tuần
  - [year] - năm
  - [yday] – ngày của năm
  - [weekday] – tên của thứ trong tuần
  - [month] – tên của tháng

Cú pháp: **getdate()**



## Hàm getdate()

---

### Ví dụ:

```
<h1> VI DU HAM GETDATE () </h1>
<?PHP
    $my_t=getdate(date("U"));
    print("$my_t[hours]: $my_t[minutes]:
        $my_t[seconds]\t");
    print("$my_t[weekday], $my_t[month]
        $my_t[mday], $my_t[year]");
?>
```

## Hàm checkdate()

---

- ❖ Dùng để kiểm tra ngày, tháng, năm nào đó có hợp lệ không. Nếu có trả về true, ngược lại trả về false

Cú pháp: **checkdate(month, day, year)**

**Ví dụ:**

```
<h1> Ví dụ hàm checkdate() </h1>
```

```
<?PHP
```

```
    echo (checkdate(12, 31, 2000));
```

```
    echo (checkdate(2, 29, 2003));
```

```
    echo (checkdate(2, 29, 2004));
```

```
?>
```

## Hàm mktime()

---

❖ Dùng để tính toán ngày hợp lệ.

Cú pháp: **mktime(hour, minute, second, month, day, year)**

Ví dụ:

```
<h1> Ví dụ hàm mktime() </h1>
```

```
<?PHP
```

```
echo (date ("M-d-Y", mktime (0, 0, 0, 12, 36, 2001)) . "<br  
>");
```

```
echo (date ("M-d-Y", mktime (0, 0, 0, 14, 1, 2001)) . "<br  
>");
```

```
echo (date ("M-d-Y", mktime (0, 0, 0, 1, 1, 2001)) . "<br  
>");
```

```
echo (date ("M-d-Y", mktime (0, 0, 0, 1, 1, 99)) . "<br />");
```

```
?>
```



## Các hàm thao tác với MySQL

---

- ◆ Dùng để mở hoặc hủy kết nối, thao tác với CSDL MySQL
- ◆ Một số hàm cơ bản:
  - ◆ Hàm **MySQL\_connect()**
  - ◆ Hàm **MySQL\_close()**
  - ◆ Hàm **MySQL\_pconnect()**
  - ◆ Hàm **MySQL\_select\_db()**
  - ◆ Hàm **MySQL\_query()**

## Hàm MySQL\_connect()

---

◆ Dùng để mở kết nối tới CSDL chủ MySQL

Cú pháp: **MySQL\_connect(server, user, pwd)**

**Ví dụ:** Thực hiện kết nối tới máy cục bộ bằng tài khoản **MySQL\_user** và password là **MySQL\_pwd**

```
$con=mysql_connect("localhost","mysql_user","mysql_pwd");  
If (!$con)  
{die('Could not connect: ' . mysql_error());}
```

Có thể dùng địa chỉ IP thay cho tên server:

```
$con=mysql_connect("10.10.20.5","mysql_user","mysql_pwd");
```



## Hàm MySQL\_close()

---

◆ Dùng để đóng kết nối tới CSDL MySQL

Cú pháp: **MySQL\_close(connection)**

Ví dụ:

```
<?php
$con = mysql_connect("localhost","mysql_user","mysql_pwd");
if (!$con)
{die('Could not connect: ' . mysql_error());}
mysql_close($con);
?>
```

## Hàm MySQL\_pconnect()

---

- ◆ Cũng giống như hàm MySQL\_connect() nhưng có điểm khác.
- ◆ Tìm kết nối đã được mở trên cùng host, user, pass. Nếu một kết nối được tìm thấy thì nó sẽ không mở thêm kết nối mới
- ◆ Liên kết với MySQL không đóng khi mã thực thi kết thúc

Cú pháp: **MySQL\_pconnect(server, user, pwd)**

## Hàm MySQL\_select\_db()

---

◆ Dùng để chuyển từ CSDL này qua CSDL khác

Cú pháp: **MySQL\_select\_db(database, [connection])**

Ví dụ:

```
<?php
$con = mysql_connect("localhost", "peter", "abc123");
if (!$con)
    die('Could not connect: ' . mysql_error());
$db_selected=mysql_select_db("test_db", $con);
if (!$db_selected)
    die ("Can\'t use test_db : " . mysql_error());
mysql_close($con);
?>
```

## Hàm MySQL\_query()

---

◆ Gửi các câu truy vấn tới MySQL server

Cú pháp: **MySQL\_query(query, [connection])**

Ví dụ:

```
<?php
$con =mysql_connect("localhost","mysql_user","mysql_pwd");
if (!$con)
    die('Could not connect: ' . mysql_error());
$sql = "SELECT * FROM Person";
mysql_query($sql,$con);
mysql_close($con);
?>
```



## Các hàm thao tác với MySQL

---

- ◆ Ngoài ra, để thao tác với CSDL còn có một số hàm sau:
  - ◆ Hàm **MySQL\_fetch\_array(data, [array\_type])**: trả về bản ghi trong bảng dữ liệu như là một mảng kết hợp với các cột là khóa hoặc mảng numeric.
  - ◆ Hàm **MySQL\_affected\_rows([connection])**: trả về số bản ghi trong bảng bị ảnh hưởng bởi update, delete, insert
  - ◆ Hàm **MySQL\_errno([connection])**: trả về số của thông điệp lỗi từ hoạt động của MySQL.
  - ◆ Hàm **MySQL\_error([connection])**: trả về chi tiết thông điệp bị lỗi.



# Các hàm thao tác với MySQL

- ❖ Hàm **MySQL\_num\_rows(data)**: trả về số bản ghi trong bảng khi select.
- ❖ Hàm **MySQL\_result(data of query,[m, n])**: trả về giá trị trong bảng tương ứng với hàng m và cột n.
- ❖ Hàm **MySQL\_num\_fields([connection])**: trả về số trường của bảng.
- ❖ Hàm **MySQL\_ping()**: kiểm tra kết nối server. Nếu chưa có kết nối thì reconnect.
- ❖ Hàm **MySQL\_fetch\_assoc(data)**: Trả các bản ghi về như mảng kết hợp
- ❖ ...



## Các hàm thao tác với file

---

- ◆ Dùng để mở, đọc, ghi, dữ liệu vào file
- ◆ Một số hàm cơ bản:
  - ◆ Hàm **fopen(file\_name, format)**: dùng để mở file có tên file\_name theo định dạng format (r, r++, w, w++, a ...)
  - ◆ Hàm **fclose(biến\_file)**: dùng để đóng file đang mở
  - ◆ Hàm **feof(biến\_file)**: dùng để kiểm tra xem đã kết thúc file hay chưa
  - ◆ Hàm **fgets(biến\_file)**: dùng để đọc theo từng dòng
  - ◆ Hàm **fgetc(biến\_file)**: đọc theo từng ký tự
  - ◆ ...



## Các hàm thao tác với file

---

### Ví dụ

```
$file = fopen("thu.txt", "r") or exit("Unable to open file!");  
while(!feof($file))  
{  
    echo fgets($file). "<br />";  
    echo fgetc($file);  
}  
fclose($file);
```





## Các hàm thao tác với file

---

### Ví dụ

```
$file=fopen("thu.txt","a") or exit ("co loi");  
echo fwrite($file,"chào các bạn!");  
$file=fopen("thu.txt","r") or exit("unable to open the file");  
while (!feof($file))  
{  
    echo fgets($file)."<br/>";  
    //echo fgetc($file)."<br/>";  
}  
fclose($file);
```





## Các hàm lọc

---

### Ví dụ 1:

```
<?PHP
```

```
    $int = 123;
```

```
    if(!filter_var($int,  
FILTER_VALIDATE_INT)) echo  
    "Integer is not valid";
```

```
    else echo "Integer is valid";
```

```
?>
```



## Các hàm lọc

---

### Ví dụ 2:

```
<?PHP
$var=300;
$int_options =
    array("options"=>array("min_range"
=>0, "max_range"=>256));
if(!filter_var($var,
    FILTER_VALIDATE_INT,
    $int_options)) echo"Integer is not
    valid";
else
echo"Integer is valid";
?>
```



## Các hàm lọc

---

### Ví dụ 3:

```
<?PHP
if (!filter_input(INPUT_GET, "email",
    FILTER_VALIDATE_EMAIL))
echo "E-Mail is not valid";
else echo "E-Mail is valid";
?>
```



## Các hàm thông báo lỗi

---

- ◆ Dùng để thông báo và xử lý lỗi
- ◆ Một số hàm cơ bản:
  - ◆ Hàm **error\_log()**
  - ◆ Hàm **user\_error()**
  - ◆ Hàm **error\_get\_last()**
  - ◆ Hàm **set\_error\_handler()**
  - ◆ Hàm **restore\_error\_handler()**
  - ◆ ...



## Hàm error\_log()

---

- ❖ Truyền một lỗi đến error log chủ, một file hoặc một đích ở xa.

Cú pháp: **error\_log(error,[type, destination, header])**

Ví dụ:

```
<?PHP
    $test=2;
    if ($test>1)
    {error_log("A custom error has been
    triggered",
    1,"someone@example.com","From:
    webmaster@example.com");
    }

?>
```

## Hàm user\_error()

---

◆ Dùng để tạo thông báo lỗi người dùng định nghĩa

Cú pháp: **user\_error(error\_message)**

Ví dụ:

```
<?PHP
    $test=2;
    if ($test>1)
    {
        user_error("A custom error has been
        triggered");
    }

?>
```





## Hàm `error_get_last()`

---

◆ Thông báo lỗi xuất hiện cuối cùng. Kết quả trả về là một mảng gồm 4 khóa và giá trị:

[type] – loại lỗi

[message] – thông điệp lỗi

[file] – file có lỗi xuất hiện

[line] – dòng có lỗi xuất hiện

Cú pháp: **`error_get_last()`**



## Hàm error\_get\_last()

---

### Ví dụ:

```
<h1> Vi du ham error_get_last()</h1>
<?PHP
    echo $test;
    $arr=error_get_last();
    print("loi: $arr[type] <br> thong diep:
        $arr[message]<br> tap tin: $arr[file]<br>
        dong: $arr[line]");
?>
```



## Hàm set\_error\_handler()

---

❖ Xây dựng hàm người dùng để xử lý lỗi

Cú pháp: **set\_error\_handler(error\_function)**

Ví dụ:

```
<?PHP //error handler function
function customError($errno, $errstr, $errfile,
    $errline)
{echo "<b>Custom error:</b> [$errno] $errstr<br />";
echo " Error on line $errline in $errfile<br />";
echo "Ending Script";
die();}
set_error_handler("customError");
$test=2;
if ($test>1)trigger_error("A custom error has been
    triggered");
?>
```



## Hàm restore\_error\_handler()

- ◆ Khôi phục xử lý lỗi trước đó sau khi đã dùng hàm set\_error\_handler()

Cú pháp: **restore\_error\_handler()**

Ví dụ:

```
<?PHP
function customError($errno, $errstr, $errfile,
$errline)
{ echo "<b>Custom error:</b> [$errno] $errstr<br
/>";
echo " Error on line $errline in $errfile<br />";}
set_error_handler("customError");
$test=2;
if ($test>1)
{ trigger_error("A custom error has been
triggered");}
restore_error_handler();
if ($test>1)
{ trigger_error("A custom error has been
triggered");}
?>
```



## 4.3 các hàm tự tạo

---



**Cách xây dựng hàm trong PHP**



**Một số lưu ý khi xây dựng hàm**



## Cách xây dựng hàm trong PHP

---

Cú pháp:

```
function Ten_ham(ds tham số nếu có)
{
    thân hàm;
    [return giá trị]
}
```

## Truyền tham số

---

PHP hỗ trợ truyền tham số vào hàm, truyền tham số dưới dạng tham chiếu (reference), truyền tham số với giá trị mặc định

```
<?PHP
function Tong_mang($input) { $tg=0;
foreach ($input as $value) $tg+=$value;
echo "tong=$tg";}

function Them_xau(&$string) {$string .= "gồm 3 tín chỉ.";}

$str = "Học phần lt web, ";
$input=array(1,2,2);
Tong_mang($input);
echo $str;
echo Them_xau($str)
?>
```

## Truyền tham số (2)

---

```
function Loai_cafe ($type = "cappuccino")
{   return "Making a cup of $type.\n";}
echo Loai_cafe (); echo Loai_cafe ("espresso");

function Loai_yogurt ($type = "acidophilus", $flavour) {   return
"Making a bowl of $type $flavour.\n";}

// Không thực thi đúng
echo Loai_yogurt ("raspberry");

function Loai_yogurt ($flavour, $type = "acidophilus") {
return "Making a bowl of $type $flavour.\n";}

// Thực thi
echo Loai_yogurt ("raspberry");
```



## Truyền tham số (3)

---

Ta có thể truyền tham số với số lượng không xác định.  
Khi đó sẽ sử dụng các hàm sau

**func\_num\_args()**: Số lượng tham số

**func\_get\_args()**: Lấy danh sách các tham số

```
<?PHP
function mwrite() {
    $arg = func_get_args();
    foreach($arg as $value) echo "$value\n";
}
mwrite("Khoa KHMT", "Truong V-H", "LT03A");
?>
```

## Giá trị trả về

Để trả về giá trị của hàm, sử dụng lệnh return. Có thể sử dụng bất cứ kiểu nào trong giá trị trả về của hàm (kể cả mảng và đối tượng)

```
<?PHP
    function square ($num) { return $num *
$num; }
    echo square (4);    // outputs '16'.
    function small_numbers() {return
array(0,1,2);}
    list ($zero, $one, $two) =
small_numbers();
    function &returns_reference() {return
$someref;}
    $newref =& returns_reference();
?>
```

## Biến hàm

Bạn có thể sử dụng các biến hàm (tức là bạn có thể gọi hàm dưới dạng `ten_bien()` ).

Biến hàm thường được sử dụng cho các hàm callback

```
<?PHP
function foo() {echo "In foo()<br>\n";}
function bar($arg = "") {echo "In bar(); argument was '$arg'.<br>\n";}
// This is a wrapper function around echo
function echoit($string) {echo $string;}
$func = 'foo';
$func(); // This calls foo()
$func = 'bar';
$func('test'); // This calls bar()
$func = 'echoit';
$func('test'); // This calls echoit()
?>
```

## Biến hàm (2)

---

```
<?PHP
class Foo {
    function Var() {
        $name = 'Bar';
        $this->$name(); // This calls the Bar() method
    }
    function Bar() {
        echo "This is Bar";
    }
}
$foo = new Foo();
$funcname = "Var";
$foo->$funcname(); // This calls $foo->Var()
?>
```



## 4.4 sử dụng lại mã PHP

---



**Vì sao phải sử dụng lại mã PHP**



**Cách sử dụng lại mã PHP**



## Vì sao sử dụng lại mã PHP?

---

- ◆ Chi phí
- ◆ Độ tin cậy
- ◆ Tính nhất quán



## Cách sử dụng lại mã PHP

---

- ◆ Để sử dụng lại mã PHP ta có thể dùng các khai báo sau:
  - ◆ Khai báo `require()`
  - ◆ Khai báo `include()`

## Khai báo require()

- ❖ Dùng để nhúng một trang này vào đoạn mã của một trang khác mà không cần phải viết lại mã.
- ❖ Ví dụ: Ta có trang Bienluan.PHP

```
1 <html>
2 <head>
3 <title>trang 2</title>
4 </head>
5 <body>
6 <?php
7     $a=$_POST['t1'];
8     $b=$_POST['t2'];
9         if($a==0)
10             if($b!=0)
11                 echo "PT vo nghiem";
12             else
13                 echo "PT vo so nghiem";
14         else
15             echo "Nghiem PT: ".$b/$a;
16 ?>
17 </body>
```



## Khai báo require()

- ◆ Sau đó ta xây dựng trang PTBN.PHP và có sử dụng require() để nhúng trang Bienluan.PHP vào.

```
1 <html>
2 <head>
3 <title>Bail</title>
4 </head>
5 <body>
6 <table bgcolor="#0099FF">
7 <tr><td>
8 <form method="POST" action="trang11.php">
9     <p>Giai PT Bac Nhat </p>
10    <p>a>Nhap a: <input type="text" name="t1" size="20" value=""></p>
11    <p>a>Nhap b: <input type="text" name="t2" size="20" value=""></p>
12    <p><input type="submit" value="Submit" name="B1">&nbsp;
13    <input type="reset" value="Reset" name="B2"></p>
14 </form></td></tr>
15 <tr><td>
16 <?php require("Bienluan.php");?>
17 </td></tr>
18 </table>
19 </body>
20 </html>
```

## Khai báo require()

---

### Lưu ý:

- ☐ Có thể nhúng bất kỳ dạng file có phần mở rộng nào vào trang PHP. Tuy nhiên, chỉ có những trang có phần mở rộng .PHP hoặc có đoạn mã PHP sẽ được thực thi.
- ☐ Khi nhúng file bằng require() thì nội dung của file trong khai báo require() sẽ là một phần của trang PHP hiện tại, và chúng có thể được thực thi khi trang PHP được gọi.
- ☐ Khai báo require() cần được đặt giữa cặp thẻ `<?PHP` và `?>`

## Khai báo include()

---

- ❖ Cả include và require đều có tác dụng là nhúng file vào trang PHP. Tuy nhiên, giữa chúng có điểm khác là Khi nạp bằng require thì web server cần phải dịch lại mỗi khi thực thi.
- ❖ Trong trường hợp nhúng nhiều file vào trang PHP mà phụ thuộc vào điều kiện nào đó thì ta nên sử dụng include.



## Khai báo include()

---



Ví dụ: Xét đoạn mã

```
21 <?php
22     $a=$_POST['hsa'];
23     if ($a==0)
24         include("giaiptbacnhat.php");
25     else
26         include("giaiptbachai.php");
27 ?>
```



## Thực hành

---

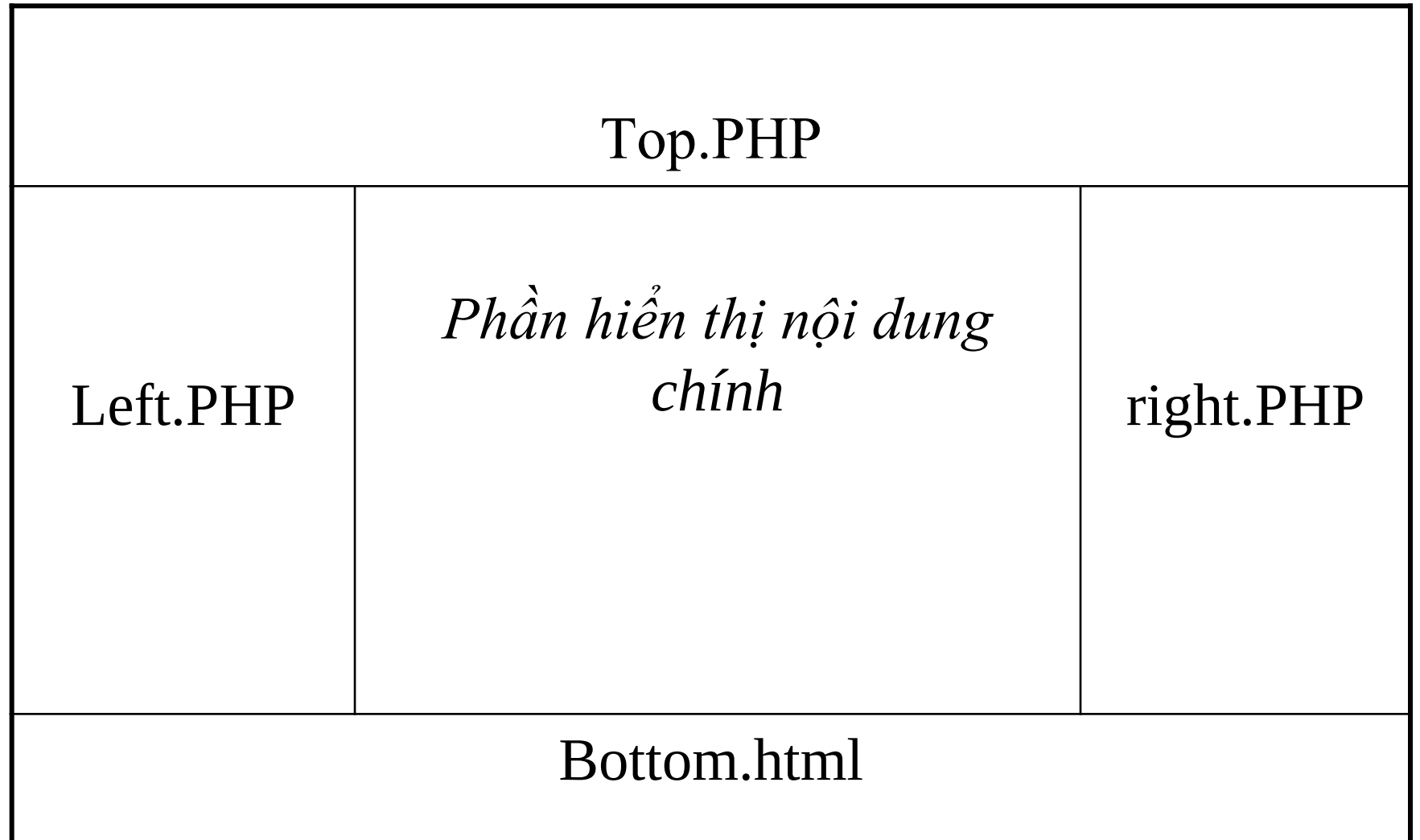
1. Tạo các file có tên như sau:

- Top.PHP
- Left.PHP
- Right.PHP
- Bottom.html

Sau đó, sử dụng require hoặc include để chèn vào trang index.PHP như sau:



## Thực hành





## Thực hành

---

2. Tạo 2 file có tên như sau:

- Giaiptbn.PHP
- Giaiptbh.PHP

Sau đó, sử dụng require hoặc include để chèn vào trang giainghiem.PHP.

3. Nhập vào chuỗi ngày tháng năm, sau đó tách chuỗi, kiểm tra và thông báo chính xác ngày, tháng, năm nhập vào. Nếu giá trị nhập vào không hợp lệ thì đưa ra thông báo.

4. Tạo một file .txt sau đó sử dụng các hàm thao tác với file để:

- Đọc dữ liệu từ file và hiển thị ra màn hình
- Ghi nội dung mới vào file



# Chương 5: Session và cookie





## Chương 5: session và cookie

---

**5.1 GIỚI THIỆU VỀ SESSION**

**5.2 GIỚI THIỆU VỀ COOKIE**

**5.3 SỬ DỤNG SESSION VỚI COOKIE**



## 5.1. Giới thiệu về session

---

- ◆ **Khái niệm session**
- ◆ **Chức năng của session**
- ◆ **Làm việc với session**

## Khái niệm session

---

- ◆ Session là một biến được dùng để lưu trữ thông tin hoặc sự thay đổi về phiên làm việc của một người dùng nào đó từ trang này đến trang khác trong cùng một Website.
- ◆ Các biến session chứa thông tin của người dùng và có tác dụng đối với tất cả các trang trong một ứng dụng.
- ◆ Các session được hỗ trợ trong các phiên bản PHP khác nhau cũng có điểm khác nhau.



## Chức năng của session

---

- ◆ Session có chức năng lưu trữ thông tin của người dùng trên server cho lần sử dụng tiếp theo.
- ◆ Thông tin session chỉ mang tính chất tạm thời, nghĩa là nó sẽ bị mất đi khi người dùng hủy chúng, thời gian sống đã hết hoặc kết thúc trình duyệt.
- ◆ Nếu muốn lưu trữ lâu dài thì cần lưu nó trong CSDL.
- ◆ Các session làm việc bằng cách tạo ra một unique id (UID) cho mỗi Browser và lưu trữ các biến dựa trên UID này.



## Chức năng của session

---

- ◆ Khi Browser mở ra ứng với trang Web bất kỳ của Website, session ID có giá trị là một dãy số ngẫu nhiên được lưu trữ trên phía client.
- ◆ Session ID có chu kỳ sống cho đến khi hết hạn sử dụng
- ◆ Có thể truyền session ID với URL hoặc ghi chúng ra Cookie

Lưu ý:

- ◆ Thời hạn hết sử dụng của session do cấu hình của trình chủ Web server quy định.
- ◆ Session ID là thông tin duy nhất lưu trữ trên Client, các biến session được lưu trữ trong một tập tin dạng text trên server



## Làm việc với session

---

- ◆ **Khởi động session**
- ◆ **Khai báo và gán giá trị cho biến session**
- ◆ **Lấy giá trị của biến session**
- ◆ **Hủy bỏ biến session**

## Khởi động session

---

- ◆ Để sử dụng session, trước tiên cần khởi động nó.
- ◆ Có hai cách dùng để khởi động session:
  - ◆ Có thể sử dụng cú pháp: `session_start()`;
  - ◆ Cấu hình lại trong trang `PHP.ini` như sau:  
`session.auto_start = 0`
- ◆ Nếu sử dụng hàm `session_start()` thì hàm này cần phải đặt trước thẻ `<html>`



## Khai báo và gán giá trị cho biến session

❖ Sau khi khởi động session, để khai báo và lưu trữ giá trị cho biến session ta sử dụng biến `$_SESSION`:

`$_SESSION['chỉ số'] = giá trị;`

Lưu ý: tên biến không có dấu \$ ở trước

**Ví dụ:** ta có đoạn code trong *file1.PHP* như sau:

```
<?PHP session_start();  
$username = "admin";  
$email = "admin@gmail.com";  
$name = "lap trinh 02A";  
$_SESSION['user'] = $username;  
$_SESSION['email'] = "admin@gmail.com";  
$_SESSION['name'] = "Lap trinh 02A";?>
```





## Khai báo và gán giá trị cho biến session

---

```
<html>
  <head>
    <title>Khoi dong va dang ky
    session</title>
  </head>
  <body>
    3 session da duoc dang ky.
    <a href="file2.PHP">next
    page</a>
  </body>
</html>
```



## Lấy giá trị của biến session

---

- ❖ Sau khi được khởi động và lưu trữ thông tin, các session này có hiệu lực trên mọi trang PHP mà người dùng truy cập của Website ứng với Browser đang mở.
- ❖ Chẳng hạn, đoạn mã sau dùng để lấy nội dung được lưu trữ trong các session ở trang file1.PHP



## Lấy giá trị của biến session

---

```
<?PHP  session_start();?>
<html>
<head>
    <title>Lay gia tri bien session</title>
</head>
<body>
<h4> Doc gia tri luu tru trong cac bien session
    o file1.PHP</h4>
<?PHP
echo "Username:". $username. "<br/>";
echo "Username:<b>".
    $_SESSION['user']. "</b><br/>";
echo "Email:<b>".
    $_SESSION['email']. "</b><br/>";
echo "Name:<b>". $_SESSION['name']. "</b>"; ?>
</body>
</html>
```

## Session ID

- ◆ Php sử dụng cookie để lưu trữ thông tin lấy được từ session ID.
- ◆ Ngoài ra, cũng có thể lưu trữ session ID vào URL

Ví dụ:

```
1  <?php
2  session_start();
3  $id=session_id();
4  ?>
5  <html>
6  <head>
7  <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
8  <title>Untitled Document</title>
9  </head>
10 <body>
11 <p> lưu session ID vào url</p>
12 <a href="vd2.php?php=$id?"> Next page </a>
13 </body>
14 </html>
```



# Session ID

Dw File Edit View Insert Modify Format Commands Site Window Help

DESIGNER

bail.html x trang1.php x top.html x Untitled-11\* x trang1.php x vd1.php\* x vd2.php\* x C:\xampp\htdocs\chuong 5\vd2.php

Code Split Design Live Code Live View Inspect Title: Untitled Document

Address: file:///C:/xampp/htdocs/chuong%205/top.html

1 / 2 No syntax errors.

```
1 <?php
2 session_start();
3 $id=session_id();
4 echo "Get session id: ".$id."<br/>";
5 ?>
6 <html>
7 <head>
8 <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
9 <title>Untitled Document</title>
10 </head>
11 <body>
12 <p> Lấy session id</p>
13 <a href="vd1.php"> Previous page </a>
14 </body>
15 </html>
```

Lấy session id

[Previous page](#)

PROPERTIES

Code view You have made changes to the code. To edit selection properties, click Refresh or press F5. Refresh

CSS STYLES AP ELEMENTS

You have made changes to the code. To update the CSS panel or edit CSS properties, click Refresh or press F5. Refresh

EN 4:57 AM 10/6/2010



## Hủy bỏ biến session

---

- ❖ Trường hợp không muốn sử dụng các biến session, ta có thể hủy bỏ chúng bằng cách gọi hàm `session_unset()` hoặc `session_destroy()`.
- ❖ Hàm `session_unset()` dùng để hủy bỏ các biến session đang có hiệu lực. Nếu chỉ muốn hủy bỏ một biến session nào đó ta cần truyền tham số cho hàm
- ❖ Hàm `session_destroy()` dùng để kết thúc tất cả các biến session và dữ liệu lưu trữ trong đó.



## Hủy bỏ biến session

---

```
<?PHP
session_start();
echo "Username:<b>".
    $_SESSION['user']."</b><br/>";
session_unset($_SESSION['user']);
echo "Email:<b>".
    $_SESSION['email']."</b><br/>";
echo "Name:<b>". $_SESSION['name']."</b>";
?>
```



## 5.2. Giới thiệu về cookie

---



**Cookie là gì?**



**Làm việc với Cookie**



## Cookie là gì?

---

- ❖ Cookie thường được dùng để xác định người sử dụng
- ❖ Nó được xem như session nhưng lưu trữ thông tin trên trình khách (client)
- ❖ Tại thời điểm nào đó, một máy tính yêu cầu một trang web cùng với trình duyệt browser thì nó cũng gửi Cookie đi kèm.



## Làm việc với cookie

---

- ◆ **Gán giá trị cho cookie**
- ◆ **Lấy giá trị của cookie**
- ◆ **Xóa cookie**



## Gán giá trị cho cookie

---

- ◆ Để gán giá trị cho cookie ta sử dụng hàm `setcookie` theo cú pháp:

**`setcookie(name, [value, expire, path, domain]);`**

- ◆ Hàm `setcookie` cần đặt trước thẻ `<html>`

Ví dụ:

```
<?PHP
```

```
$expire = time() + 60*60*24*30;  
setcookie("user", "administrator",  
        $expire);
```

```
?>
```



## Lấy giá trị của cookie

---

- ◆ Để lấy giá trị của cookie ta sử dụng biến `$_COOKIE` theo cú pháp: `$_COOKIE['tenbien'];`
- ◆ Chẳng hạn, để lấy thông tin được lưu trong cookie ở trên ta viết

```
<html>
<head> <title>Vi du ve cookie</title>
</head>
<body>
<h4>Lay gia tri cua cookie! </h4>
<?PHP echo "Username:". $_COOKIE["user"];
?>
</body> </html>
```



## Xóa cookie

---

- ◆ Để xóa cookie, cần chắc chắn thời điểm có hiệu lực là trong quá khứ

Ví dụ:

```
<?PHP
```

```
// set the expiration date to one hour  
ago
```

```
setcookie("user", "", time()-3600);
```

```
?>
```



## Làm việc với cookie

---

- ❖ Trong trường hợp Browser không hỗ trợ cookie, cần phải sử dụng phương thức khác để chuyển thông tin từ trang này sang trang khác trong ứng dụng
- ❖ Một phương thức được sử dụng là chuyển thông tin thông qua form



## Làm việc với cookie

---

Ví dụ: ta có trang index.html như sau:

```
<html>
<body>
<h3> Xây dựng form nhập thông tin!</h3>
<form action="welcome.PHP"
method="post">
  Name: <input type="text" name="name" />
  <p>Age: <input type="text" name="age"
/></p>
  <p><input type="submit"
value="Enter"></p>
</form>
</body> </html>
```



## Làm việc với cookie

---

Trang welcome.PHP có nội dung như sau:

```
<html>
<body>
Welcome <b><?PHP echo $_POST["name"];
?></b> <br />
You are <b><?PHP echo $_POST["age"]; ?>
</b> years old.
</body>
</html>
```





## 5.3. Sử dụng session với cookie

---

- ◆ **Gán cookie bằng session**
- ◆ **Sử dụng cookie để lưu trữ session ID**



## Gán cookie bằng session

---

Đối với cookie có một số vấn đề sau đây:

- Một vài webbrowser không hỗ trợ cookie và một số người dùng không có các cookie trên browser của họ.
- Đây là lý do để PHP sử dụng cả hai cách thức: cookie và URL method.
- Khi sử dụng PHP session, ta sẽ không phải thiết lập các cookie, những hàm session sẽ lưu giữ những thông tin này cho chúng ta



## Gán cookie bằng session

---

- Để thiết lập session cookie ta dùng hàm: void session\_set\_cookie\_params (int lifetime [, string path [, string domain]])

```
<?PHP session_start();
```

```
$myvalue="Online Recruitment";
```

```
session_set_cookie_params($myvalue);
```

```
?>
```

```
<HTML> <HEAD> <TITLE>::Welcome to PHP</TITLE>
```

```
</HEAD>
```

```
<BODY>
```

```
<h4>Session-Cookie</h4>
```

```
</BODY>
```

```
</HTML>
```



## Gán cookie bằng session

---

- Để xem nội dung của các cookie đã được thiết lập bởi session ta sử dụng hàm:  
`session_get_cookie_params()`.
- Hàm này sẽ trả về một mảng liên kết mà các phần tử của mảng chứa các thông tin như: lifetime, path, domain,...



## Sử dụng cookie để lưu trữ session ID

---

- PHP sẽ sử dụng các cookie mặc định cùng với session. Nếu có thể được, một cookie sẽ được thiết lập chứa SessionID.
- Cách khác: đưa sessionID vào trong thẻ link. SessionID được chứa trong hằng SID.
- Thêm vào cuối thẻ link hằng SID để dùng nó như là phương thức GET. Ví dụ sau:



## Sử dụng cookie để lưu trữ session ID

---

```
<?PHP session_start();  
$myvalue=session_id();  
?>  
<html> <head>  
    <title>Khoi dong va dang ky  
    session</title>  
</head><body>  
<a href="cookie1.PHP?<?=$myvalue?>">next  
    page</a>  
</body>  
</html>
```



## Sử dụng cookie để lưu trữ session ID

---

```
<?PHP    session_start(); ?>
<html>
<head><title>Lay gia tri bien
      session</title>
</head> <body>
<?PHP
foreach ($_COOKIE as $c_name => $c_value)
print "$c_name = $c_value<br>";
list($a,$b) = each($_GET);
echo "SessionID:". $a;
?>
</body>
</html>
```



HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

# Chương 6: Tổng quan về MySQL





## Chương 6: Tổng quan về MySQL

---

**6.1 GIỚI THIỆU VỀ MySQL**

**6.2 TẠO CƠ SỞ DỮ LIỆU VÀ NGƯỜI DÙNG**

**6.3 CÁC KIỂU DỮ LIỆU TRONG MySQL**

**6.4 CÁC CÂU LỆNH SQL**

**6.5 CÁC HÀM TRONG MySQL**



## 6.1 Giới thiệu về MySQL

---

- ◆ **MySQL là gì?**
- ◆ **Các đặc điểm của MySQL**
- ◆ **Lịch sử phát triển**
- ◆ **Download, cài đặt và cấu hình**

# MySQL là gì?

---

- ◆ MySQL là:
  - ◆ Một hệ quản trị CSDL
  - ◆ Một hệ quản trị CSDL quan hệ
  - ◆ Được phát triển, phân phối và hỗ trợ bởi MySQL AB.
- ◆ Để làm việc với MySQL cần đăng ký kết nối, tạo CSDL, quản lý người dùng, phân quyền sử dụng, thiết kế đối tượng Table của CSDL và xử lý dữ liệu.
- ◆ Để quản lý và thao tác trên CSDL ta có thể sử dụng giao diện đồ họa hoặc dạng Command line.

## Đặc điểm của MySQL

---

- ❖ MySQL cũng giống như các hệ quản trị CSDL: Access, SQL Server, PostgreSQL, Oracle, ...
- ❖ Phần mềm mã nguồn mở do đó có thể tải miễn phí từ trang chủ.
- ❖ Nó có nhiều phiên bản cho các hệ điều hành khác nhau: phiên bản Win32 cho các hệ điều hành dòng Windows, Linux, Mac OS X, Unix, Solaris...



## Đặc điểm của MySQL

---

- ❖ MySQL là cơ sở dữ liệu tốc độ cao, ổn định và dễ sử dụng, có tính khả chuyển, hoạt động trên nhiều hệ điều hành cung cấp một hệ thống lớn các hàm tiện ích rất mạnh.
- ❖ Với tốc độ và tính bảo mật cao, MySQL rất thích hợp cho các ứng dụng có truy cập CSDL trên Internet.
- ❖ MySQL server hoạt động trong các hệ thống nhúng hoặc client/server.



# Đặc điểm của MySQL

---

## Why MySQL?

---

The MySQL® database has become the world's most popular open source database because of its consistent fast performance, high reliability and ease of use. It's used on every continent -- Yes, even Antarctica! -- by individual Web developers as well as many of the world's largest and fastest-growing organizations to save time and money powering their high-volume Web sites, business-critical systems and packaged software -- including industry leaders such as Yahoo!, Alcatel-Lucent, Google, Nokia, YouTube, and Zappos.com.

Not only is MySQL the world's most popular open source database, it's also become the database of choice for a new generation of applications built on the LAMP stack (Linux, Apache, MySQL, PHP / Perl / Python.) MySQL runs on more than 20 platforms including Linux, Windows, Mac OS, Solaris, HP-UX, IBM AIX, giving you the kind of flexibility that puts you in control.

Whether you're new to database technology or an experienced developer or DBA, MySQL offers a comprehensive range of certified software, support, training and consulting to make you successful.



- Original development of MySQL by Michael Widenius and David Axmark beginning in 1994
- First internal release on 23 May 1995
- Windows version was released on 8 January 1998 for Windows 95 and NT
- Version 3.23: beta from June 2000, production release January 2001
- Version 4.0: beta from August 2002, production release March 2003 (unions)

Original development of MySQL by Michael Widenius and David Axmark beginning in 1994

First internal release on 23 May 1995

Windows version was released on 8 January 1998 for Windows 95 and NT

Version 3.23: beta from June 2000, production release January 2001

Version 4.0: beta from August 2002, production release March 2003  
(unions)



## Lịch sử phát triển

Version 4.01: beta from August 2003, Jyoti adopts MySQL for database tracking

Version 4.1: beta from June 2004, production release October 2004 (R-trees and B-trees, subqueries, prepared statements)

Version 5.0: beta from March 2005, production release October 2005  
(cursors, stored procedures, triggers, views, XA transactions)



Sun Microsystems acquired MySQL AB on 26 February 2008.]

Version 5.1: production release 27 November 2008 (event scheduler, partitioning, plugin API, row-based replication, server log tables)

Version 5.1 contained 20 known crashing and wrong result bugs in addition to the 35 present in version 5.0.

MySQL 5.1 and 6.0 showed poor performance when used for data warehousing — partly due to its inability to utilize multiple CPU cores for processing a single query.

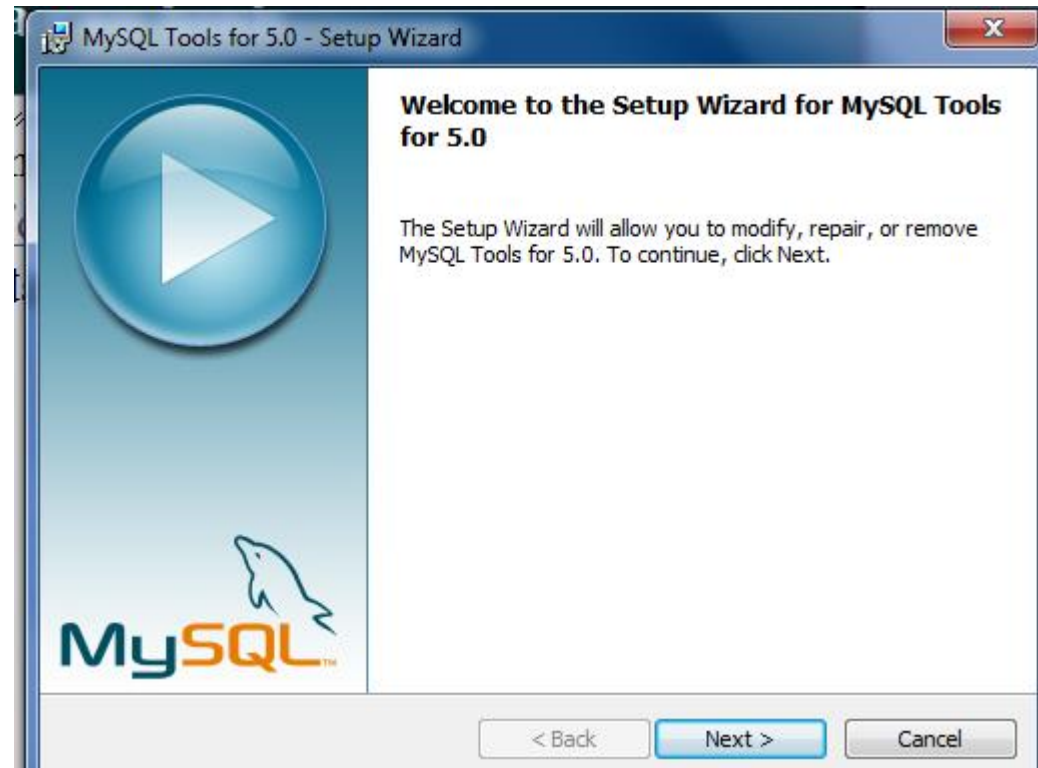
Oracle acquired Sun Microsystems on 27 January 2010. Oracle and Sun



## Download, cài đặt và cấu hình

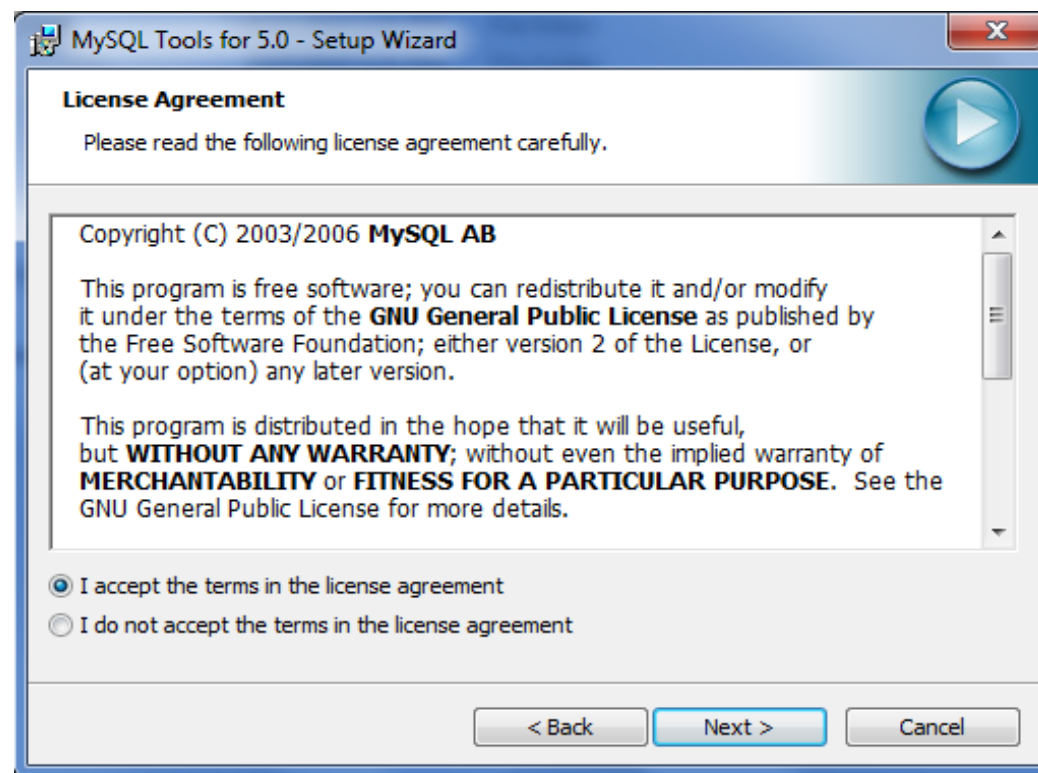
- ❖ Có thể download miễn phí phần mềm MySQL tại địa chỉ:  
<http://dev.MySQL.com/downloads>
- ❖ Sau đó tiến hành cài đặt theo các bước:

Bước 1: Nhấn next



# Download, cài đặt và cấu hình

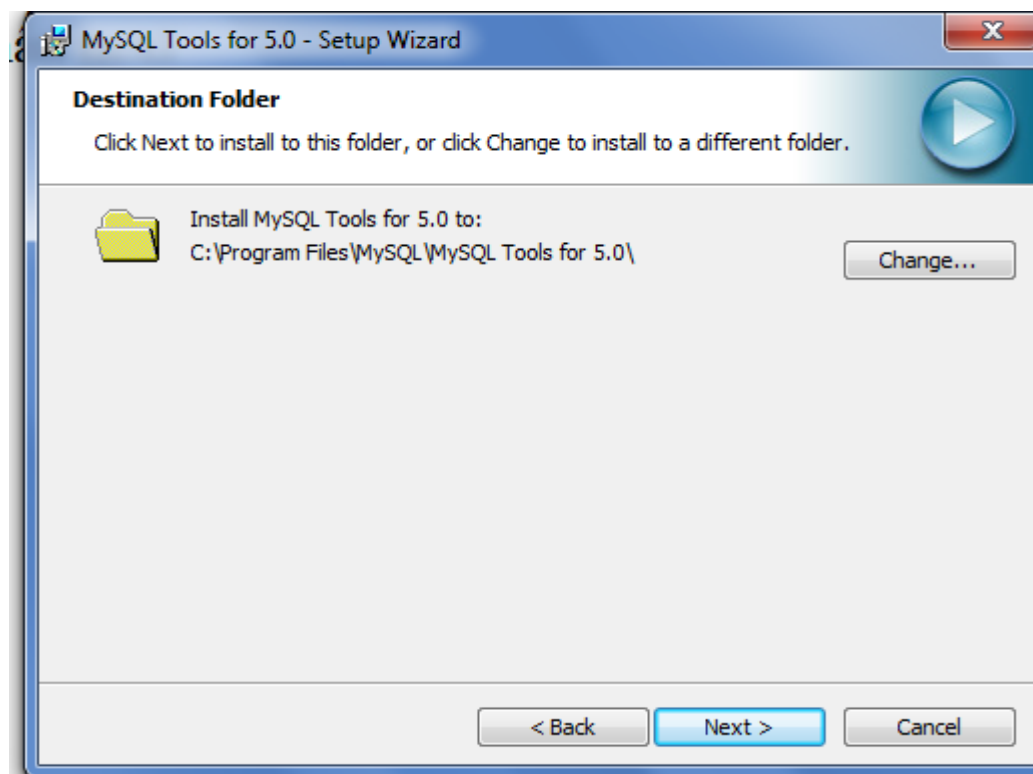
## Bước 2: Nhấn next



## Download, cài đặt và cấu hình

---

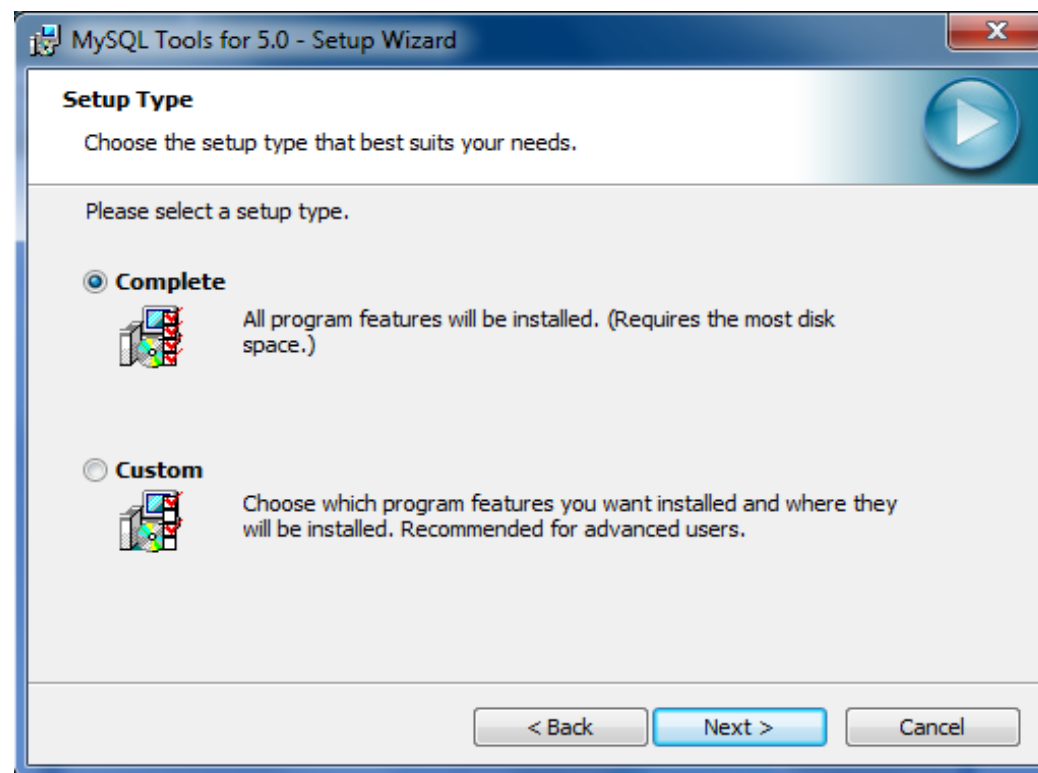
### Bước 3: Nhấn next



# Download, cài đặt và cấu hình

---

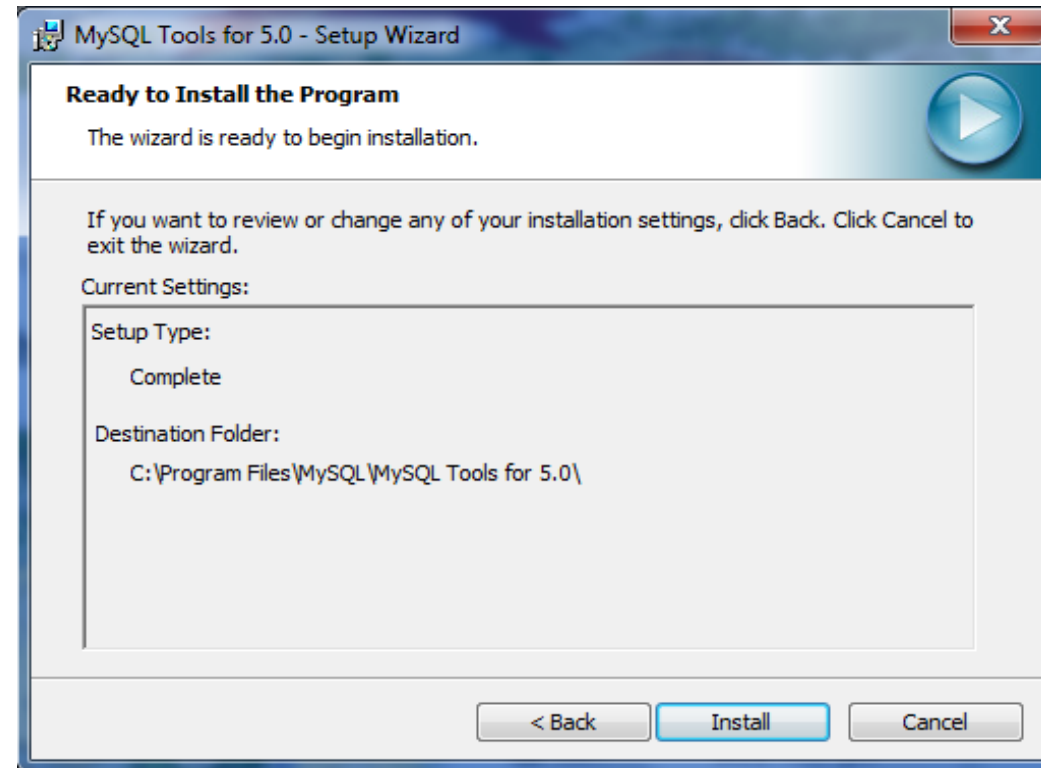
## Bước 4: Nhấn next





## Download, cài đặt và cấu hình

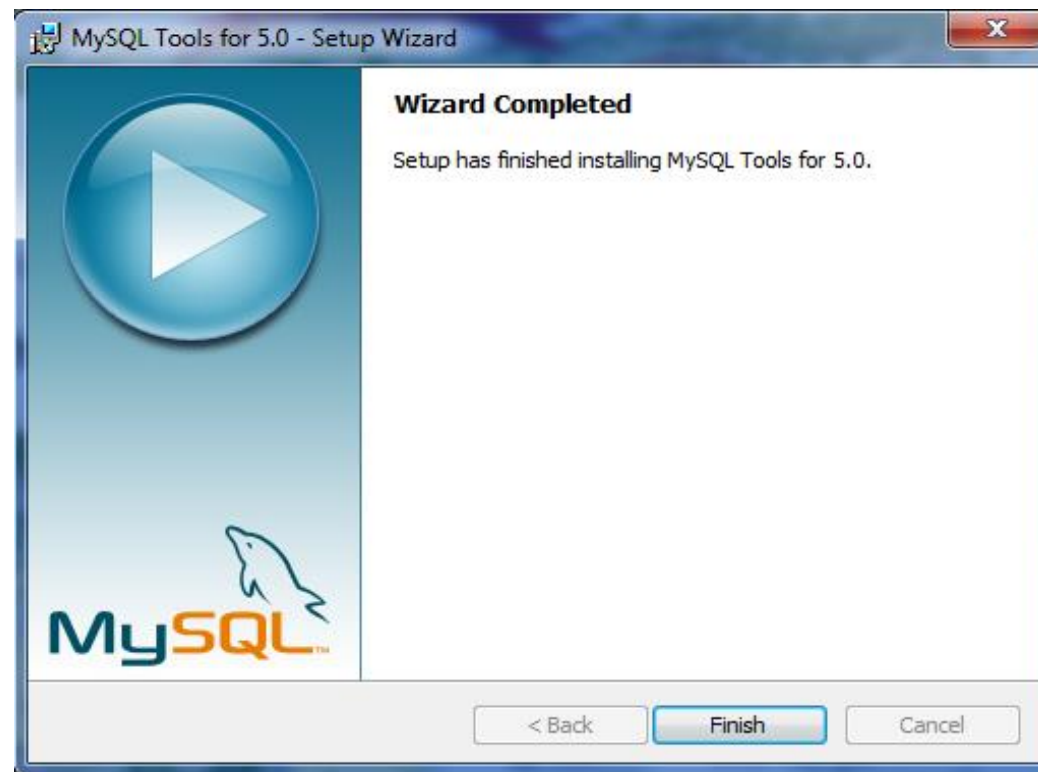
### Bước 5: Nhấn install sau đó nhấn next



# Download, cài đặt và cấu hình

---

## Bước 6: Finish





## Download, cài đặt và cấu hình

---



Ngoài ra ta có thể sử dụng PHPmyadmin được tích hợp trong XAMMP





## 6.2 Tạo CSDL và người dùng

---

- ◆ **Kết nối và tạo CSDL**
- ◆ **Quản lý người dùng**
- ◆ **Cấp quyền cho người dùng**
- ◆ **Xóa quyền của người dùng**



## Kết nối và tạo CSDL

---

- ◆ Để kết nối cơ sở dữ liệu ta có thể thực hiện theo hai cách:
  - ◆ Kết nối và tạo CSDL bằng Command line
  - ◆ Kết nối và tạo CSDL bằng giao diện đồ họa: MySQL Administrator hoặc PHPmyadmin



## Quản lý người dùng

---

- ◆ Để đăng nhập vào MySQL ta có thể sử dụng user là root và pass là rỗng
- ◆ Ngoài tài khoản này ta có thể tạo thêm các tài khoản cho người dùng với các users và pass khác nhau.



## Cấp quyền cho người dùng

---

- ❖ Với quyền root ta có thể thực hiện mọi thao tác trên CSDL: select, update, insert, delete, ...
- ❖ Tuy nhiên, khi tạo quyền người dùng ta cũng có thể hạn chế bớt một số quyền nhất định nào đó



## Xóa quyền hoặc tài khoản người dùng

---

- ❖ Sau khi cấp quyền cho người dùng ta có thể thêm hoặc loại bỏ một số quyền nào đó.
- ❖ Với việc truy cập vào tài khoản root ta có thể xóa các tài khoản người dùng đã được tạo ra.



## 6.3 Các kiểu dữ liệu trong MySQL

---

- ◆ **Dữ liệu kiểu numeric**
- ◆ **Dữ liệu kiểu date and time**
- ◆ **Dữ liệu kiểu string**

## Dữ liệu kiểu numeric

---

- ◆ Dữ liệu kiểu numeric gồm các kiểu sau:
  - ◆ Tinyint: Lưu các số nguyên không dấu (unsigned) từ 0-255 hoặc các số nguyên có dấu từ -128 -> 127
  - ◆ Mediumint: Lưu các số nguyên từ 0 đến 16.777.215 hoặc từ -8.388.608 đến 8.388.607
  - ◆ Int: Lưu các số nguyên từ 0 đến 4.294.967.295 hoặc từ -2.147.483.648 đến 2.147.483.647
  - ◆ Bigint
  - ◆ Float
  - ◆ Double
  - ◆ Decimal/real

## Dữ liệu kiểu date and time

---

- ◆ Dữ liệu kiểu date and time gồm các kiểu sau:
  - ◆ Date: Lưu trữ ngày dạng yyyy-mm-dd. Cho phép giá trị từ 1000-01-01 đến 9999-12-31
  - ◆ Datetime: Lưu trữ dạng yyyy-mm-dd hh:mm:ss
  - ◆ Timestamp: Tự động ghi nhận thời gian thay đổi gần nhất. Tùy thuộc vào độ rộng của cột nằm trong khoảng từ 2 đến 14 (yy đến yyyy-mm-dd hh:mm:ss)
  - ◆ Time: Dùng để lưu trữ giờ định dạng hh:mm:ss.
  - ◆ Year: Dùng để lưu năm bắt đầu từ 1970



- ◆ Dữ liệu kiểu string gồm các kiểu sau:
  - ◆ Char: Chiều dài tối đa 255 ký tự, đây là kiểu có chiều dài cố định.
  - ◆ Varchar: Cũng tương tự kiểu char có chiều dài tối đa 255 ký tự, song có điểm khác là có chiều dài thay đổi, các giá trị sẽ không bị nối thêm ký tự trắng.
  - ◆ nVarchar: chứa tiếng Việt
  - ◆ Tinytext: là kiểu ký tự văn bản nhị phân. Có chiều dài tối đa 255.
  - ◆ Text: có chiều dài 65.535 ký tự. Các chỉ mục có thể được tạo trên 255 ký tự đầu của cột text
  - ◆ Mediumtext: có chiều dài 16.777.215
  - ◆ Longtext: có chiều dài >4 tỉ ký tự
  - ◆ Enum, Set, ...



## 6.4 Các câu lệnh SQL

---

- ◆ **SQL là gì?**
- ◆ **Các câu lệnh SQL cơ bản**

## 6.4 Các câu lệnh SQL

---

- ◆ SQL là ngôn ngữ dùng để truy vấn CSDL
- ◆ Được chia làm 4 loại:
  - ◆ DDL (Data Definition Language)
  - ◆ DML (Data Manipulation Language)
  - ◆ DCL (Data Control Language)
  - ◆ TCL (Transaction Control Language)
- ◆ Các câu lệnh SQL thông dụng:
  - ◆ Câu lệnh Select
  - ◆ Câu lệnh Insert
  - ◆ Câu lệnh Update
  - ◆ Câu lệnh Delete
  - ◆ Câu lệnh Join

## Câu lệnh SELECT

---

- ❖ Dùng để truy vấn dữ liệu từ một hay nhiều bảng khác nhau và trả về kết quả là một tập bản ghi thỏa mãn điều kiện nào đó
- ❖ Cú pháp: **SELECT <Danh sách các cột>**  
**[FROM <danh sách các bảng>]**  
**[WHERE <các điều kiện ràng buộc>]**  
**[GROUP BY <tên cột/ biểu thức trong SELECT>]**  
**[HAVING <đk bắt buộc của GROUP BY>]**  
**[ORDER BY <danh sách cột>]**  
**[LIMIT FromNumber | ToNumber]**

## Câu lệnh SELECT

---

- ❖ Trong đó, danh sách các cột: Tên các cột, biểu thức kết hợp giữa các cột của bảng
- ❖ Trường hợp truy vấn tất cả các cột của bảng ta sử dụng toán tử \* thay vì chỉ ra danh sách tất cả các cột
- ❖ Trường hợp, có các cột cùng tên ở các bảng khác nhau thì ta cần chỉ ra tên bảng đi trước theo cú pháp:  
Tên\_bảng.Tên\_cột

## Câu lệnh SELECT

---

- ❖ Câu lệnh SELECT với mệnh đề FROM: dùng để truy vấn dữ liệu từ các cột hoặc biểu thức cho cột đó từ bảng được chỉ ra sau mệnh đề FROM

Ví dụ:

```
SELECT * FROM Sinhvien;
```

```
SELECT Masv, HoTen FROM Sinhvien;
```

```
SELECT * FROM Sinhvien LIMIT 0, 10;
```

## Câu lệnh SELECT

---

- ❖ Câu lệnh SELECT với mệnh đề WHERE: dùng để truy vấn dữ liệu từ các cột hoặc biểu thức cho cột đó từ bảng được chỉ ra sau mệnh đề FROM và thỏa mãn điều kiện nào đó được chỉ ra sau mệnh đề WHERE

Ví dụ:

```
SELECT * FROM Sinhvien WHERE conditions;
```

```
SELECT Masv, HoTen FROM Sinhvien WHERE  
conditions;
```

Lưu ý: các phép toán được sử dụng để thiết lập các điều kiện sau WHERE là các phép toán Quan hệ và Logic



## Câu lệnh SELECT

---

- ❖ Các phép toán quan hệ: >, >=, <, <=, =, !=, <>
- ❖ Các phép toán Logic: and, or, not, not in, between, like, not like, in

Ví dụ:

```
SELECT * FROM Sinhvien WHERE Tongdiem > 2.0
```

```
SELECT * FROM Sinhvien WHERE Hoten like  
    '%Hoa';
```



## Câu lệnh SELECT

---

- ❖ Câu lệnh SELECT với mệnh đề ORDER BY: dùng để truy vấn dữ liệu và kết quả trả về được sắp xếp tăng dần (ASC) hoặc giảm dần (DESC) trên cột nào đó.

Ví dụ:

```
SELECT * FROM Sinhvien ORDER BY  
Tongdiem DESC;
```

```
SELECT * FROM Sinhvien ORDER BY Hoten  
ASC;
```

Trường hợp sắp xếp theo nhiều cột thì các cột được phân cách nhau bởi dấu phẩy (,)



## Câu lệnh SELECT

---

- ❖ Câu lệnh SELECT với mệnh đề GROUP BY: dùng để truy vấn dữ liệu và kết quả trả về được nhóm lại theo một cột nào đó.

Ví dụ:

```
SELECT Tongdiem, count(Tongdiem) FROM  
Sinhvien GROUP BY Tongdiem ORDER BY  
Tongdiem;
```



## Câu lệnh SELECT

---

- ❖ Câu lệnh SELECT với mệnh đề AS: sử dụng khi cần phải thay đổi tên cột nào đó trong câu truy vấn.

Ví dụ:

```
SELECT Tongdiem, count(Tongdiem) AS Sosv  
FROM Sinhvien GROUP BY Tongdiem ORDER  
BY Tongdiem;
```

## Câu lệnh SELECT

---

- ❖ Câu lệnh SELECT với mệnh đề LIMIT N, M: dùng để giới hạn số bản ghi cần truy vấn từ vị trí thứ N đến vị trí thứ M.

Ví dụ:

```
SELECT * FROM Sinhvien LIMIT 0, 10;
```

```
SELECT * FROM Sinhvien ORDER BY  
Tongdiem DESC LIMIT 0, 10;
```



## Câu lệnh SELECT

---

- ❖ Câu lệnh SELECT với mệnh đề DISTINCT: dùng để truy vấn dữ liệu và kết quả trả về được nhóm lại theo một cột nào đó.

Ví dụ:

```
SELECT Tongdiem, count(Tongdiem) FROM  
Sinhvien GROUP BY Tongdiem ORDER BY  
Tongdiem;
```

## Câu lệnh INSERT

---

- ◆ Được sử dụng khi cần thêm bản ghi vào bảng trong CSDL MySQL.
- ◆ Khi thêm dữ liệu, cần chú ý đến kiểu dữ liệu của các cột mình cần thêm dữ liệu.
- ◆ Cần quan tâm đến quyền của User đăng nhập có được phép Insert hay không
- ◆ Khi Insert dữ liệu vào bảng có 3 trường hợp:
  - ◆ Insert từ giá trị cụ thể
  - ◆ Lấy giá trị từ một hoặc nhiều bảng khác
  - ◆ Bao gồm cả hai trường hợp

## Câu lệnh INSERT

---

Ví dụ: Insert vào bảng từ giá trị cụ thể.

```
INSERT INTO Sinhvien (Masv, Hoten,  
Tongdiem) VALUES ('0073', 'Lê Anh Ngọc',  
2.37);
```

❑ Insert vào bảng từ giá trị của bảng khác

```
INSERT INTO Sinhvien (Masv, Hoten,  
Tongdiem) SELECT Mahs, Hoten, Tongdiem  
FROM Hocsinh;
```

Insert vào bảng từ giá trị cụ thể, bảng khác:

```
INSERT INTO <bảng 1> [<danh sách các cột>]  
SELECT [danh sách các cột], danh sách giá trị  
FROM <bảng 2> WHERE <conditions> ...
```

## Câu lệnh UPDATE

---

- ◆ Dùng để cập nhật lại dữ liệu đã tồn tại trong bảng.
- ◆ Nếu cập nhật giá trị cụ thể: UPDATE <tên bảng> SET <cột> = <giá trị>, [<cột> = <giá trị>] [WHERE <conditions>]
- ◆ Cập nhật giá trị từ bảng khác: UPDATE <tên bảng> SET <tên cột>=<SELECT ... FROM tên bảng ...> ...
- ◆ UPDATE có thể ảnh hưởng đến nhiều bảng nhưng cập nhật giá trị chỉ có hiệu lực trên bảng đó.



## Câu lệnh UPDATE

---

Ví dụ:

- ❑ UPDATE Sinhvien SET Hoten='Le Thi B'  
WHERE Masv = '003';
- ❑ UPDATE Sinhvien SET Hoten= (SELECT Hoten  
FROM Hocsinh WHERE Mahs = Sinhvien.Masv)

## Câu lệnh DELETE

---

- ❖ Dùng để xóa bản ghi trong bảng được chỉ ra bởi tên bảng và mệnh đề WHERE (nếu có) nhằm xác định bản ghi cần xóa theo một điều kiện nào đó.
- ❖ `DELETE FROM <tên bảng> WHERE <conditions>`
- ❖ Conditions: có thể là phép toán giữa các cột và giá trị hoặc giá trị là kết quả trả về của một câu lệnh SELECT khác
- ❖ Lưu ý: không có khái niệm xóa giá trị trong một cột, vì xóa giá trị một cột đồng nghĩa với cập nhật cột đó bằng giá trị rỗng



## Câu lệnh DELETE

---

Ví dụ:

❑ `DELETE FROM Sinhvien WHERE Masv = '001';`

Lưu ý: trong trường hợp có ràng buộc về quan hệ của dữ liệu, thì việc xóa bản ghi cần được thực hiện ở bảng con trước bảng cha.

## Câu lệnh JOIN

---

- ◆ Dùng để kết hợp dữ liệu trên hai hay nhiều bảng lại với nhau
- ◆ Cần xác định cột nào trong bảng này có quan hệ với cột nào trong bảng kia
- ◆ Các dạng của JOIN
  - ◆ Inner Join
  - ◆ Left Join
  - ◆ Right Join

## Câu lệnh JOIN

---

- ◆ Inner Join: Dùng để kết hợp các bảng dữ liệu
- ◆ Cú pháp: `SELECT [các cột] FROM <bảng 1>  
INNER JOIN <bảng 2> ON <điều kiện kết hợp>  
WHERE .... ORDER BY ...`

Ví dụ: `SELECT Hoten, Tongdiem FROM Sinhvien  
INNER JOIN Hocsinh ON Sinhvien.Masv =  
Hocsinh.Masv WHERE Tongdiem >2.0 ORDER  
BY Tongdiem ASC`

Lưu ý: nếu cần trả về kết quả là tất cả các cột của các bảng tham gia Inner Join ta áp dụng cú pháp:

## Câu lệnh JOIN

---

Lưu ý: nếu cần trả về kết quả là tất cả các cột của các bảng tham gia Inner Join ta áp dụng cú pháp:

SELECT bảng 1.\*, bảng 2.\* [, bảng n.\*] FROM bảng 1  
INNER JOIN bảng 2 ON <điều kiện> ...

nếu trong các bảng cần kết nối có tên cột giống nhau thì câu lệnh SQL dạng SELECT cần chỉ rõ cột thuộc bảng nào. Trường hợp cả hai cùng lấy dữ liệu ra thì cần chuyển ánh xạ tên khác cho cột thông qua mệnh đề AS

## Câu lệnh JOIN

---

- ❖ Left Join: Dùng để kết hợp các bảng dữ liệu khi muốn trả về kết quả là những bản ghi của bảng bên trái tồn tại ứng với những bản ghi ở bảng bên phải không tồn tại.
- ❖ Cú pháp: `SELECT [các cột] FROM <bảng trái> LEFT JOIN <bảng phải> ON <điều kiện kết hợp> WHERE .... ORDER BY ...`

## Câu lệnh JOIN

---

- ◆ Right Join: Dùng để kết hợp các bảng dữ liệu khi muốn trả về kết quả là những bản ghi của bảng bên phải tồn tại dù bảng bên trái không tồn tại
- ◆ Cú pháp: `SELECT [các cột] FROM <bảng trái> RIGHT JOIN <bảng phải> ON <điều kiện kết hợp> WHERE .... ORDER BY ...`





## 6.5 Các hàm trong MySQL

---

- ❖ **Các hàm trong phát biểu Group by**
- ❖ **Các hàm về xử lý chuỗi**
- ❖ **Các hàm về xử lý thời gian**
- ❖ **Các hàm về số học**



## Các hàm trong phát biểu Group by

---

- ◆ Hàm AVG: trả về giá trị bình quân của cột hay trường trong câu truy vấn
- ◆ Hàm MIN: trả về giá trị nhỏ nhất của cột hay trường trong câu truy vấn
- ◆ Hàm MAX: trả về giá trị lớn nhất của cột hay trường trong câu truy vấn
- ◆ Hàm SUM: trả về tổng các giá trị của cột hay trường trong câu truy vấn

Ví dụ: `SELECT Masv, Hoten, Sum(Tongdiem),  
Min(Tongdiem), Max(Tongdiem),  
Avg(Tongdiem) FROM Sinhvien`



## Các hàm xử lý chuỗi

---

- ◆ Hàm ASCII: trả về mã ASCII của ký tự bên trái của chuỗi
- ◆ Hàm Char(number): chuyển đổi từ số nguyên sang dạng chuỗi
- ◆ Hàm Upper(string): chuyển chuỗi sang chữ hoa
- ◆ Hàm Lower(string): chuyển chuỗi sang chữ thường
- ◆ Hàm Len(string): trả về chiều dài của chuỗi
- ◆ Hàm Ltrim(string): loại bỏ khoảng trắng bên trái của chuỗi
- ◆ Hàm Rtrim(string): loại bỏ khoảng trắng bên phải của chuỗi
- ◆ Hàm left(string, n): trả về chuỗi bên trái tính từ đầu đến vị trí n
- ◆ Hàm right(string, n): trả về chuỗi bên phải tính từ cuối đến vị trí n
- ◆ Hàm instr(chuoi 1, chuoi 2): trả về vị trí chuỗi bắt đầu của chuỗi 1 trong chuỗi 2



## Các hàm về xử lý thời gian

---

- ◆ Hàm `CurDate()`: trả về ngày, tháng, năm hiện hành của hệ thống
- ◆ Hàm `CurTime()`: trả về giờ, phút, giây hiện hành của hệ thống
- ◆ Hàm `Period_Diff`(ngày đầu, ngày cuối): trả về số ngày trong khoảng thời gian giữa ngày đầu và ngày cuối
- ◆ Hàm `dayofmonth`: trả về ngày thứ mấy trong tháng
- ◆ ...



## Các hàm về số học

---

- ◆ Hàm sqrt: trả về căn bậc hai
- ◆ Hàm round: làm tròn giá trị biểu thức
- ◆ ...



## Các hàm về số học

```
<?php
session_start();
create_image();
exit();
function create_image()
{
    $mahoa = md5(rand(0,999));
    $chuoicon = substr($mahoa, 15, 5);
    $_SESSION["chuoicon"] = $chuoicon;
    $width = 100;
    $height = 25;
    $image = ImageCreate($width, $height);
    $bgcolor = ImageColorAllocate($image, 0,128, 255);
    $charcolor = ImageColorAllocate($image, 255,255, 128);
    ImageFill($image, 0, 0, $bgcolor);
    ImageString($image, 5, 30, 6, $chuoicon, $charcolor);
    header("Content-Type: image/jpeg");
    ImageJpeg($image);
    ImageDestroy($image);
}
?>
```



HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

# Chương 7: Kết nối CSDL MySQL trong PHP



## Chương 7: Kết nối CSDL MySQL trong PHP

---

**7.1 THIẾT LẬP KẾT NỐI CSDL**

**7.2 THÊM THÔNG TIN VÀO CSDL**

**7.3 KÍCH HOẠT DỮ LIỆU**





## 7.1 thiết lập kết nối CSDL

---

- ◆ **Mở và đóng kết nối**
- ◆ **Mở CSDL**
- ◆ **Chọn CSDL**
- ◆ **Truy vấn CSDL**
- ◆ **Làm việc với các bản ghi**



## Mở và đóng kết nối

---



### Cú pháp: Mở kết nối CSDL

```
$bien_con=mysqli_connect("servername",  
    "username", "password")  
if (!$bien_con)  
{    echo "Kết nối không thành công";  
    exit;}
```

Hoặc

```
$bien_con=mysqli_connect("servername",  
    "username", "password") or die("Kết  
    nối không thành công");
```



## Mở và đóng kết nối

---



Cú pháp: Đóng kết nối CSDL

```
Mysql_close ($bien_con) ;
```



### Cú pháp:

```
MySQL_select_db("Database name")  
or die("Không mở được CSDL");
```

Hoặc

```
MySQL_select_db("Database name",  
                $bien_con)  
or die("Không mở được CSDL");
```



## Mở CSDL

---

Ví dụ:

```
<?php
error_reporting(0);
$connect=mysql_connect("localhost","nvhung","123") or die("kết nối không thành công".mysql_error());
mysql_select_db("sinhvien") or die("Không mở được csdl".mysql_error());
// các câu lệnh
mysql_close($connect);
?>
```



## Chọn CSDL

---



Sau khi mở CSDL, ta có thể thay đổi CSDL theo cú pháp:

```
use Database name;
```

- ❖ Để truy vấn đến CSDL ta có thể sử dụng 1 trong 3 cú pháp sau:

```
$sql="câu lệnh sql";
```

```
1. $kq=MySQL_query($sql);
```

```
2. $kq=MySQL_query($sql, [$bien_conn]);
```

```
3. $kq=MySQL_query(database_select,  
$sql, [$bien_conn]);
```

Kết quả trả về: là true hoặc false tương ứng với thành công hoặc thất bại. Cho phép truy vấn CSDL.



## Truy vấn CSDL

---

Ví dụ:

```
<?php
error_reporting(0);
$connect=mysql_connect("localhost","nvhung","123") or die("kết nối không thành công".mysql_error())
mysql_select_db("sinhvien") or die("Không mở được csdl".mysql_error());
$sql="select * from sinhvien";
$kq=mysql_query($sql) or die("không thao tác được với dl".mysql_errno());
?>
```





## Làm việc với các bản ghi

---

- ◆ Lấy số lượng bản ghi khi truy vấn bảng dữ liệu theo cú pháp sau:

```
$sl=MySQL_num_rows($kq);  
if(!$sl) echo"không tìm thấy bản ghi  
nào";  
else echo"số bản ghi thỏa mãn đk".$sl;
```



## Làm việc với các mẫu tin

---



Để lấy từng bản ghi từ kết quả \$kq trả về ta sử dụng hàm MySQL\_fetch\_array() hoặc MySQL\_fetch\_assoc():

```
while ($row=MySQL_fetch_array($kq) )
{
    $gtri1=$row["cột 1"];
    $gtri2=$row["cột 2"];
    ...
}
```



## Làm việc với các mẫu tin

---

- ◆ Ngoài các hàm trên, ta có thể tạo, xóa CSDL hoặc table từ trang PHP thông qua các hàm

```
MySQL_creat_db(database name,  
                [$bien_con]);
```

```
MySQL_drop_db(database name,  
                [$bien_con]);
```

```
drop table    table name;
```



## 7.2 thêm thông tin vào CSDL

---

- ◆ **Đăng ký người dùng**
- ◆ **Xác nhận thông tin**
- ◆ **Lưu thông tin**



## Đăng ký người dùng

---

- ◆ Tạo bảng người dùng gồm các trường cần thiết
- ◆ Tạo trang đăng ký chứa form nhập thông tin với các phần tử tương ứng với các trường.
- ◆ Xây dựng các hàm kiểm tra tính hợp lệ của dữ liệu nhập vào.



## Xác nhận thông tin

---

- ◆ Kết nối CSDL để kiểm tra email đã tồn tại hay chưa?
- ◆ Nếu chưa thì hiển thị toàn bộ thông tin người dùng vừa nhập để đăng ký.
- ◆ Trong trường hợp không chỉnh sửa gì thêm thì chọn lưu thông tin.



## Lưu thông tin vào CSDL

---

- ◆ Mở kết nối CSDL.
- ◆ Lưu các thông tin lấy từ các phần tử form tương ứng vào các trường trong CSDL.
- ◆ Thông báo đăng ký thành công cho người dùng.



## 7.2 Thêm thông tin vào CSDL

---

**Ví dụ:** Đăng ký người dùng vào bảng users từ form đăng ký được tiến hành qua các bước:

- Form đăng ký yêu cầu người dùng nhập thông tin
- Người dùng nhập thông tin và chấp nhận đăng ký.
- Hệ thống kiểm tra tính hợp lệ của các trường nhập vào.  
Kết nối CSDL để kiểm tra email đã tồn tại hay chưa?
  - Nếu email đã tồn tại, yêu cầu về trang trước
  - Nếu chưa tồn tại, thì cho phép về trang trước hay qua trang xác nhận thông tin.





## 7.2 Thêm thông tin vào CSDL

---

- Ở trang xác nhận thông tin, người dùng có thể về trang trước để chỉnh sửa hoặc chọn lưu thông tin.
- Khi người dùng chọn lưu thông tin thì thông tin người dùng đã đăng ký sẽ được lưu vào CSDL và thông báo đăng ký thành công.



## 7.3 Kích hoạt dữ liệu

---

- ◆ **Xem dữ liệu**
- ◆ **Cập nhật dữ liệu**
- ◆ **Xóa dữ liệu**



 Xem dữ liệu

- ❖ Sau khi đăng ký thành công. Người dùng có thể đăng nhập vào hệ thống bằng user và pass đã đăng ký.
- ❖ Người dùng có thể xem select.



## Cập nhật dữ liệu

---



Thông qua câu lệnh update người dùng có thể xem hiệu chỉnh và lưu dữ liệu.



## Xóa dữ liệu

---

- ◆ Để xóa dữ liệu, người dùng cần lựa chọn ra các thông tin phụ thuộc vào điều kiện nào đó.
- ◆ Sau đó chọn bản ghi cần xóa hoặc xóa tất cả.
- ◆ Cần thao tác với checkbox và radio khi thiết kế form xóa bản ghi.